

A Set of Tutorials for the LAMMPS Simulation Package: 2. Materials Science [Article v0.3]

Simon Gravelle¹, Mark A. Tschopp², Jacob R. Gissinger³, Axel Kohlmeyer^{4*}

¹ Université Grenoble Alpes, CNRS, LIPhy, 38000 Grenoble, France; ² Army Research Laboratory, Chicago, USA; ³ Stevens Institute of Technology, Hoboken, NJ 07030, USA; ⁴ Institute for Computational Molecular Science, Temple University, Philadelphia, PA 19122, USA

This is a draft document maintained on GitHub at <https://github.com/lammpstutorials/matsci-tutorials-inputs>; to provide feedback or suggestions, please use the repository issue tracker.

This version dated June 22, 2026

Abstract This set of tutorials introduces molecular simulations of *materials* with the LAMMPS simulation package, with an emphasis on crystalline metals, lattice defects, and mechanical behavior. A connected sequence of four tutorials covers an embedded-atom-method (EAM) potential framework, automation and parameter scans, uniaxial deformation of single crystals, and grain-boundary construction and fracture. Further tutorials address nanoindentation, elastic constants, stacking fault energies, thermal conductivity, phonon dispersions, zone-center vibrational frequencies, radiation damage cascades, melting point determination, diffusion in a liquid metal, and chemical short-range order in a high-entropy alloy. Like the companion LiveCoMS article with soft-matter tutorials, these tutorials are built around LAMMPS–GUI and use current LAMMPS syntax and features, with Python scripts for additional analysis and visualization. The first four tutorials are a modernized recreation of the long-standing MSU/CAVS LAMMPS tutorials; the remaining tutorials are original or adapted from LAMMPS example scripts.

***For correspondence:**
akohlmey@gmail.com (A.K.)

1 Introduction

Molecular dynamics (MD) is widely used in materials science to study the structure, thermodynamics, and mechanical properties of metals and other crystalline solids. LAMMPS [1–3] is a flexible, open-source MD code well suited to such simulations. It has broad support for many-body interatomic potentials commonly used in materials-science simulations, and its input file syntax supports the sometimes complex workflows for it directly. More recently, machine-learning interatomic potentials (MLIPs) [4, 5] have become increasingly popular in this field since they promise accuracy comparable to quantum calculations at a fraction of the computational

cost. LAMMPS supports a variety of such potentials and apart from small differences in the input file syntax to enable such potentials and requiring specific parameter files, the simulation workflows remain essentially the same as those presented here for empirical models.

This document is a companion to the LiveCoMS tutorial article “A Set of Tutorials for the LAMMPS Simulation Package: 1. Soft-Matter” [6], which focuses on soft-matter and chemistry applications. Here we cover several materials-science topics — crystalline metals, lattice defects, and mechanical behavior — in a similar style and with the same tooling as the companion article, using LAMMPS–GUI [7] as input file editor and

front end and supported by the graphics and on-the-fly analysis capabilities in LAMMPS [8].

Tutorials 1–4 are modernized recreations of the content from five of the eight MSU/CAVS LAMMPS tutorials originally conceived and maintained by Mark A. Tschoop and collaborators [9, 10]. Tutorials 5 and beyond are either original tutorials conceived for this article or adapted from commented example scripts bundled with LAMMPS [11] and additional information in the LAMMPS documentation; the primary source material is identified at the beginning of each tutorial.

A recurring theme in atomistic materials modeling is that results — when converged for size effects and statistical fluctuations — are only as good as the *interatomic potential*: the same simulation protocol applied with different potentials can give noticeably different lattice constants, elastic moduli, defect energies, and strengths. These tutorials therefore pay attention to where a potential comes from, how it is invoked, and how the computed properties compare with experiment. They are deliberately built around metals described by the embedded-atom method [12], long the workhorse of metallic simulations, while also showing how potentials of different kinds can be combined. Two appendices set these classical potentials between the extremes of model complexity: Appendix D shows why the simpler pairwise potentials that are sometimes applied to metals fail on the very properties these tutorials compute, and Appendix E extends the comparison upward to an MLIP — which fits into exactly the same workflows — quantifying its accuracy and computational cost against the classical potentials used here.

The first four tutorials build on one another, progressing from a defect-free crystal to increasingly complex microstructures and processes: first a perfect single crystal and its cohesive properties, then automated parameter scans, then the mechanical response and dislocation activity of a strained crystal, and finally the construction of grain boundaries and their fracture. The remaining tutorials address a range of materials-modeling topics that can be followed in any order after completing the first four. Throughout, the workflow follows the companion article: inputs are written and run in LAMMPS-GUI, snapshots are produced with the built-in LAMMPS renderer, and quantitative results are post-processed with short Python scripts. Every figure in this document is generated from the shipped inputs, and each input is meant to be opened, run, and modified by the reader as instructed during the tutorial.

1.1 Scope

Readers who are new to LAMMPS are advised to first work through Tutorial 1 of the companion article [6], which introduces LAMMPS and LAMMPS-GUI by building up an input

file step by step. The tutorials here take a complementary approach: each tutorial ships a complete, fully working input that can be opened and run immediately and is then studied, discussed, and modified in detail.

Tutorials 1–3 form the essential core: they introduce the EAM potential framework, LAMMPS scripting and automation, and the simulation of mechanical deformation. After completing these three tutorials readers will have the background needed to follow any of the remaining tutorials in the order that suits their interests.

In Tutorial 1, a face-centered cubic metal is constructed, an EAM potential is applied, and the equilibrium lattice constant and cohesive energy of the crystal are obtained by energy minimization.

In Tutorial 2, the use of variables, loops, and command-line arguments in LAMMPS is demonstrated to scan the lattice parameter of a metal crystal, plot its energy–volume curve, and fit it to an equation of state to determine the equilibrium lattice constant and bulk modulus.

In Tutorial 3, a single crystal is equilibrated and then subjected to tensile and compressive strain, the stress–strain response is extracted, and dislocation nucleation is visualized.

In Tutorial 4, grain boundaries are introduced: first a $\Sigma 5(310)$ symmetric tilt boundary in fcc aluminum is constructed and its energy computed, then a $\Sigma 3(111)$ boundary in bcc iron is strained to fracture and the per-atom stress is analyzed.

In Tutorial 5, an aluminum surface is indented with rigid objects and a shaped tool is stamped into the surface using a hybrid potential.

In Tutorial 6, the full elastic stiffness tensor of crystalline silicon is computed from small-strain deformations using the Stillinger–Weber potential.

In Tutorial 7, the generalized stacking fault energy (γ -surface) of fcc aluminum is traced along the $\langle 112 \rangle$ direction in the (111) plane, explaining dislocation glide and partial splitting.

In Tutorial 8, the thermal conductivity of silicon is computed using reverse non-equilibrium molecular dynamics (NEMD), and its finite-size dependence is examined.

In Tutorial 9, the full phonon dispersion of fcc copper is obtained by accumulating displacement correlations during a finite-temperature MD run using `fix phonon`.

In Tutorial 10, the zone-center phonon frequencies of silicon are computed at zero temperature from the dynamical matrix, and their connection to elastic constants is discussed.

In Tutorial 11, a primary knock-on atom (PKA) displacement cascade is simulated in bcc iron, and the resulting Frenkel-pair defects are counted on the fly with the `compute frenkel` command.

In Tutorial 12, the melting temperature of fcc aluminum is determined first by direct NPT heating and then by the more rigorous two-phase coexistence method.

In Tutorial 13, the self-diffusion coefficient of liquid aluminum is measured from the mean-squared displacement, and its temperature dependence is fitted to an Arrhenius law to obtain an activation energy.

In Tutorial 14, the chemical short-range order of the high-entropy alloy CoCrFeNi is equilibrated with a hybrid molecular dynamics / Monte Carlo simulation and quantified with the Warren–Cowley parameters using three different published potentials.

These tutorials are intended to be *educational*. To keep run times short enough to follow them interactively on a desktop or laptop computer, the simulated systems are deliberately chosen to be small and thus the simulations brief, so the inputs should be regarded as illustrative starting points rather than production-ready calculations. For any research application, it is good practice to verify that results are converged with respect to the system size and the length of the simulation — and, where relevant, the time step, strain rate, and similar parameters. The accuracy genuinely required by a problem should not be traded away for convenience; when a well-converged calculation becomes too demanding for a desktop computer, it can be carried out on one of the high-performance computing (HPC) clusters that are widely available to researchers today.

2 Prerequisites

2.1 Background knowledge

Like its companion article [6], this set of tutorials assumes no prior knowledge of the LAMMPS software itself. To complete the tutorials, a text editor and a suitable LAMMPS executable are required (see below). We use LAMMPS–GUI [7] here, as it offers many features that make it particularly convenient for tutorials, but other console or graphical text editors, such as GNU nano, vi/vim, Emacs, Notepad, Gedit, and Visual Studio Code, can also be used. LAMMPS can be executed either directly from LAMMPS–GUI (see Appendix A) or from a command prompt (see Appendix B), the latter of which requires some familiarity with executing commands from a terminal or command-line prompt.

In addition, some knowledge of the basic concepts of molecular dynamics, crystallography (e.g. lattice systems, Miller indices), and mechanics (stress and strain) is expected. We also recommend studying at least Tutorial 1 of the companion article [6] before starting these tutorials.

2.2 Software requirements

Some of the tutorials rely on features that were added to LAMMPS and LAMMPS–GUI relatively recently, so a LAMMPS [11] release of 23 June 2026 or later and LAMMPS–GUI [7] version 2.1 or later are recommended, as these are the versions that the tutorials were tested and validated with. As in the companion article, LAMMPS–GUI is used to edit inputs, run LAMMPS, render snapshots, and display plots of thermodynamic data or user-defined quantities. For additional analysis and visualization, Python 3 with packages NumPy [13], Matplotlib [14], pandas [15], SciPy [16], and PyYAML [17] is used. For interactive three-dimensional visualization and analysis (in particular for dislocation and defect analysis in Tutorials 3 and 11), OVITO (Open Visualization Tool) [18] is recommended. Tutorial 9 also uses the standalone `phana` post-processing tool, which is distributed with LAMMPS.

2.3 About LAMMPS–GUI

LAMMPS–GUI is a graphical text editor, enhanced for editing LAMMPS input files and linked to the LAMMPS library, allowing it to run LAMMPS directly. The text editor is similar to other graphical editors, such as Notepad or Gedit, but offers the following enhancements for running LAMMPS:

- Wizard dialogs to set up these and other tutorials
- Auto-completion of LAMMPS commands and options
- Context-sensitive online help for LAMMPS commands
- Syntax highlighting for LAMMPS input files
- Syntax-aware line indentation
- Automatic change of working directory to input location
- Visualization using LAMMPS’ built-in renderer
- Start and stop simulations via mouse or keyboard
- Monitor simulation progress and parallelization
- Live capture of log output in a persistent text window
- Automatic plotting of thermodynamic data during runs
- Capture of “dump image” outputs for animations
- Viewing of images created by LAMMPS in batch or parallel runs or created by other tools
- Export of thermodynamic data for external plotting
- Import and plotting of column-oriented data
- Inspection of binary restart files
- Post-processing of plot data: curve, equation-of-state, and custom fits

Appendix A contains basic instructions for using LAMMPS–GUI with the tutorials presented here. A complete description of all LAMMPS–GUI features can be found in the online documentation [7].

For reproducibility and publication-quality figures, the analysis in these tutorials is carried out with the provided Python scripts, and we recommend them as the reference

workflow. However, LAMMPS-GUI has grown several built-in conveniences — plotting external data files, fitting them (including to an equation of state), and viewing images that LAMMPS rendered in a separate run — that handle many of these steps without leaving the editor or installing extra tools. They are well suited to quick checks while developing an input, and are summarized in Appendix A.

3 Content and links

All of the input files, potentials, and helper scripts referenced below are provided in a GitHub repository (<https://github.com/lammpstutorials/matsci-tutorials-inputs>) organized as one folder per tutorial. Each tutorial folder contains a small `.manifest` file listing its files; the first listed file is the one a reader normally opens first, and a `solution/` subfolder holds reference outputs (logs, plots, and rendered figures) that are fetched only on request. Downloadable files are highlighted in the text in blue. The tutorial files are also available directly from the «Tutorials» menu of LAMMPS-GUI, which downloads each tutorial's files and opens the first one ready to edit and run. In the following, all LAMMPS input or console commands are formatted with a colored background. File and folder names are formatted in monospace, and LAMMPS-GUI options and menus are displayed in «quoted monospace». Paragraphs describing simulations that require more than a few minutes of computer time on a typical laptop or desktop computer are marked with an hourglass symbol (⌚). Many calculations can be accelerated by configuring multi-threading in the LAMMPS-GUI preferences.

3.1 Tutorial 1: Crystalline metals and the EAM potential

The objective of this tutorial is to set up a crystalline metal in LAMMPS and characterize it at zero temperature. A face-centered cubic (fcc) aluminum crystal is created (Fig. 1), an embedded-atom-method (EAM) potential [12] is applied, and the equilibrium lattice constant and cohesive energy are obtained by relaxing the simulation box with energy minimization. Compared with the Lennard-Jones fluid of the companion soft-matter tutorials [6], the two new ingredients here are the use of a periodic crystal *lattice* to place atoms and the use of a tabulated many-body *metal* potential read from a file.

3.1.1 Setting up an fcc metal

To set up this tutorial, select «Tutorial 1» under «Materials Science» in the «Tutorials» menu of LAMMPS-GUI, and follow the instructions. This will select (or create, if needed) a folder, place the initial input file `initial.lmp` in it, and open the file in the LAMMPS-GUI

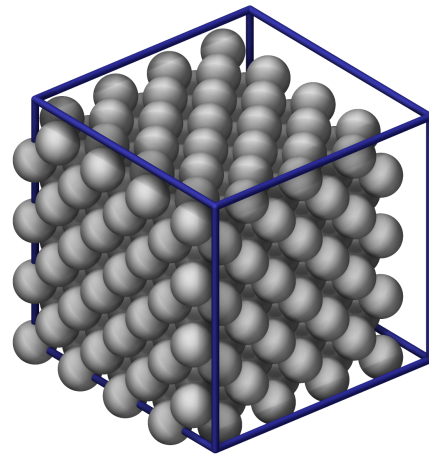


Figure 1. The $4 \times 4 \times 4$ face-centered cubic aluminum supercell built in Tutorial 1 (256 atoms). The image was generated with the built-in LAMMPS renderer, as used by the LAMMPS-GUI «Image Viewer».

«Editor» window. Its first two sections, Initialization and System definition, read:

```
# 1) Initialization
units metal
dimension 3
boundary p p p
atom_style atomic

# 2) System definition
lattice fcc 4.049
region box block 0 4 0 4 0 4
create_box 1 box
create_atoms 1 box
```

The command `units metal` selects the *metal* unit system, in which distances are in Ångströms (Å), energies in electronvolts (eV), and time in picoseconds (ps) — the natural choice for atomistic simulations of metals.

The `boundary p p p` command applies periodic boundary conditions in all three directions (the default setting), and `atom_style atomic` represents each atom as a simple point mass without bonds or other additional per-atom properties (also the default). The `lattice fcc 4.049` command defines a face-centered cubic lattice with a *lattice constant* of 4.049 Å [19], the experimental value for aluminum. Unlike in Tutorial 2 and beyond, here the `lattice` command does more than set a density: it provides the template of atom positions used by `create_atoms`. The `region box block 0 4 0 4 0 4` command defines a cubic region spanning four lattice spacings in each direction, `create_box 1 box` creates a simulation box for one atom type of the dimensions of the `box` region, and `create_atoms 1 box` fills it with type-1 atoms on the fcc lattice sites. This produces $4^3 \times 4 = 256$ atoms, since the conventional fcc cell contains four atoms.

By default, LAMMPS uses reduced units (`units lj`). The choice of unit system is often dictated by the interatomic potential: a potential file's parameters are expressed in particular units, and the `units` setting must match them. The potential files bundled with LAMMPS record their units in a `UNITS:` tag in the header comments, which lets LAMMPS verify compatibility and stop with an error on a mismatch. For a few pair styles, including `eam`, `sw`, and `tersoff`, LAMMPS can convert automatically between `metal` and `real` units, so a potential written for one set of units can be used with the other (`solution/initial-real.lmp` is an example of such an input). This convenience *relies* on the `UNITS:` tag; if it is missing, LAMMPS cannot detect a mismatched `units` setting and uses the file's numbers as-is, thus silently producing incorrect results.

The numbers after the `box` keyword in the `region` command are interpreted in lattice units by default, so the box size automatically follows the lattice constant. Appending `units box` would instead interpret the bounds as multiples of the current length units, that is in Ångströms for `units metal`. Choosing lattice units makes it easy to change the system size and density by editing a single number.

3.1.2 Applying the EAM potential

Metals are not well described by simple pairwise potentials because the strength of the atomic interaction in a metal depends on the local coordination. The embedded-atom method (EAM) [12] captures this many-body effect by adding, to a pairwise term, an *embedding energy* that is a non-linear function of the local electron density contributed by neighboring atoms. (Appendix D quantifies the difference between the EAM method and pairwise potentials). The parameters for the EAM potential are tabulated in a potential file; here we use the aluminum potential of Zhou *et al.* [20], distributed with LAMMPS as `Al_zhou.eam.alloy` in the `setf1 (eam/alloy)` format. The `Settings`, `Monitoring`, and `Run` categories of `initial.lmp` are:

```
# 3) Settings
pair_style eam/alloy
pair_coeff * * Al_zhou.eam.alloy Al

# 4) Monitoring
compute eng all pe/atom
compute eatoms all reduce sum c_eng
thermo 10
thermo_style custom step pe c_eatoms press lx ly lz
```

```
# 5) Run
run 0
```

The `pair_coeff * * Al_zhou.eam.alloy Al` command maps atom type 1 to the entry “Al” in the potential file. Many-body potentials like pair style `eam/alloy` allow only a single `pair_coeff` command. Since the potential cannot be decomposed into independent pairs of atom types, the wild-card pattern “* *” sets it for all type combinations at once, and the many-body interaction is evaluated internally over all tuples of types, as the model requires. What *is* required is that each atom type be matched to an entry in the potential file (typically an element symbol); LAMMPS checks that the list of names following the file name has exactly one entry per atom type. Multiple atom types may be mapped to the same potential entry, but *all* atom types have to be mapped. For example, if the box was created with two atom types and both of them should be aluminum, the command would become `pair_coeff * * Al_zhou.eam.alloy Al Al`. The two `compute` commands evaluate the per-atom potential energy and sum it over all atoms; this is used here only to illustrate per-atom `compute` styles, which become essential in Tutorial 4. Finally, `run 0` performs a single force and energy evaluation without advancing the atoms.

No `mass` command is needed here: the EAM potential files contain the atomic mass, which LAMMPS assigns automatically. This is specific to potentials that store the mass; with most other pair styles, a `mass` command is required or else LAMMPS will abort with an error since it is not possible to solve Newton's equations of motion for massless objects.

Running `initial.lmp` reports a total potential energy of about -915.75 eV (i.e., -3.577 eV per atom) and, notably, a large positive pressure of about 23 600 bar. The non-zero pressure shows that the guessed lattice constant of 4.049 Å is *not* the equilibrium value for this potential: the crystal is slightly compressed and “pushes outward”. We determine the true equilibrium lattice constant next.

3.1.3 Energy minimization and the lattice constant

To find the equilibrium lattice constant, we let LAMMPS adjust the box size until the pressure vanishes, while simultaneously minimizing the potential energy. Open `minimize.lmp`, which is identical to `initial.lmp` up to the `Run` category (with the thermo output extended to show detailed pressure component information), where `run 0` is replaced by:

```
fix relax all box/relax iso 0.0 vmax 0.001
min_style cg
minimize 1.0e-12 1.0e-12 10000 100000
```

The images of crystals in this tutorial were made with the renderer built into LAMMPS, and you can reproduce them quasi-interactively in LAMMPS-GUI. After reading the input file, open the «Image Viewer». The most convenient way is to click the snapshot icon () in the status bar at the bottom-left of the editor window; the same action is also available as «Create Image» in the «Run» menu. The most important settings are available as buttons in the toolbar. More details and advanced settings are available from the «Global», «Atoms/Bonds», and «Colors» dialogs. The equivalent scripted commands are `dump image` and `dump_modify`, as used in `solution/render.lmp`. The commands for the currently shown visualization can be copied to the clipboard using «Copy dump image command» in the «File» menu, and then pasted into an input file and further adjusted. For more detail, see the LAMMPS-GUI documentation [7], the `dump image` command documentation [21], and the LAMMPS visualization how-to [8].

The `fix box/relax iso 0.0` command allows the simulation box to change size isotropically (the same scaling in x , y , and z) so as to drive the pressure toward the target value of 0 bar. The `minimize` command then relaxes both the atomic positions and the box dimensions using the conjugate-gradient algorithm [23], as requested by the `min_style cg` command, until the energy and force tolerances are met. For this perfect crystal the atoms remain on their lattice sites by symmetry, so only the box size changes.

A few `variable` and `print` commands at the end report the results in the log file:

```
variable ncell equal 4
variable a0 equal lx/v_ncell
variable ecoh equal pe/count(all)
print "-----"
print "Number of atoms = $(count(all))"
print "Equilibrium a0 (Ang) = ${a0}"
print "Cohesive energy (eV) = ${ecoh}"
```

Table 1. Equilibrium lattice constant and cohesive energy (per atom) of fcc aluminum computed with three interatomic potentials, compared with experiment. All three reproduce the lattice constant to within about a percent, whereas the cohesive energy is more sensitive to the choice of potential.

Potential	a_0 (Å)	E_{coh} (eV)
EAM, Zhou (<code>eam/alloy</code>)	4.082	-3.580
EAM, Mendeleev (<code>eam/fs</code>)	4.045	-3.411
MEAM, Jelinek <i>et al.</i> (<code>meam</code>)	4.050	-3.353
Experiment [19, 22]	4.049	-3.36

By default, the cubic axes of the box are aligned with the crystallographic $\langle 100 \rangle$ directions. Other orientations can be selected with the `orient` keywords of the `lattice` command, for example `lattice fcc 4.049 orient x 1 1 0 orient y -1 1 0 orient z 0 0 1` for a $\langle 110 \rangle$ -oriented crystal. The orientation vectors must be mutually orthogonal. When using periodic boundary conditions and creating a system from atoms on lattice positions, special care must be taken when choosing the number of lattice spacings used for the box region dimensions so that the cell remains periodic and no unintentional grain boundary is created. The choice of crystal orientation plays an important role in the deformation tutorial (Tutorial 3), where the loading direction matters.

```
print "-----"
```

The pressure drops from 23 600 bar to essentially zero, the box expands, and we obtain an equilibrium lattice constant $a_0 = 4.082$ Å and a cohesive energy $E_{\text{coh}} = -3.580$ eV per atom. These values depend on the choice of pair style and potential file.

As a demonstration, we have repeated the minimization calculation with two other aluminum potentials that are bundled with LAMMPS, the Mendeleev `eam/fs` potential (`Al_mm.eam.fs`) and the `meam` potential of Jelinek *et al.* [24] (`AlSiMgCuFe.meam`). Each potential is fitted to a different set of target properties, so it is expected that the results differ by amounts comparable to the model's error margin. Note that for a single element, the pair style `eam/fs` uses the exact same embedded-atom model as `eam` or `eam/alloy`; it differs only in the file format of the potential file and how the potential for mixed-element interactions is obtained, so the difference between those two runs reflects the differences in the parameterization alone; the modified EAM (`meam`) potential is a more elaborate model and thus can reproduce the experimental values more closely.

Table 1 collects these values and compares them to the experimental reference for aluminum. For your reference, the corresponding inputs, logs, and potential files for this comparison are provided in the `solution` folder. The important lesson from this experiment is that the ability of a potential to reproduce properties relevant to a given application should always be verified to ensure that it is adequate for the intended use.

3.1.4 Atom placement and the simulation box

Atoms in the crystal constructed so far (Fig. 1, Fig. 2a) appear to protrude from one face of the box while the opposite face

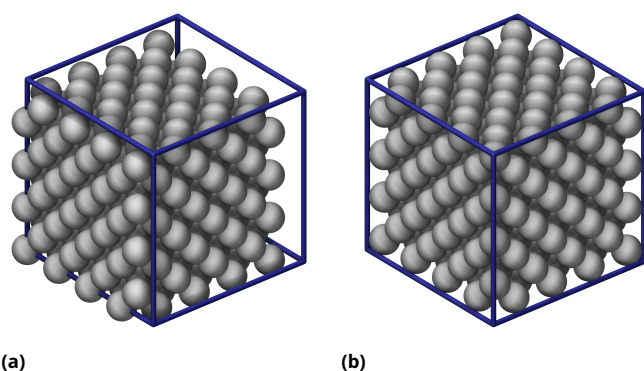


Figure 2. The same fcc aluminum crystal, simply translated within the periodic box. (a) With the default lattice origin, atoms lie on the box faces, so they appear to stick out of one side and to be missing from the opposite side. (b) After shifting the lattice origin by a quarter spacing (`origin 0.25 0.25 0.25`), all atoms lie inside the box with symmetric margins. Because the two configurations differ only by a rigid translation, they are physically identical under periodic boundary conditions.

appears bare. This happens because the box boundaries fall *exactly* on lattice planes, so atoms lie precisely on the faces of the simulation box. LAMMPS resolves the resulting periodic-image ambiguity for this simple case (it correctly creates 256 atoms), but placing atoms exactly on a periodic boundary is bad practice in general: floating-point rounding can cause an atom and its periodic image to both be retained — creating two atoms on top of each other — or to be erroneously discarded. These errors are easy to trigger in the more complex cells of later tutorials.

The remedy is to ensure that no atoms sit on the box faces by shifting the lattice relative to the box. Note that simply moving the region from `0 4` to `0.5 4.5` is *not* sufficient for an fcc lattice: because fcc has lattice planes every half spacing, the face-centered atoms would then land on the new boundary. Instead, we shift the lattice *origin* by a quarter spacing using the `origin` keyword. Open `shifted.lmp`, which is identical to `minimize.lmp` except for the `lattice` command:

```
lattice fcc 4.049 origin 0.25 0.25 0.25
```

With this single change, all atoms lie strictly inside the box with a symmetric margin on every face (Fig. 2b).

Running `shifted.lmp` yields exactly the same equilibrium lattice constant ($a_0 = 4.082 \text{ \AA}$) and cohesive energy (-3.580 eV per atom) as `minimize.lmp`, down to the last digit. This is a useful sanity check: a uniform translation of all atoms in a periodic box must leave every physical property unchanged. Whenever a system is sensitive to such a translation, it is a sign that atoms are interacting with their own periodic images incorrectly — often because they sit on a boundary.

3.2 Tutorial 2: Variables, automation, and the energy–volume curve

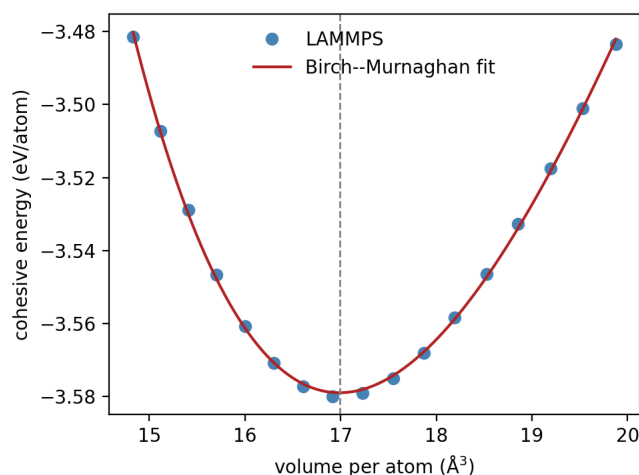


Figure 3. Energy–volume curve of fcc aluminum from Tutorial 2: cohesive energy versus volume per atom for the Zhou EAM potential (points), with a third-order Birch–Murnaghan fit (line). The dashed line marks the equilibrium volume.

The objective of this tutorial is to automate a series of LAMMPS runs and to extract a material property from them. Building on Tutorial 1, we make the lattice constant of fcc aluminum an adjustable *variable*, scan it over a range of values, and record the cohesive energy at each one. The resulting energy–volume curve (Fig. 3) is then fitted to obtain the equilibrium lattice constant and the bulk modulus.

3.2.1 Index and equal-style variables

Open the file `energy.lmp`. It resembles the input from Tutorial 1, except that the lattice constant is no longer hard-coded. Its first line defines a *variable*,

```
variable a0 index 4.049
```

and the `lattice` command refers to it through the `{a0}` variable expansion syntax:

```
lattice fcc {a0} origin 0.25 0.25 0.25
```

Variables of the `index` style hold a fixed value (or a list of values). The definition in the input file acts as a default: if the variable is set before that line is reached— via the «Set Variables» dialog of LAMMPS–GUI, the `-var` flag on the command line, e.g. `lmp -in energy.lmp -var a0 4.10`, or an earlier `variable` command— the new definition is silently skipped.

After the `run 0` command, three *equal*-style variables are defined that compute derived quantities:

You can refer to variables in LAMMPS in two different ways: as an immediate expansion (`{a0}`) or as a variable reference (`v_a0`). In the case of immediate expansion, the substitution of the variable with its value happens immediately when the line of the input file is parsed and it is performed by the input file processor. Consequently, the `lattice` command does not process the expression as a variable; instead, it receives the evaluated numerical value (4.049).

Variable references behave differently: the corresponding command must explicitly support their interpretation. For example, `lattice fcc v_a0 origin 0.25 0.25 0.25` results in an error, since the text `v_a0` is not a numeric value and the `lattice` command *requires* a number as the lattice constant.

```
variable natoms equal count(all)
variable vat equal vol/v_natoms
variable ecoh equal pe/v_natoms
print "EV {a0} ${vat} ${ecoh}"
```

Unlike an index variable, an equal-style variable is re-evaluated every time it is used, and within the expression on the right you can refer to the *current* value with a variable reference (here `v_natoms`). It always reflects the current state of the system. The final `print` command then writes a line of the form `EV 4.049 16.60 -3.577`, which reports the lattice constant (Å), the volume per atom (Å³), and the cohesive energy (eV).

3.2.2 Scanning the lattice parameter

Setting the lattice constant by hand is convenient for a few values, but a systematic scan is better automated. Rather than executing LAMMPS multiple times, we let LAMMPS perform the loop itself, so that the entire scan can run inside LAMMPS-GUI with a single input file.

The input `scan.lmp` stores the list of lattice constants in an `index` style variable and iterates over them with the `next` and `jump` commands. The `next` command will change the current value of the index style variable to the next value in the list. If there is no more value left, the variable will be deleted and the *next* line in the input file is ignored (thus breaking the loop by skipping the `jump` command). The `jump` command in turn will tell LAMMPS to rewind the input to the beginning and skip over lines until it reaches a `label` command with the same keyword as used in `jump`.

The `clear` command resets the system and deletes the simulation cell and atoms and so on (except for variables which retain their values) at the start of each iteration:

```
variable a index 3.90 3.925 3.95 ... 4.30
label loop
clear
# ... build the fcc cell at lattice constant ${a}, then run 0 ...
variable vat equal vol/count(all)
variable ecoh equal pe/count(all)
print "- {lattice_constant: ${a}, volume_per_atom: $(v_vat:%.6f), &
      cohesive_energy: $(v_ecoh:%.6f)}" append ev.yaml screen no
next a
jump SELF loop
```

The ampersand (`&`) is a line-continuation marker: when it is the *last* character in a line, LAMMPS joins that line with the next one before parsing the command. This way you can spread long command-lines across multiple lines.

At each step, the `print` command appends one record to a *structured* data file, `ev.yaml`, containing the lattice constant, the volume per atom, and the cohesive energy. Writing the results in a standard format such as YAML (or JSON, or CSV) [25] avoids the fragile log-file scraping: such files can be read back *without* writing a custom parser — both by spreadsheet programs and, as we do next, by Python through the `pandas` library [15].

For purely tabular output like this, CSV is an equally convenient choice and imports directly into spreadsheet programs, whereas YAML and JSON additionally accommodate nested data and metadata. The LAMMPS structured-data how-to [25] describes how to emit each of these formats, including the native YAML thermodynamic output produced by `thermo_style yaml`. Since version v2.1, LAMMPS-GUI can also read and plot files in any of these formats directly, and offers post-processing options such as the equation-of-state fit used in the next task.

3.2.3 Plotting the energy-volume curve

Finally, the Python script `plot.py` loads `ev.yaml` using `pandas` and `PyYAML`, plots the cohesive energy against the volume per atom (Fig. 3), and fits a third-order Birch–Murnaghan equation of state [26] to the data:

$$E(V) = E_0 + \frac{9V_0B_0}{16} \left\{ \left[(V_0/V)^{2/3} - 1 \right]^3 B'_0 + \left[(V_0/V)^{2/3} - 1 \right]^2 \left[6 - 4(V_0/V)^{2/3} \right] \right\}, \quad (1)$$

where V_0 is the equilibrium volume, E_0 the minimum energy, B_0 the bulk modulus, and B'_0 its pressure derivative. The fit gives the equilibrium volume per atom, and hence the lattice constant $a_0 = (4V_0)^{1/3}$ for the four-atom fcc cell, together with the bulk modulus $B_0 = V(d^2E/dV^2)$. For the Zhou potential we obtain $a_0 = 4.08$ Å, $E_0 = -3.58$ eV per atom, and $B_0 = 87$ GPa.

The lattice constant matches the value from the box relaxation in Tutorial 1, a reassuring consistency check

between two independent methods. The computed bulk modulus is somewhat larger than the experimental value of about 76 GPa [27], again reflecting the choice of potential.

The same equation-of-state fit can be done without leaving LAMMPS-GUI. Open `ev.yaml` with «Plot Data File...», choose the volume-per-atom column for x and the cohesive energy for y , and run the «Birch-Murnaghan EOS» fit from the chart's «Postprocess...» dialog (entering four atoms per conventional fcc cell). It reports the same a_0 , E_0 , B_0 , and B'_0 as the Python script, which is a quick way to check the curve before producing the final figure (Appendix A).

3.3 Tutorial 3: Uniaxial deformation of a single crystal

The objective of this tutorial is to deform a single crystal and measure its mechanical response. A $\langle 100 \rangle$ -oriented fcc aluminum crystal is equilibrated at finite temperature and then strained uniaxially at a constant engineering strain rate, while the transverse directions are held at zero pressure. The stress-strain response is recorded (Fig. 4) for both tension and compression, and the onset of plasticity through dislocation nucleation is visualized with a per-atom structure metric (Fig. 6). All of this is driven by a single input, `deform.lmp`; the loading mode (tension or compression) is selected via the «Set Variables» dialog or with `-var mode compress` on the command line.

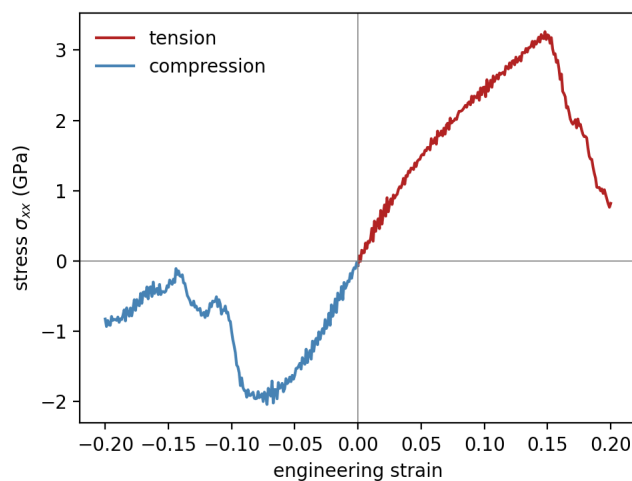


Figure 4. Stress-strain response of $\langle 100 \rangle$ single-crystal aluminum in Tutorial 3, for tension (positive strain) and compression (negative strain) at 300 K. Each curve rises elastically to a peak at which dislocations nucleate, then drops as the crystal yields. Tension and compression are not symmetric: here yielding occurs near 3.2 GPa in tension but only 2.0 GPa in compression.

3.3.1 Equilibrating the crystal

The first part of `deform.lmp` builds a $\langle 100 \rangle$ -oriented fcc aluminum crystal of $10 \times 10 \times 10$ unit cells (4000 atoms) and equilibrates it at 300 K and zero pressure:

```
lattice fcc ${a0} orient x 1 0 0 orient y 0 1 0 orient z 0 0 1 &
  origin 0.25 0.25 0.25
region box block 0 10 0 10 0 10
create_box 1 box
create_atoms 1 box
pair_style eam/alloy
pair_coeff * * Al_zhou.eam.alloy Al
velocity all create ${temp} 12345 mom yes rot no
fix equi all npt temp ${temp} ${temp} 0.1 iso 0.0 0.0 1.0
run ${nequil}
```

The `orient` keywords align the cube axes with the crystallographic $\langle 100 \rangle$ directions, so the loading axis is well defined. The `velocity` command assigns initial velocities for the target temperature, and `fix npt` relaxes both the temperature and the (isotropic) pressure during a 20 ps equilibration. The temperature is an index variable `temp`, so it can be changed without editing the script.

This is the first tutorial in this set whose simulations take more than a few seconds, so it is a good point to discuss performance, parallelization, and GPU acceleration.

Molecular dynamics simulations of larger systems and with a larger number of simulation steps can take a considerable amount of time when run using a single CPU core. This time can be reduced by parallelization, GPU acceleration, or both. LAMMPS offers two complementary kinds of parallelization: distributed-memory parallelism with *MPI*, which splits the simulation box across many CPU cores or compute nodes, and shared-memory *threading* within a node (the *OPENMP* and *KOKKOS* packages); it can also offload work to *GPUs* (the *GPU* and *KOKKOS* packages). Because LAMMPS-GUI runs LAMMPS as a library inside a single process, it does *not* use *MPI*; instead it speeds up runs with threads (*OPENMP* or *KOKKOS*, depending on how LAMMPS was configured and built) and, where available, with GPU acceleration via the *GPU* package. These options are chosen in the «Preferences» dialog under the «Accelerators» tab. For large production runs, the command-line LAMMPS executable with *MPI* — optionally combined with threads or *GPUs* — is usually fastest (Appendix B).

While a simulation runs in LAMMPS-GUI, a progress bar in the status bar at the bottom of the editor window shows how far it has advanced, and a marker in the editor margin highlights the input line currently being executed (Fig. 5). Together they let you follow the progress of a long run at a glance.

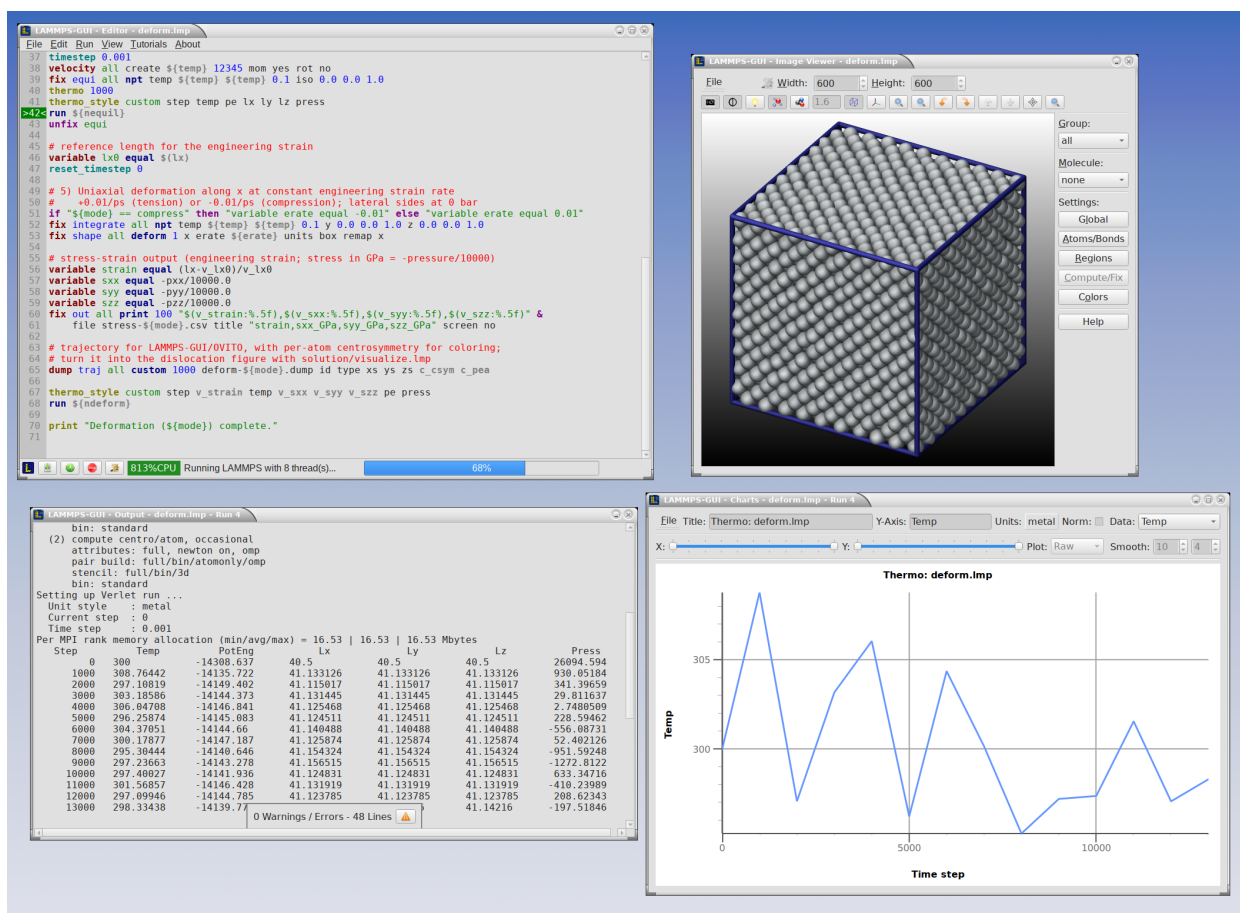


Figure 5. LAMMPS-GUI running the `deform.lmp` simulation of Tutorial 3 in flight. Clockwise from top left: the «Editor», where a marker in the margin highlights the input line currently being executed and the status bar shows a progress bar and that the (equilibration) run is using 8 threads and is 68% complete; the «Image Viewer» showing the equilibrating crystal; the «Charts» window plotting a thermodynamic quantity (here the temperature) live as the run proceeds; and the «Output» window with the running thermodynamic log. All of these windows update continuously while the simulation runs.

3.3.2 Applying tensile and compressive strain

After equilibration, the box length is stored as a reference, and the crystal is strained along x at a constant engineering strain rate of 0.01 ps^{-1} (10^{10} s^{-1}). A single `if` command selects the sign of the strain rate from the loading mode, so the same input performs both tension and compression:

```
variable lx0 equal $(lx)
if "${mode}" == compress then &
    "variable erate equal -0.01" else &
    "variable erate equal 0.01"
fix integrate all npt temp $(temp) $(temp) 0.1 &
    y 0.0 0.0 1.0 z 0.0 0.0 1.0
fix shape all deform 1 x erate ${erate} units box remap x
```

Here `fix deform` rescales the box along x at the chosen rate (`remap x` moves the atoms affinely with the box), while `fix npt` thermostats the system and keeps the lateral directions (y and z) at zero pressure, so the sample is free to contract or expand transversely.

3.3.3 Stress-strain response

The engineering strain and the stress (in GPa, since the metal-units pressure is in bars) are computed as equal-style variables and written to a structured CSV file every 100 steps:

```
variable strain equal (lx-v_lx0)/v_lx0
variable sxx equal -pxx/10000.0
fix out all print 100 "${v_strain:%.5f},${v_sxx:%.5f},..." &
    file stress-${mode}.csv title "strain,sxx_GPa,..." screen no
```

Running `deform.lmp` once for each mode produces `stress-tension.csv` and `stress-compress.csv`, which `plot.py` reads them with `pandas` and plots them together (Fig. 4). Both curves rise linearly in the elastic regime and then drop sharply when dislocations nucleate and the crystal yields. The peak (yield) stress differs between the two modes — about 3.2 GPa in tension versus 2.0 GPa in compression — a manifestation of the tension-compression asymmetry of dislocation nucleation in fcc metals [28].

The yield stresses obtained here are very high because

a perfect, defect-free crystal can only deform once dislocations nucleate *homogeneously*, and because the imposed strain rate (10^{10} s^{-1}) and small system size are far from experimental conditions. Real crystals yield at much lower stresses through pre-existing dislocations. Lower strain rates, larger systems, and different temperatures all shift the response; these are instructive parameters to vary [28].

3.3.4 Visualizing dislocation nucleation

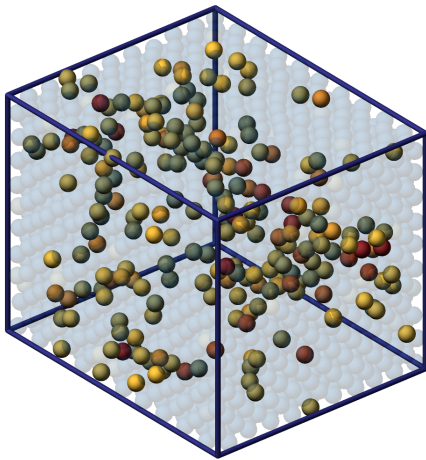


Figure 6. Onset of plasticity in $\langle 100 \rangle$ aluminum just past the tensile yield point (Tutorial 3, 300 K), colored by centrosymmetry. Atoms with a high centrosymmetry (dislocation cores and stacking faults) are drawn opaque, while the perfect-crystal atoms are rendered nearly transparent to provide context. Because the crystal is initially defect-free, dislocations nucleate homogeneously and densely rather than from isolated sources.

To see *where* and *how* the crystal yields, `deform.lmp` computes the per-atom *centrosymmetry parameter* [29], which is near zero for atoms in a perfect centrosymmetric (fcc) environment and large near defects such as dislocation cores, stacking faults, and surfaces, and saves it in the trajectory:

```
compute csym all centro/atom fcc
dump traj all custom 1000 deform-${mode}.dump &
id type xs ys zs c_csym c_pea
```

The per-atom centrosymmetry saved in this trajectory is used by LAMMPS to color the atoms when rendering an image. The helper input `solution/visualize.lmp` turns one frame into Fig. 6. Rather than reading the full trajectory, it loads `solution/tension-yield.dump`, a pre-extracted excerpt covering steps 12000-17000 (around the tensile yield point) provided in the solution folder, so the figure can be reproduced without re-running the full simulation. The script flags the high-centrosymmetry atoms as a second atom type and draws them opaque, while the rest of the crystal is rendered nearly transparent for context:

```
variable isdef atom "c_csym > 8.0"
group defect variable isdef
set group defect type 2
# ... then write an image colored by c_csym, with
# atrans 1 0.25 (bulk) atrans 2 1.0 (defects)
```

The result (Fig. 6) shows the crystal just past the tensile yield point: the nucleated dislocations stand out yet remain in context. They form homogeneously throughout the defect-free crystal and immediately interact, since there are no pre-existing sources.

The transparency is set per atom type: the atoms whose centrosymmetry exceeds a threshold are re-assigned to a second type and drawn opaque (with `dump_modify atrans`), while the rest stay nearly transparent. Because the renderer emulates transparency with a screen-door pattern (an ordered-dithering scheme [30]) rather than true alpha blending, the ambient-occlusion shading is best turned *off* (`ssao no`) for these images — it interacts poorly with the partially transparent atoms — and the colormap lower bound is raised above the bulk centrosymmetry values so all perfect-crystal atoms map to the same minimum color.

At 300 K, thermal displacements give many atoms a moderate centrosymmetry, so a fairly high threshold is used to isolate the defects. Lowering the temperature or time-averaging the per-atom values sharpens the picture, and identifying structures with common neighbor analysis (`compute cna/atom`) or polyhedral template matching is a common, noise-tolerant alternative.

3.3.5 Nanowires: surfaces, solutes, and failure

So far the crystal filled the whole periodic box. Real components have surfaces, and at the nanoscale the surface is most of the material. The input `wire.lmp` repeats the tension test on a cylindrical $\langle 100 \rangle$ nanowire of 3.2 nm diameter (4160 atoms), periodic only along its axis:

```
boundary p s s
region wire cylinder x 0.0 0.0 4.0 INF INF
create_atoms 1 region wire
```

Free surfaces change the bookkeeping before they change the physics: the global pressure is averaged over the entire box, including the empty space around the wire, so the meaningful axial stress must be summed from the per-atom stress over the wire volume instead:

```
compute satom all stress/atom NULL
compute sxxsum all reduce sum c_satom[1]
variable sxx equal c_sxxsum/(count(all)*v_vat)/10000.0
```

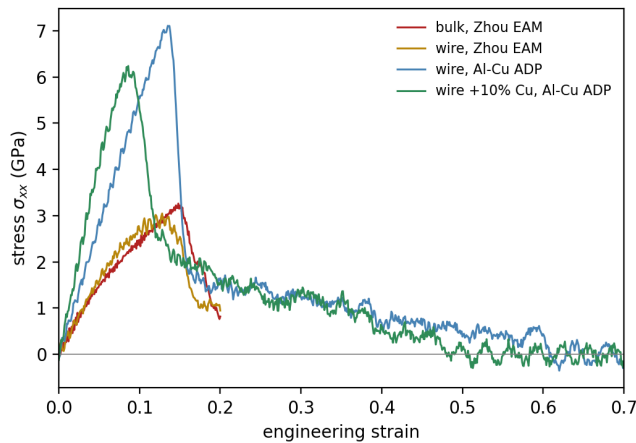


Figure 7. Tensile stress-strain curves of the bulk crystal and of 3.2 nm nanowires (Tutorial 3). With the same Zhou potential, the wire (gold) yields slightly below the bulk (red): dislocations now nucleate at the free surface. With the Apostol-Mishin Al-Cu potential the pure wire (blue) is more than twice as strong — the same observable, two very different answers from two published potentials. Adding 10 at % copper (green) *lowers* the nucleation peak but *raises* the flow stress that follows, and the alloyed wire separates earlier (its curve reaches zero near a strain of 0.5).

The wire is slightly softer than the bulk (apparent modulus 32 GPa versus 34 GPa) and yields earlier and lower (3.05 GPa at a strain of 0.128 versus 3.26 GPa at 0.148, Fig. 7): the free surface offers dislocation nucleation sites that the perfect periodic crystal lacks.

⌚ Pure aluminum is soft; engineering alloys owe their strength to solutes and precipitates. The input `wire-alloy.lmp` pulls the same wire to 100 % strain twice — once pure, once as a random Al-10 at % Cu solid solution — using the same Al-Cu potential [31] for both runs (about half an hour each), so any difference is caused by the copper atoms alone. The result upends the naive expectation in an instructive way (Fig. 7): the *pure* wire has the higher yield peak (7.1 GPa versus 6.2 GPa, reached already at a strain of 0.086 in the alloy). A defect-free nanowire is strength-limited by dislocation *nucleation*, and solute atoms act as heterogeneous nucleation sites that lower that barrier. The textbook strengthening mechanism — solutes pinning *moving* dislocations — appears after yielding, where the alloy sustains a roughly 30 % higher flow stress. It also shows the classic price of strength: the alloy localizes and breaks at a lower strain, while the pure wire keeps drawing out a neck (Fig. 8). Bulk metals, whose strength is governed by dislocation *motion* rather than nucleation, therefore gain strength from alloying — nanoscale and macroscale strength are controlled by different mechanisms.

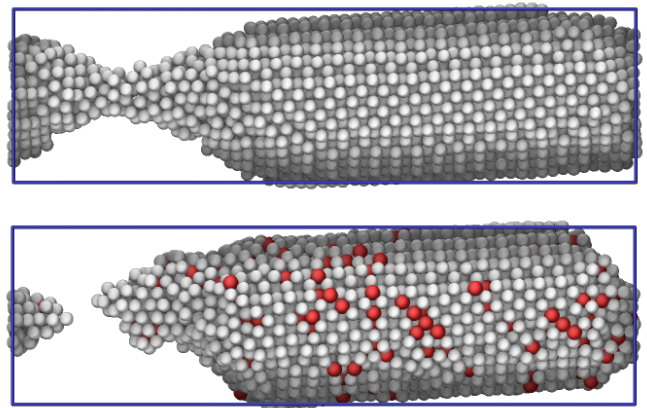


Figure 8. The two nanowires of `wire-alloy.lmp` at the same strain of 0.50 (Tutorial 3; aluminum light grey, copper red). The pure wire (top) draws out a long, thin neck and is still in one piece; the alloyed wire (bottom) has already separated — stronger in flow, but less ductile. Rendered with `solution/wireviz.lmp`.

Figure 7 hides a second lesson in plain sight: the pure-aluminum wire yields at 3.05 GPa with the Zhou potential but at 7.1 GPa with the Apostol-Mishin potential — the same geometry, the same protocol, and a factor of 2.3 between two published potentials. Dislocation nucleation probes the unstable stacking fault energy (Tutorial 7), to which neither potential was fitted. Checking a potential against the property class being computed, as done systematically in Tutorial 14, applies here as well.

3.4 Tutorial 4: Grain boundaries: construction and fracture

Real materials are rarely single crystals; they are made of many grains separated by *grain boundaries* (GBs), whose structure and energy strongly influence mechanical and transport properties. This tutorial treats grain boundaries in two stages. In the first part, a $\Sigma 5(310)$ symmetric tilt boundary in fcc aluminum (Fig. 9) is constructed and its energy computed; the whole procedure is in `gb.lmp`. In the second part, a bicrystal of bcc iron is pulled apart perpendicular to a $\Sigma 3(111)$ grain boundary until it fractures, and the fracture process is analyzed through the stress-strain curve (Fig. 11) and the per-atom stress field (Fig. 10).

3.4.1 Building a bicrystal

A symmetric tilt grain boundary is built from two grains that are rotated by equal and opposite angles about a common *tilt axis*. For the $\Sigma 5(310)$ boundary the tilt axis is $[100]$ and each grain is oriented so that a $\{310\}$ plane lies on the boundary. After defining a simulation box, two regions (`upper` and `lower`) are filled using the `lattice` command

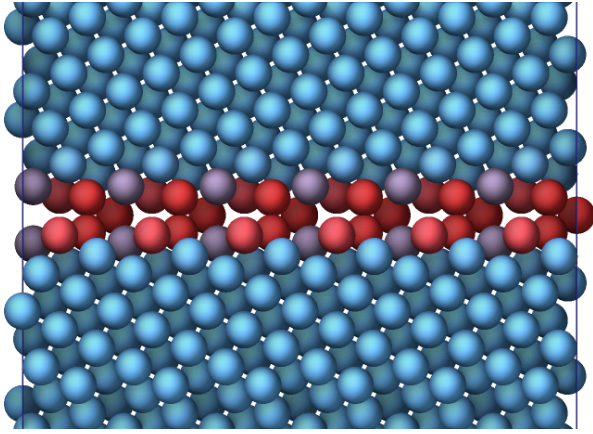


Figure 9. The relaxed $\Sigma 5(310)$ symmetric tilt grain boundary of Tuto-rial 4, viewed along the $[100]$ tilt axis and colored by centrosymmetry. The two grains (low centrosymmetry) meet along the horizontal boundary, where the periodic grain-boundary structural units (high centrosymmetry) are highlighted. Three periods along the boundary are shown.

with mirror-image orientations and a distinct atom type for each grain:

```
region upper block INF INF 0.0  $\{by\}$  INF INF units box
lattice fcc  $\{a_0\}$  orient x 0 3 1 orient y 0 -1 3 orient z 1 0 0
create_atoms 1 region upper
region lower block INF INF - $\{by\}$  0.0 INF INF units box
lattice fcc  $\{a_0\}$  orient x 0 3 -1 orient y 0 1 3 orient z 1 0 0
create_atoms 2 region lower
```

The boundary lies at $y = 0$, with the $[100]$ tilt axis along z . The box dimensions are set to whole multiples of the relevant periodicities ($\sqrt{10}a_0$ along x and y , a_0 along z), so that the crystal is commensurate with the periodic boundaries; because of periodicity, the cell actually contains *two* equivalent grain boundaries (one at $y = 0$ and one at the y cell edge). The two atom types are used only to distinguish the grains.

3.4.2 Removing overlapping atoms

Where the two grains meet, some atoms from opposite sides fall almost on top of one another. Such near-coincident atoms must be removed, or the huge repulsive forces between them would destabilize the simulation. The `delete_atoms` command does this:

```
delete_atoms overlap 0.35 lower upper
```

This deletes atoms in the `lower` group that lie within $0.35\text{\AA}$ of an atom in the `upper` group.

3.4.3 Computing the grain-boundary energy

The boundary is then relaxed in two stages: a conjugate-gradient minimization of the atomic positions, followed by a second minimization with `fix box/relax y 0.0` that also lets

The overlap distance is a parameter that affects the resulting boundary. Different overlap cutoffs (and rigid translations of one grain relative to the other before deletion) lead to different grain-boundary structures, generally with different energies. Sampling many such structures to find the lowest-energy one is the basis of more advanced grain-boundary studies [32].

the box expand or contract along the boundary normal. The grain-boundary energy is the excess energy relative to the perfect crystal, per unit of boundary area:

$$\gamma_{\text{GB}} = \frac{E_{\text{total}} - NE_{\text{coh}}}{2A}, \quad (2)$$

where E_{coh} is the perfect-crystal cohesive energy per atom (from Tutorial 1), $A = L_x L_z$ is the area of one boundary, and the factor of two accounts for the two boundaries in the periodic cell. Evaluated with a few `variable` commands and converted from $\text{eV}\text{\AA}^{-2}$ to mJ m^{-2} , this gives $\gamma_{\text{GB}} \approx 563 \text{ mJ m}^{-2}$ for the structure found here.

This value corresponds to the particular (metastable) boundary structure produced by the procedure above; a different structure would give a different energy. The result is reassuringly close to literature values for this boundary in aluminum, but a definitive grain-boundary energy requires searching over translations and atom-deletion criteria for the minimum-energy structure [32].

3.4.4 Grain-boundary fracture in iron

Grain boundaries are frequently where materials fail. The second part of this tutorial pulls a bicrystal of bcc iron apart *perpendicular* to a grain boundary until it fractures, and analyzes the process through the stress-strain response (Fig. 11) and the *per-atom* stress (Fig. 10). Two inputs are used: `makegb.lmp` builds and relaxes the boundary and writes it to a data file, and `fracture.lmp` carries out the fracture test. The interatomic potential is the Mendelev *et al.* embedded-atom iron potential [33], supplied with LAMMPS as `Fe_mm.eam.fs`.

3.4.5 Preparing the grain boundary

The boundary chosen here is the $\langle 110 \rangle \Sigma 3(111)$ symmetric tilt boundary in bcc iron. It is built with the same mirror-orientation technique as in the first part of this tutorial: two grains share the $[1\bar{1}0]$ tilt axis (along z) and meet at the (111) plane at $y = 0$, with the lower grain obtained from the upper one by a 180° rotation about the in-plane $[11\bar{2}]$ axis (a $\Sigma 3$ misorientation):

```
lattice bcc  $\{a_0\}$  orient x 1 1 -2 orient y 1 1 1 orient z 1 -1 0
```

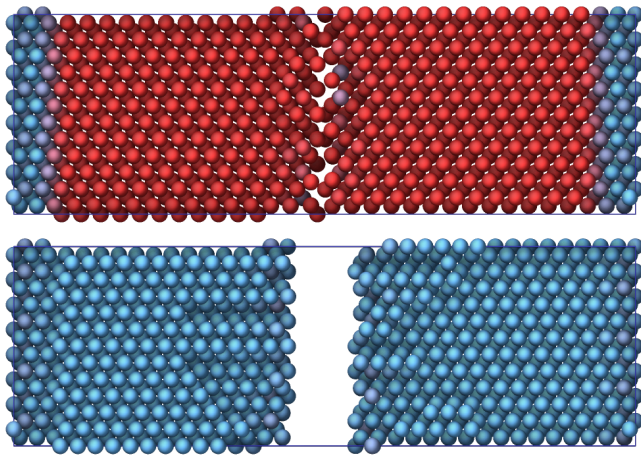



Figure 10. Grain-boundary fracture of the bcc-iron $\langle 110 \rangle \Sigma 3(111)$ bicrystal in Tutorial 4, colored by the per-atom stress σ_{yy} along the loading direction. *Top*: just below the peak load, the gauge section carries a large tensile stress (red) while the rigid grips at the two ends are unloaded (blue). *Bottom*: immediately after the boundary cleaves, a crack has opened along the interface and the stress has relaxed throughout (blue).

```
create_atoms 1 region upper
lattice bcc $a0 orient x 1 1 -2 orient y -1 -1 -1 orient z -1 1 0
create_atoms 2 region lower
```

After `delete_atoms overlap` removes near-coincident atoms at the interface, the structure is relaxed (a conjugate-gradient minimization followed by a second minimization with `fix box/relax x 0.0 z 0.0` to relieve the in-plane stress) and written to `Fe_gb.data` with `write_data`. The grain-boundary energy, computed as in Eq. 2, is about 1470 mJ m^{-2} — much higher than the aluminum $\Sigma 5(310)$ boundary of the first part of this tutorial, which is why this boundary is a likely fracture path.

The oriented cell has a repeat period along x of only two-thirds of a coincidence-site-lattice period, so the box width must be a multiple of three lattice steps to remain commensurate with the boundary; `makegb.lmp` uses six. Using an incommensurate width produces a strained, artificially high-energy boundary — a useful reminder to verify that a constructed interface is actually periodic.

3.4.6 Straining to fracture

`fracture.lmp` reads `Fe_gb.data` with a *non-periodic* `y` boundary (`boundary p s p`), so the bicrystal has free surfaces at the top and bottom and a single grain boundary in the middle. Thin slabs of atoms at each end act as rigid *grips*: their forces are zeroed so they move only when displaced. The load is applied quasi-statically (and athermally, in contrast to the finite-temperature dynamics of Tutorial 3) by a

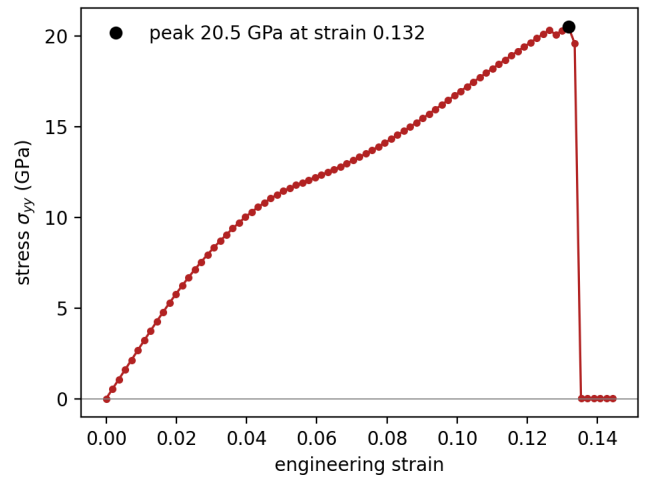


Figure 11. Stress-strain response of the grain boundary in Tutorial 4 under quasi-static tension. The stress rises until the boundary cleaves, then drops abruptly to zero — brittle fracture. The peak here is the *ideal* (athermal, defect-free) cleavage strength; see the note in Sec. 3.4.7.

loop that displaces the grips apart by a small increment and re-minimizes the energy after each step:

```
fix holdtop gtop setforce 0.0 0.0 0.0
fix holdbot gbot setforce 0.0 0.0 0.0
# ... in a loop over loading increments:
displace_atoms gtop move 0 $delta 0 units box
displace_atoms gbot move 0 - $delta 0 units box
minimize 1.0e-10 1.0e-10 500 5000
```

Because `fix setforce` reports the total force it zeroed, the reaction force on the top grip, `f_holdtop[2]`, gives the load carried by the bicrystal; dividing by the cross-sectional area $L_x L_z$ yields the tensile stress, which is written together with the engineering strain to `stress-fracture.csv` for pandas.

3.4.7 Stress-strain response and per-atom stress

Running `fracture.lmp` and plotting the result with `plot.py` gives Fig. 11: the stress rises with strain and then, at a peak near 20 GPa, drops abruptly to zero as the boundary cleaves in a single step. This sudden drop is the signature of *brittle* fracture.

To see *where* the material fails, `fracture.lmp` also computes the per-atom virial stress and saves it in the trajectory. The component along the loading direction, σ_{yy} , is obtained by dividing the per-atom stress (which has units of pressure $\times$ volume) by the atomic volume:

```
compute satom all stress/atom NULL
variable syy atom c_satom[2]/v_vatom/10000.0
```

The helper input `solution/visualize.lmp` colors one frame of the trajectory by σ_{yy} to produce Fig. 10 (run

`fracture.lmp` first to generate `fracture.dump`, which `solution/visualize.lmp` reads): just below the peak load the gauge section is uniformly stressed (red) while the grips carry no load (blue), and immediately after the peak a clean crack has opened along the boundary with the stress relaxed everywhere. The same per-atom trajectory can be rendered into images with LAMMPS, as `solution/visualize.lmp` does, and reviewed in the LAMMPS-GUI «Image Viewer», or loaded into OVITO [18] or VMD [34] for interactive three-dimensional analysis.

The peak stress obtained here (about 20 GPa) is the *ideal* cleavage strength of a perfect, defect-free boundary loaded athermally: with no pre-existing cracks, pores, or thermal fluctuations, the boundary must be stretched all the way to its cohesive limit before it fails. Real materials contain flaws that concentrate stress and fracture at far lower loads, just as the perfect crystals of Tutorial 3 yielded only at the very high stress required for *homogeneous* dislocation nucleation. Lower, more realistic strengths require larger samples that include defects, finite temperature, and a finite loading rate.

3.5 Tutorial 5: Nanoindentation and nano-stamping

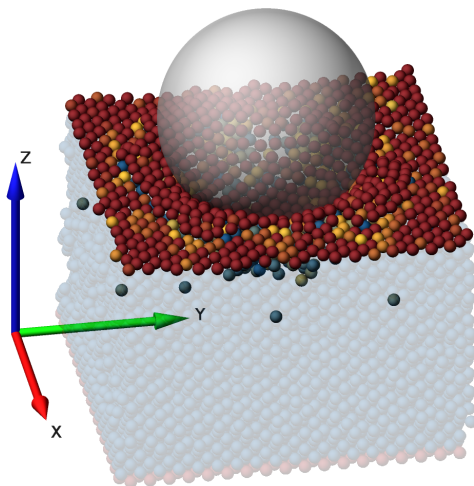


Figure 12. Dislocations nucleated beneath the spherical indenter in Tutorial 5. As in Fig. 6, the perfect-crystal atoms are drawn as transparent ghosts while the high-centrosymmetry atoms (the contact and the dislocation loops emitted beneath it) are opaque and colored by centrosymmetry. The semi-transparent silver sphere shows the indenter position at the selected frame.

This tutorial probes a surface mechanically. A free aluminum surface is first *indented* with a rigid tip — measuring

its load-displacement response and watching dislocations nucleate beneath the contact (Figs. 13 and 12) — and is then *stamped* with a shaped rigid tool that presses a pattern into it (Fig. 14). The indentation is driven by `indent.lmp` and the stamping by `stamp.lmp`. Both introduce techniques used throughout surface and interface simulations: a properly thermostatted slab, indenters of different shapes, and a multi-element `hybrid` interatomic potential.

Since LAMMPS version 11 Feb 2026, `fix indent` provides its geometry natively to `dump image` via the `fix` keyword, replacing the older workaround of adding an auxiliary `region`. The input `indent.lmp` therefore includes a `dump image` command (named `snap`) with `fix ind type 0 0`, which renders the indenter alongside the atoms in every frame; when run from LAMMPS-GUI, the frames appear in the Slideshow viewer so you can watch the indenter descend into the surface. The two trailing numbers are `fflag1` and `fflag2`; for the sphere, a small negative `fflag2` (e.g. `fix ind type 0 -5`) reduces the rendered radius so that atoms in the contact zone remain visible. Configuring `dump image` for `fix` and `compute` objects by hand can be challenging because the interface is generic: each `fix` or `compute` defines its own meaning for `fflag1` and `fflag2`. The LAMMPS-GUI Image Viewer addresses this with a dedicated «Compute/Fix» dialog. After a run, open the «Image Viewer», click «Compute/Fix» in the settings panel, tick «Show» next to `fix ind`, set «Flag #2» to adjust the rendered radius, and press «Apply» to preview. The resulting `dump image` line can then be copied to the clipboard with the «Copy» button and pasted into the input script for further fine-tuning; rerunning and monitoring the output in the Slideshow viewer closes the loop. For full details see the `fix indent` and `dump image` documentation [8, 21]. An extended example that overlays an animated force arrow, a live text legend, and a colour-scale bar — following the style of the `in.breakable` LAMMPS example — is provided as `solution/indent_viz_advanced.lmp`.

3.5.1 Building and equilibrating the slab

Both inputs start from an fcc aluminum slab with a free (100) surface at the top. With a free surface the slab needs more care than a fully periodic crystal: the bottom must be held in place, and the thermostat must not interfere with the very process under study. The slab is therefore divided into three layers along the surface normal:

```
group bottom region rbot
group mobile subtract all bottom
```

```
group buffer region rbuf
fix integrate mobile nve
fix thermostat buffer langevin ${temp} ${temp} 0.1 90210
```

The `bottom` layer is simply never time-integrated, so it stays fixed and supports the slab. Only the `mobile` atoms above it move (`fix nve`), and the *thermostat acts only on the `buffer layer`* just above the fixed base. This mimics heat flowing into a much larger bulk: the surface region is integrated with `fix nve` alone, so the heat generated by indentation flows out through the buffer by conduction rather than being removed everywhere by an artificial bath, which would unphysically damp the local response [29]. The boundary is non-periodic along the surface normal (`boundary p p f`) and periodic in the surface plane.

3.5.2 Indenting with a sphere and a flat punch

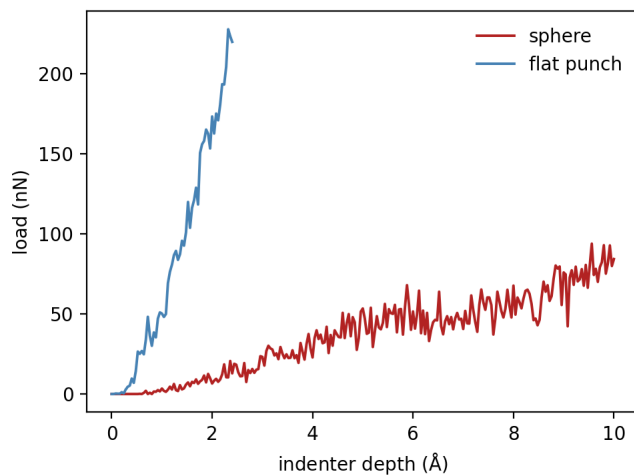


Figure 13. Load-displacement curves for the spherical indenter and the flat punch of Tutorial 5. The sphere shows discrete load drops (pop-ins) as dislocations nucleate under the tip; the flat punch compresses the whole surface and rises more steeply.

The indenter is a rigid, purely repulsive object added with `fix indent`. Its position is an equal-style variable, so it descends at a constant rate as the simulation proceeds:

```
variable zind equal v_z0-v_rate*step*0.001
fix ind all indent 100.0 sphere v_xc v_yc v_zind ${radius} units box
```

The same input also offers a flat punch (`fix indent ... plane z v_zind hi`), selected through the `shape` variable: the sphere loads a small contact and concentrates stress under the tip, whereas the plane compresses the whole surface uniformly. `fix indent` reports the force the indenter exerts on the atoms, so `f_ind[3]` versus the indenter depth is the load-displacement curve, written to a CSV file and plotted by `plot.py` (Fig. 13). For the sphere the load rises elastically

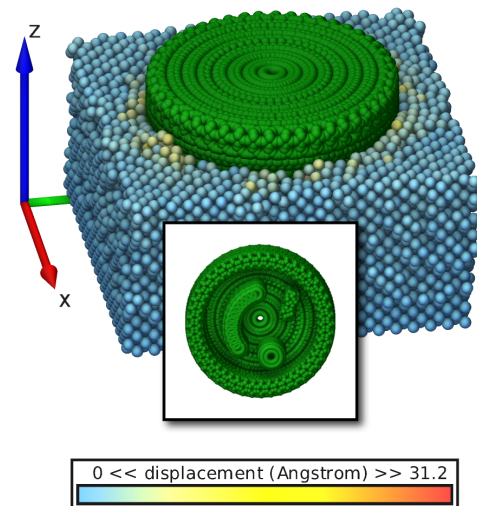


Figure 14. Nano-stamping in Tutorial 5, with the metal atoms colored by their displacement from the initial state (as shown by the color scale in the middle). *Top*: the tool (green; atoms created using an STL mesh file) pressed into the aluminum. The inset shows the tool turned over. *Bottom*: after the tool is removed and the surface has relaxed, the displaced metal atoms form a raised structure with an emoji-shaped imprint.

and then shows a sharp *pop-in* — a load drop where the first dislocations nucleate [29].

3.5.3 Nano-stamping with a shaped tool

Replacing the analytical indenter with a real, atomistic tool of arbitrary shape turns indentation into *stamping*. The tool is built from a triangle mesh: `generate_tool.py` writes its shape as an STL file (here, for fun, a winking smiley), and `create_atoms` fills the mesh surface with (carbon) atoms:

```
create_atoms 2 mesh tool.stl meshmode bisect 1.4 units box
```

The aluminum substrate and the tool are described with `pair_style hybrid`: an embedded-atom potential for the metal and a *pairwise* Lennard-Jones term for the interaction *between* the tool and the metal:

Any watertight STL mesh works as the tool, so the thousands of models in online 3D-printing libraries can be stamped into a surface — an easy and entertaining way to explore `create_atoms mesh`. As in the earlier mechanical tutorials, the rates here are far above experimental ones and the system is tiny, so the result is qualitative: its value is in showing the *mechanisms* (plastic imprint, pile-up) and in demonstrating how a multi-element `hybrid` potential with a rigid body is assembled.

```
pair_style hybrid eam/alloy lj/cut 5.0
pair_coeff * * eam/alloy Al_zhou.eam.alloy Al NULL
pair_coeff 1 2 lj/cut 0.10 2.5
```

The cross interaction *must* be a pairwise-additive style (here `lj/cut`): the EAM potential is many-body and defined only among the aluminum atoms, so it cannot describe the metal-tool contact. Because the tool is held rigid (with `fix move`), no potential is needed *within* it — only its repulsion against the surface matters — so its internal interactions are simply switched off with `neigh_modify exclude`. (A *deformable* tool would instead need its own potential, for example `tersoff` [35] for a diamond tip, while the cross term would still have to be pairwise.) The tool is pressed straight into the surface and then lifted off:

```
fix drive tool move linear 0.0 0.0 -${vpress} units box
```

To see how the material flows, the atoms are colored by how far they have moved from the equilibrated surface, obtained with `compute displace/atom`. What remains (Fig. 14) is the imprint of the tool, with material plastically displaced into the surface and piled up around the edges of the features.

3.6 Tutorial 6: Elastic constants of crystalline silicon

The elastic stiffness tensor C_{ij} describes the linear relationship between stress and strain and is one of the most fundamental mechanical properties of a crystal. In Tutorial 2 the bulk modulus B_0 was extracted from the curvature of the energy-volume curve; in Tutorial 3 the Young's modulus was estimated from the slope of the stress-strain curve. This tutorial goes further: the full 6×6 Voigt stiffness matrix is assembled by applying small, independent strains in each of the six symmetry-distinct directions and recording the corresponding stress response. Within Voigt notation the six stress and strain components are numbered 1–6 (xx , yy , zz , yz , xz , xy), and column j of the stiffness matrix is the stress response to a strain in direction j , $C_{ij} = \partial\sigma_i/\partial\epsilon_j$. For a cubic crystal such as silicon symmetry reduces these 36 numbers to three independent constants C_{11} , C_{12} , and C_{44} , from which the bulk

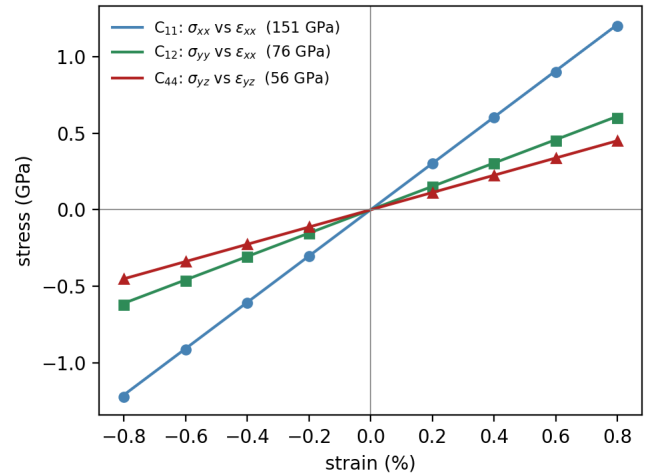


Figure 15. Stress-strain response of Stillinger-Weber silicon from Tutorial 6. Each elastic constant is the slope of a stress component against an applied strain: stretching along x gives C_{11} from σ_{xx} (blue) and C_{12} from the transverse σ_{yy} (green); a shear of the box gives C_{44} from σ_{yz} (red). Symbols are LAMMPS data; lines are linear fits.

modulus, shear modulus, and other aggregate properties follow.

The tutorial uses crystalline silicon described by the Stillinger-Weber potential [36] (pair style `sw`, parameter file `Si.sw`). Silicon was chosen because the SW potential converges rapidly and the experimental elastic constants are well established; the same approach applies directly to the EAM metals used in the earlier tutorials. The method follows the `ELASTIC/` example in the LAMMPS distribution [11]; a comprehensive discussion of finite-strain methods for elastic constants is given by Clavier et al. [37].

3.6.1 Setting up and relaxing the silicon crystal

Open `elastic.lmp`. It builds a $3 \times 3 \times 3$ diamond-cubic supercell of silicon (Fig. 16) and first relaxes both the box and the atom positions to a state of zero stress, which provides the strain-free reference for all later deformations:

```
lattice diamond ${a0}
region box prism 0 3 0 3 0 3 0 0 0
create_box 1 box
create_atoms 1 box
mass 1 28.0855
pair_style sw
pair_coeff * * Si.sw Si

fix relax all box/relax aniso 0.0
minimize 0.0 1.0e-12 1000 10000
unfix relax
```

The `region` is declared as a `prism` (a triclinic box with zero initial tilt) so that the shear deformations applied later are allowed. `fix box/relax aniso 0.0` lets each box edge relax inde-

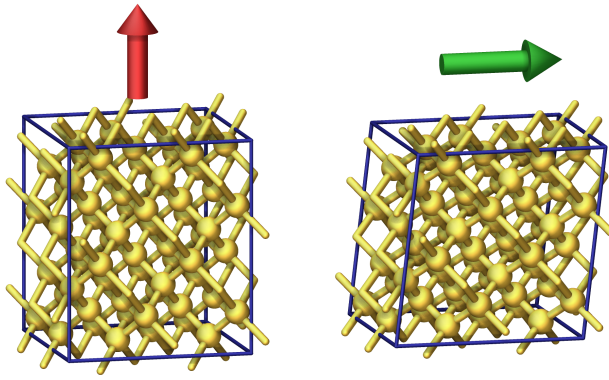


Figure 16. The two deformations used to measure the elastic constants of diamond-cubic silicon (Tutorial 6), exaggerated for clarity; the arrow marks the direction of the applied strain. At left, a uniaxial tension along z (red): the normal stress it induces gives C_{11} and the transverse stress gives C_{12} . At right, a simple shear (green) that tilts the cell into a parallelepiped and gives C_{44} . The Si-Si bonds are not part of the model but added during visualization based on the atom-atom distances with the `autobond` feature of `dump image`.

pendently to zero pressure. Minimization yields the SW equilibrium lattice constant $a_0 = 5.431 \text{ \AA}$ and a cohesive energy of -4.336 eV per atom. The relaxed cell is written to a data file and its edge lengths are stored as numerical constants:

```
write_data Si_equil.data
variable lx0 equal $(lx)
```

The `$(...)` form is an *immediate* (anonymous) variable expansion: like `$(a0)`, it is evaluated right away during input processing and freezes the current box length into a plain number. This matters because an equal-style *formula* such as `lx` is re-evaluated every time it is used and would error once the box has been deleted and rebuilt in the loop below, whereas the frozen constant survives. (The same trick captures a reference energy as `variable e0 equal $(pe)`.)

Because the model uses `atom_style atomic`, it carries no explicit bond topology, so a plain rendering would show only isolated spheres. The script `visualize.lmp` that produced Fig. 16 instead lets `dump image` infer the bonds on the fly from the neighbor list with the `autobond` keyword — `autobond 2.7 0.45` draws a stick between any pair of atoms closer than $2.7 \text{ \AA}$ (just beyond the $2.35 \text{ \AA}$ Si-Si nearest-neighbor distance). The same script adds the labeled deformation arrows with `fix graphics/objects` and `fix graphics/labels`.

3.6.2 Scanning Voigt strains

The heart of the tutorial is a double loop: an outer loop over the six Voigt directions and an inner loop over a list of small strains. For each combination the script reloads the refer-

ence cell, deforms it, relaxes the internal atom positions *at fixed box shape*, and records the full pressure tensor:

```
variable dir loop 6
label dirloop
  variable eps index -0.008 -0.006 -0.004 -0.002 &
                        0.002 0.004 0.006 0.008

  label epsloop
    clear
    units metal
    read_data Si_equil.data
    pair_style sw
    pair_coeff * * Si.sw Si
    # ... set len0 for this direction
    if "${dir} == 1" then "variable len0 equal ${lx0}"
    variable d equal ${eps}*${len0}
    if "${dir} == 1" then &
      "change_box all x delta 0 $d remap units box"
    minimize 0.0 1.0e-12 1000 10000
    print "- {dir: ${dir}, eps: ${eps}, pxx: ${pxx:%.6f}, ...}" &
      append scan.yaml screen no
  next eps
jump SELF epsloop
next dir
jump SELF dirloop
```

Directions 1~3 stretch one box edge with `change_box x|y|z`, while directions 4~6 add a tilt with `change_box yz|xz|xy`; the `remap` keyword moves the atoms affinely with the box. Crucially, the subsequent `minimize` relaxes only the atoms *without* `fix box/relax`, so the imposed strain is held fixed and the resulting stress is the elastic response. As in Tutorial 2, each result is appended as one record to a structured `scan.yaml` file.

The `clear` command resets *everything* except variables, including the unit system, which reverts to the `lj` default. The `read_data` command does not restore units, so `units metal` must be repeated after every `clear`. Omitting it is a classic silent error: the run completes, but the reported pressures are in Lennard-Jones units and the elastic constants come out wrong by the metal-units pressure conversion factor of $\approx 1.6 \times 10^6$. (A binary `restart` file does store the units, which is why the original `ELASTIC/` example uses `read_restart` instead.)

3.6.3 Assembling and interpreting the stiffness matrix

The script `plot.py` reads `scan.yaml`, and for each direction j fits a straight line to each stress component $\sigma_i = -p_i$ against the applied strain. The slope, converted from bar to GPa, is the matrix element C_{ij} :

```
for j in range(6):
  sub = df[df["dir"] == j + 1]
```



```

eps = sub["eps"].to_numpy()
for i, comp in enumerate(comps):
    sigma = -sub[comp].to_numpy()
    C[i, j] = np.polyfit(eps, sigma, 1)[0] * BAR_TO_GPA

```

Figure 15 plots the three responses whose slopes are the three independent constants. The assembled matrix has the expected cubic form (values in GPa):

$$C_{ij} = \begin{bmatrix} 151.4 & 76.4 & 76.4 & 0 & 0 & 0 \\ 76.4 & 151.4 & 76.4 & 0 & 0 & 0 \\ 76.4 & 76.4 & 151.4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 56.4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 56.4 & 0 \\ 0 & 0 & 0 & 0 & 0 & 56.4 \end{bmatrix}. \quad (3)$$

The three distinct entries, $C_{11} = 151.4$ GPa, $C_{12} = 76.4$ GPa, and $C_{44} = 56.4$ GPa, reproduce the analytic values for SW silicon [37] to the tabulated precision, and the off-diagonal blocks vanish as required by cubic symmetry. From the cubic constants the script forms the Voigt–Reuss–Hill aggregate moduli [27] — bulk modulus $B = 101$ GPa, shear modulus $G = 48$ GPa, Young’s modulus $E = 124$ GPa, and Poisson ratio $\nu = 0.30$ — together with the Zener anisotropy ratio $A = 2C_{44}/(C_{11} - C_{12}) = 1.51$.

Compared with experiment ($C_{11} = 165.6$ GPa, $C_{12} = 63.9$ GPa, $C_{44} = 79.6$ GPa, $A = 1.56$ [27]), the SW potential captures the bulk modulus and the qualitative anisotropy well but underestimates the shear constant C_{44} and overestimates C_{12} . This is a known limitation of the short-ranged three-body SW form and a reminder that an empirical potential fit to a few properties need not reproduce the full elastic tensor — exactly the kind of check this tutorial enables.

An alternative, more rigorous approach available in LAMMPS is the Born matrix method (`compute born/matrix`, `EXTRA-COMPUTE` package), which evaluates the stiffness tensor analytically from the second derivatives of the potential energy and extends naturally to finite temperature. The small-strain method used here is simpler, needs no special compute, and builds directly on the deformation concepts from Tutorials 2 and 3.

3.7 Tutorial 7: Generalized stacking fault energy

Dislocations in fcc metals glide primarily on close-packed $\{111\}$ planes. The resistance to glide and the tendency of a dislocation to split into two Shockley partials are governed by the generalized stacking fault energy (GSFE): the energy cost per unit area of rigidly shifting one half of the crystal relative

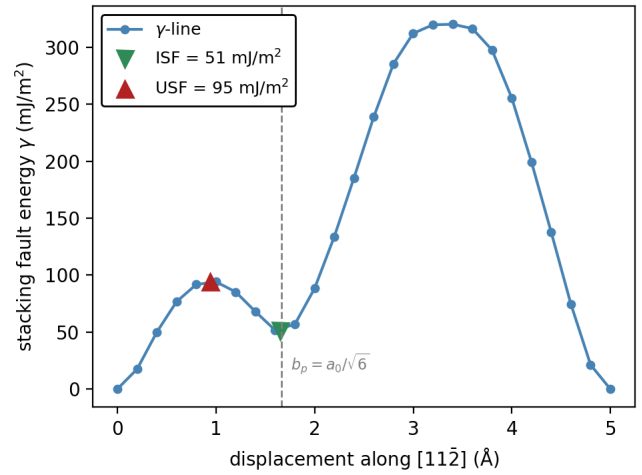


Figure 17. Generalized stacking fault energy (γ -line) of fcc aluminum along $[11\bar{2}]$ on the (111) plane, from Tutorial 7. The first maximum is the unstable stacking fault energy (USF, red); the local minimum at one Shockley partial $b_p = a_0/\sqrt{6}$ (dashed line) is the intrinsic stacking fault energy (ISF, green); the curve returns to zero after one full in-plane lattice repeat $a_0\sqrt{6}/2$.

to the other by a displacement vector parallel to the slip plane. Sampling this energy along a line in the slip plane produces the γ -line (Fig. 17); the full two-dimensional map is the γ -surface. This tutorial computes the γ -line for fcc aluminum (EAM/alloy Zhou potential, the same `Al_zhou.eam.alloy` as in Tutorials 1–3) along the $\langle 11\bar{2} \rangle$ direction on (111) , directly connecting to the dislocation activity observed in Tutorial 3. The method follows Vitek [38].

The γ -line has two characteristic points: the *unstable* stacking fault energy (USF), the maximum a partial dislocation must overcome to glide, and the *intrinsic* stacking fault energy (ISF), the local minimum reached after the leading Shockley partial has passed. The ISF sets the equilibrium separation of the two partials.

3.7.1 Building the stacking-fault slab

Open `gsfe.lmp`. It orients the crystal so that the slip system lies along the box axes ($x = [1\bar{1}0]$, $y = [11\bar{2}]$, $z = [111]$) and builds a slab with free surfaces in z :

```

boundary p p s # free surfaces in z: a single fault plane
lattice fcc ${a0} orient x 1 -1 0 orient y 1 1 -2 orient z 1 1 1
region box block 0 4 0 6 0 8
create_box 1 box
create_atoms 1 box

```

The z direction is non-periodic (`boundary p p s`) so that a *single* fault plane is introduced when the upper half is displaced; the constant free-surface energy cancels when energy differences are taken.

The number of cells in y must be a multiple of three. The in-plane lattice repeat along $[11\bar{2}]$ is $a_0\sqrt{6}/2$, but the oriented `lattice` cell that LAMMPS uses along y is $\frac{4}{3}$ of it. Only when the box spans a whole number of lattice repeats does the rigidly shifted upper block remain in registry with its periodic image; otherwise the boundary carries a spurious strain that inflates the computed energy. This is the same commensurability requirement met in the grain-boundary construction of Tutorial 4.

3.7.2 Scanning the shift vector

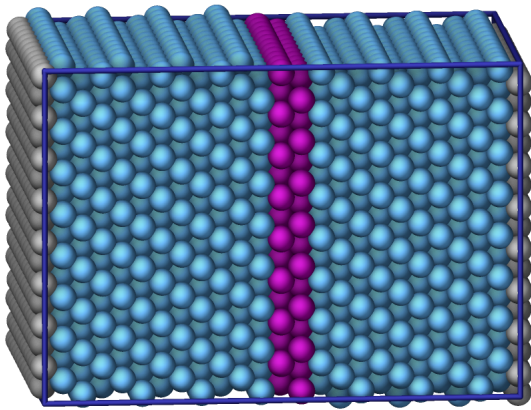


Figure 18. The intrinsic stacking fault in fcc aluminum (Tutorial 7), the configuration at the γ -line minimum. Common-neighbor analysis identifies the two close-packed planes at the fault as hcp-coordinated (purple), the fcc bulk (blue), and the free surfaces (gray).

Following the strategy of Tutorials 2 and 6, the ideal slab is written once to a data file and a loop reloads it for each displacement so that the shifts are independent. At each step the upper half is rigidly displaced along $[11\bar{2}]$ and the atoms are relaxed *only* perpendicular to the fault plane:

```
region rup block INF INF INF INF ${zcut} INF units box
group upper region rup
variable shift equal ${k}*${period}/${np}
displace_atoms upper move 0.0 ${shift} 0.0 units box
fix frz all setforce 0.0 0.0 NULL # zero in-plane forces: relax z only
minimize 1.0e-8 1.0e-8 1000 10000
```

The threshold `${zcut}` places the cut in the gap between two close-packed planes near the middle of the slab; it is computed from the lattice constant so the same script works for any fcc metal (see below). `fix setforce 0.0 0.0 NULL` zeroes the x and y force components so the minimizer relaxes atoms along z only; allowing in-plane relaxation would simply let the imposed fault slide back to the perfect crystal. Each step appends the energy and fault area to a structured `gsfe.yaml` file. The faulted configuration itself (Fig. 18) can be visualized with `visualize.lmp`, which uses `compute cna/atom` to

pick out the hcp-coordinated fault plane and draws it opaque over a ghosted bulk.

3.7.3 The γ -line and its physical interpretation

The script `plot.py` converts the energies to a stacking fault energy per unit area, $\gamma = (E - E_0)/A$, with a single fault plane so no factor of two is needed:

```
gamma = (pe - pe[0]) / area * EV_PER_A2_TO_MJm2
```

It then locates the intrinsic minimum and the unstable maximum (Fig. 17). For the Zhou potential we obtain $\gamma_{\text{ISF}} \approx 51 \text{ mJ m}^{-2}$ at the Shockley partial displacement $b_p = a_0/\sqrt{6} = 1.67 \text{ \AA}$, and $\gamma_{\text{USF}} \approx 95 \text{ mJ m}^{-2}$. The curve returns to zero after one full in-plane repeat $a_0\sqrt{6}/2 = 5.0 \text{ \AA}$, confirming that the displaced crystal is again perfect there.

These values are lower than the experimental intrinsic stacking fault energy of aluminum, usually quoted in the range 120 mJ m^{-2} to 166 mJ m^{-2} [39]; embedded-atom potentials are known to scatter widely for this quantity, so a stacking fault energy should always be checked against the intended application rather than assumed accurate. Even so, the result is physically consistent with Tutorial 3: a low ISF favors wide dissociation of glide dislocations into Shockley partials. The isotropic estimate of the partial separation, $d \approx \mu b_p^2 / (8\pi\gamma_{\text{ISF}})$ with the shear modulus $\mu \approx 26 \text{ GPa}$ of aluminum, gives d of order 1 nm, comparable to the dissociated cores seen by the centro-symmetry analysis in Tutorial 3.

3.7.4 Trying another metal: copper

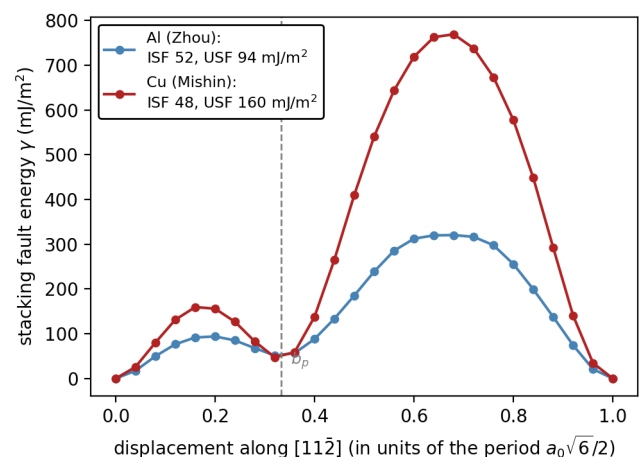


Figure 19. γ -lines of aluminum (Zhou) and copper (Mishin) from the same script (Tutorial 7), with the displacement scaled by each metal's in-plane repeat $a_0\sqrt{6}/2$ so the curves share an axis. The intrinsic minima (b_p , dashed) are similar, but copper has a much higher unstable barrier and a deeper γ -surface overall.

Because `gsfe.lmp` selects the material through three index variables, the identical calculation runs any fcc metal by

overriding them. Switching to copper with the Mishin potential [40],

```
Imp -in gsfe.lmp -var pot Cu_mishin1.eam.alloy -var elem Cu -var lat 3.61493
```

and overlaying the result with `compare.py` gives Fig. 19. Copper has nearly the same intrinsic stacking fault energy as the Zhou aluminum (48 vs 51 mJ m^{-2}) but a much larger unstable energy ($\gamma_{\text{USF}} \approx 160$ vs 95 mJ m^{-2}) and a γ -surface that is more than twice as deep. Notably, the Mishin copper value agrees well with the experimental copper SFE of about 45 mJ m^{-2} , whereas the Zhou potential *underestimates* that of aluminum: a concrete reminder that derived properties are only as good as the potential, and that the same workflow is the right way to test a new one.

3.8 Tutorial 8: Thermal conductivity by reverse NEMD

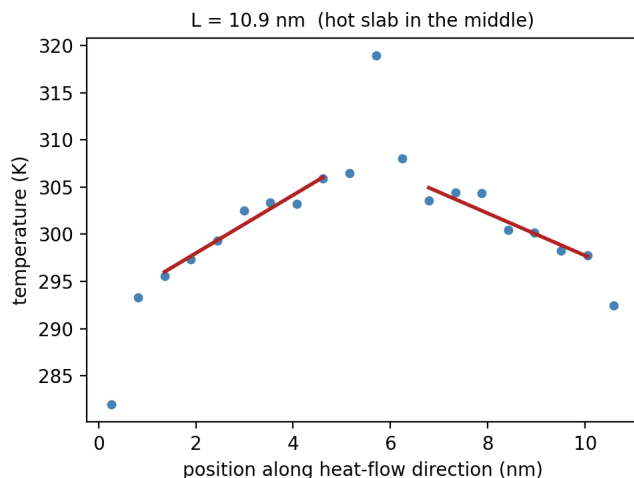


Figure 20. Steady-state temperature profile of an 11 nm silicon bar under reverse NEMD (Tutorial 8). The momentum swaps pump heat into the central slab and out of the (periodic) edge slabs, so the temperature peaks in the middle; the slope of each linear branch (red) is the gradient dT/dz in Fourier’s law.

The thermal conductivity κ governs heat transport in a material and is of central importance in applications ranging from heat sinks to thermal barrier coatings. This tutorial computes κ for silicon (Stillinger–Weber potential [36]) using the reverse non-equilibrium molecular dynamics (RNEMD) method of Müller-Plathe [41]. In this approach the momentum of the hottest atom in a “cold” slab is periodically swapped with that of the coldest atom in a “hot” slab, creating a steady-state temperature gradient without an external thermostat; κ then follows directly from Fourier’s law $\kappa = J_z/(dT/dz)$. This imposes a known flux and measures the resulting gradient — the converse of the nanoshear tu-

torial in the companion article [6], which imposes a velocity gradient and measures the resulting stress.

Silicon is chosen because its thermal conductivity is phonon-dominated (no electronic contribution) and its experimental value ($\kappa \approx 148 \text{ W m}^{-1} \text{ K}^{-1}$ at room temperature) provides a clear benchmark. Just as importantly, the tutorial exposes the central difficulty of thermal-transport simulations: the computed κ rises steeply with the length L of the cell along the heat-flow direction, because the silicon phonon mean free path (of order 100 nm to 300 nm) far exceeds any cell that runs interactively. The scan below makes this finite-size effect explicit. The input is adapted from `examples/KAPPA/in.mp` in the LAMMPS distribution [11].

3.8.1 Setting up the heat-exchange geometry

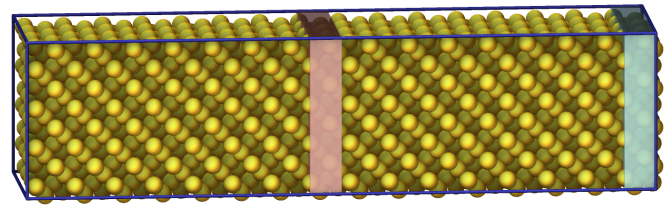


Figure 21. Reverse-NEMD geometry (Tutorial 8): a silicon bar divided into slabs along the heat-flow axis. `fix thermal/conductivity` swaps momenta between the cold edge slab (blue) and the hot central slab (red), pumping heat from cold to hot. The two exchange slabs are drawn as 50%-transparent regions with the `dump image region draw style`; atoms are shown at the covalent radius.

Open `kappa.lmp`. For each length it builds a silicon bar with a 6×6 cross-section, periodic in all three directions, and divides it into 20 slabs along the heat-flow axis z (Fig. 21):

```
region box block 0 6 0 6 0 0 ${nz}
...
fix swap all thermal/conductivity 100 z 20
```

`fix thermal/conductivity` performs the Müller-Plathe exchange: every 100 steps it swaps the velocity of the hottest atom in the cold (edge) slab with that of the coldest atom in the hot (central) slab, forcing heat to flow “uphill”. The kinetic energy moved by the swaps is an exactly known heat input; the crystal responds with a temperature gradient.

3.8.2 Running the simulation and reading the gradient

The bar is equilibrated at 300 K, switched to `fix nve`, run with the swaps active until the gradient is steady, and then sampled in production. The temperature profile is built by binning the atoms into slabs along z :

```
compute layers all chunk/atom bin/1d z lower 0.05 units reduced
```

```
fix prof all ave/chunk 100 100 10000 layers v_temp file profile_${nz}.dat
```

with `v_temp` the per-atom temperature `c_ke/(1.5*8.617e-5)`. The result is the tent-shaped profile of Fig. 20: hot in the middle, cold at the ends, with two linear branches in between. As in Tutorials 2, 6, and 7 an `index` variable loops the entire calculation over several bar lengths, appending one record per length (swapped energy, elapsed time, cross-section) to a structured `kappa.yaml`. This is the most demanding input in the set: the full scan takes of order ten minutes, so run it with threads, and ideally in the background.

3.8.3 Finite-size effects and the bulk limit

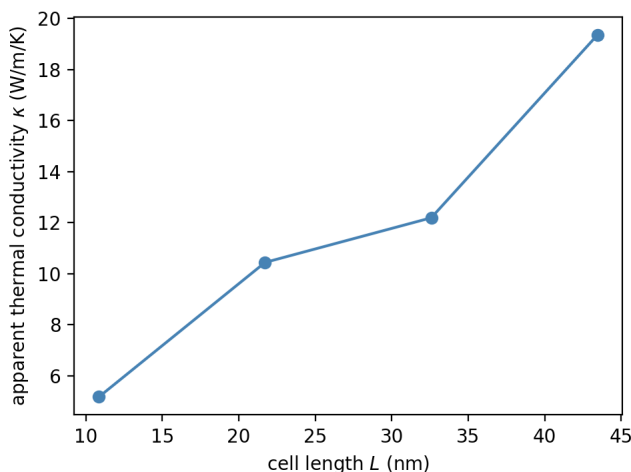


Figure 22. Apparent thermal conductivity of silicon versus cell length (Tutorial 8). κ climbs steeply and does not saturate over the accessible range — the cells are far shorter than the phonon mean free path — and stays an order of magnitude below the experimental bulk value (dashed).

The script `plot.py` fits the two linear branches of each profile (excluding the swap slabs) to obtain dT/dz and combines it with the heat flux $J_z = E_{\text{swap}}/(2At)$ to give $\kappa = J_z/(dT/dz)$. The factor of two arises because the heat flows both ways around the periodic cell; the conversion from eV per $\text{\AA}^2$ per ps to $\text{W m}^{-1} \text{K}^{-1}$ is a factor of 1602.

For the Stillinger-Weber potential the apparent conductivity climbs from $\kappa \approx 5 \text{ W m}^{-1} \text{K}^{-1}$ at $L = 11 \text{ nm}$ to roughly $19 \text{ W m}^{-1} \text{K}^{-1}$ at 43 nm and is still rising (Fig. 22). Every value lies far below the experimental $148 \text{ W m}^{-1} \text{K}^{-1}$, and the reason is physical, not numerical: these cells are much shorter than the silicon phonon mean free path, so heat is carried *ballistically* and κ scales roughly with L . Recovering the bulk value requires either much longer cells (the standard $1/\kappa$ versus $1/L$ extrapolation of Schelling et al. [42] uses bars up to $0.3 \mu\text{m}$) or an equilibrium Green-Kubo calculation — both beyond an interactive tutorial. The lesson is general:

for high-conductivity crystals, finite-size effects dominate the apparent thermal conductivity and must be controlled before any comparison with experiment. (The same reverse-NEMD workflow is applied to germanium with a classical and a machine-learning interatomic potential in Appendix E.)

3.9 Tutorial 9: Phonon dispersion from MD

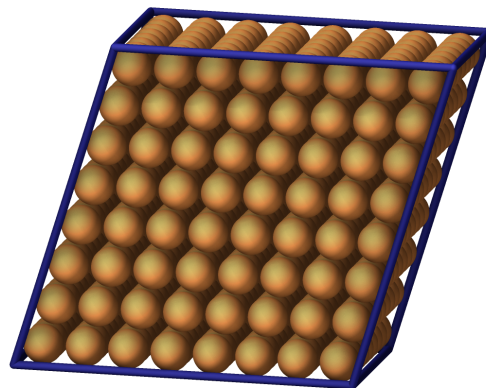


Figure 23. The simulated system (Tutorial 9): an $8 \times 8 \times 8$ supercell of the fcc-copper *primitive* cell, 512 atoms. Because the primitive cell is a 60° rhombohedron, the periodic box is triclinic rather than orthogonal. Using the one-atom primitive cell (instead of the four-atom cubic cell) means the dispersion comes out as exactly three branches, with no band folding.

Phonons—the quantized normal modes of a crystal lattice—govern the heat capacity, thermal conductivity, sound velocity, and thermal expansion of a solid. This tutorial computes the phonon dispersion of fcc copper directly from a finite-temperature MD run with the `fix phonon` command (PHONON package) [43]. As the crystal vibrates, the fix accumulates the displacement-displacement correlations $\langle u(\mathbf{q})u^*(\mathbf{q}) \rangle$ for every wave vector $\mathbf{q}$ commensurate with the simulation cell. By the fluctuation-dissipation theorem these correlations give the force-constant (dynamical) matrix $\tilde{\chi}(\mathbf{q}) = k_B T \langle u u^* \rangle^{-1}$; diagonalizing it at each $\mathbf{q}$ yields the phonon frequencies. Unlike the zero-temperature finite-difference method of Tutorial 10, this approach is fully anharmonic—the frequencies are those of the crystal at the simulation temperature, not of the static lattice.

Copper is used with the embedded-atom potential of Mishin et al. [40], so that the result can be compared with the classic neutron-scattering dispersion of Svensson, Brockhouse, and Rowe [44] (Fig. 24). Because copper is monatomic, its dispersion has only three acoustic branches and no optical ones; the diatomic silicon cell of Tutorial 10 will add optical branches.

3.9.1 Building the primitive cell and the map file

`fix phonon` needs the crystal to be expressed as a Bravais lattice with a K -atom basis, plus a *map file* that tags every atom with its unit-cell indices (l_1, l_2, l_3) and basis index k . The natural choice for an fcc metal is the *primitive* cell, which has a single atom ($K = 1$): the dispersion then comes out as exactly three branches with no band folding. The fcc primitive cell is a 60° rhombohedron, so the box is triclinic (Fig. 23). In `phonon.lmp` it is built directly with the `lattice` and `create_atoms` commands—no external structure file. The three primitive vectors $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ (in units of the lattice constant) are handed to `lattice custom`, and the simulation box is a prism whose edges are N copies of those vectors:

```
lattice custom ${a0} &
a1 0.7071068 0.0 0.0 &
a2 0.3535534 0.6123724 0.0 &
a3 0.3535534 0.2041241 0.5773503 basis 0.0 0.0 0.0
region box prism 0 $(v_N*v_lx) 0 $(v_N*v_ly) &
0 $(v_N*v_lz) $(v_N*v_ty) $(v_N*v_ty) $(v_N*v_tz) units box
create_box 1 box
create_atoms 1 box
```

Changing `a0`, `N`, or the mass adapts the script to a different fcc metal or a larger cell. The companion file `map.in` lists, for each atom, the unit cell it belongs to:

```
8 8 8 1 # nx ny nz atoms-per-cell
0 0 0 1 # cell (0,0,0), basis 0, atom-tag 1
1 0 0 2
...
```

Because the box edges *are* the primitive vectors, an atom's scaled (fractional) coordinates are exactly $(l_1, l_2, l_3)/N$, which is how `map.in` was generated; it must be regenerated to match if `N` is changed.

3.9.2 Accumulating the dynamical matrix

⌚ The supercell is built at the 300 K lattice constant ($3.6318 \text{ \AA}$, the thermally expanded value) and run in the NVT ensemble at constant volume:

```
fix 1 all nvt temp $T $T 0.2
fix 2 all phonon 20 25000 30000 map.in Cu
run 3030000
```

A fixed box is used deliberately: at this edge length the average pressure is ≈ 0 , and holding the cell shape fixed keeps the crystal's cubic symmetry exact. A barostat would relax the box, but on the tilted primitive cell it scales the edges while leaving the tilt factors fixed, distorting the cell and spoiling the dispersion. `fix phonon` samples the correlations every 20 steps after a 30 000-step equilibration wait, and writes the accumulated dynamical matrix to a binary file `Cu.bin.<step>` every 25 000 measurements. This is

the most demanding input in the set: the high-frequency longitudinal branch needs long sampling to converge, so the run is several million steps (about five minutes on eight threads, and much longer single-threaded). Run it with threads enabled and, ideally, in the background.

3.9.3 Post-processing with the phana tool

The binary file is read by `phana`, the standalone phonon analyzer bundled with LAMMPS [43]. It rebuilds the real-space force constants from the measured $\tilde{\chi}(\mathbf{q})$, which lets it evaluate the frequencies along *any* k -path, not only the commensurate points. The path and the output filename are supplied through the script `dispersion.in`:

```
phana Cu.bin.3030000 < dispersion.in
```

which writes `disp.dat` with the cumulative path coordinate and the three frequencies at each point. The script `phonon-plot.py` then plots the branches and overlays the experimental points from `Cu_exp.dat`, replacing the tool's default gnuplot output with a matplotlib figure.

3.9.4 Interpreting the dispersion

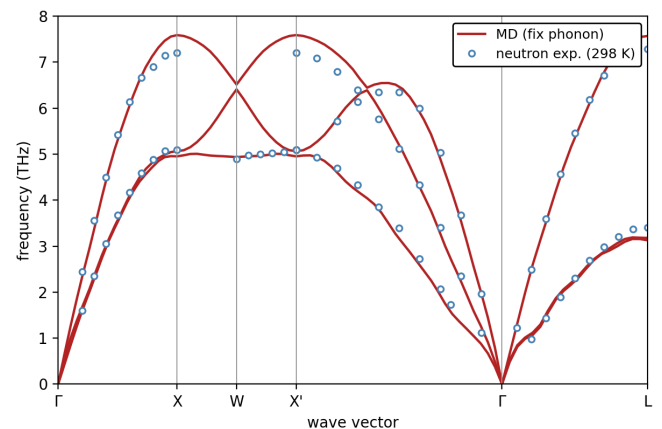


Figure 24. Phonon dispersion of fcc copper (Tutorial 9) along the Γ -X-W-X'- Γ -L path. The three acoustic branches computed from the molecular-dynamics fluctuations with `fix phonon` (red) are compared with room-temperature inelastic neutron-scattering data (circles) [44]. The two transverse branches and the long-wavelength slopes match the measurement closely; the longitudinal branch is a few percent stiff near the zone boundary, a typical and instructive limitation of an EAM potential.

Figure 24 shows the result. The two transverse acoustic branches agree with the neutron data across the whole zone, and near Γ all three branches reproduce the measured slopes—those slopes are the sound velocities, fixed by the same elastic constants computed in Tutorial 6. The longitudinal branch sits a few percent above experiment near the X and L zone boundaries (computed $\nu_L \approx 7.5 \text{ THz}$

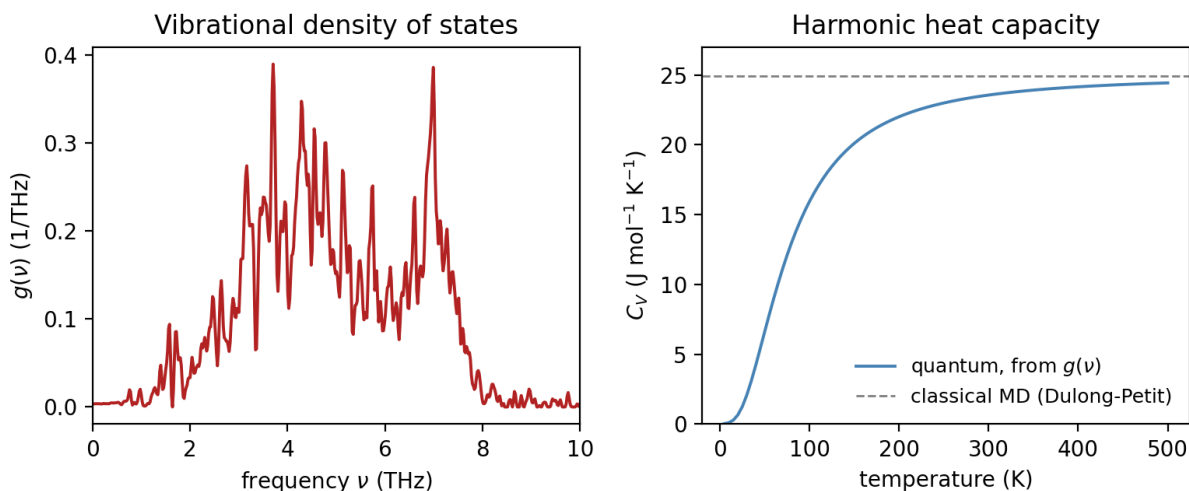


Figure 25. Left: vibrational density of states $g(\nu)$ of copper at 300 K, obtained by Fourier-transforming the velocity autocorrelation function (Tutorial 9, `vacfdos.lmp`). The spectrum rises as ν^2 at low frequency, peaks where the dispersion branches of Fig. 24 run flat (near 3.7 THz and 7 THz), and cuts off at the top of the longitudinal branch; the jitter reflects the finite 20 ps sampling window. Right: harmonic heat capacity computed from this spectrum. The quantum curve falls off below the Debye temperature and gives $C_V(300\text{ K}) = 23.6\text{ J mol}^{-1}\text{ K}^{-1}$ (experiment: $23.8\text{ J mol}^{-1}\text{ K}^{-1}$), whereas classical MD always yields the Dulong–Petit value $3R$ (dashed).

versus the measured 7.2 THz): the EAM potential is slightly too stiff for short-wavelength longitudinal modes. This is exactly why a phonon dispersion is such a sensitive test of an interatomic potential—it probes the curvature of the energy surface mode by mode, well beyond the elastic constants and the cohesive energy a potential is usually fitted to. The transverse branches and the low-frequency modes, which dominate the heat capacity and thermal transport, are captured well.

3.9.5 The density of states and the heat capacity

The dispersion shows the phonons mode by mode; their statistical summary is the vibrational density of states $g(\nu)$, and it can be measured from a plain MD trajectory without `fix phonon` at all. The key is the velocity autocorrelation function (VACF), whose Fourier transform is the vibrational spectrum [45]. The input `vacfdos.lmp` runs the same copper crystal for 20 ps in the NVE ensemble — a thermostat would damp the very vibrations being measured — and records the VACF with `compute vacf`, the same quantity used for liquid diffusion in Tutorial 13. In the solid the VACF does not decay to zero monotonically but rings like a struck bell, and `dos-plot.py` turns that ringing into $g(\nu)$ (Fig. 25, left); its peaks line up with the flat parts of the dispersion, where many modes share one frequency (van Hove singularities).

The spectrum also repairs a known blind spot of classical MD. By equipartition every classical oscillator carries $k_B T$, so the classical heat capacity is $3R$ per mole at *any* temperature — the Dulong–Petit law, which real solids obey only above

their Debye temperature. Weighting the measured $g(\nu)$ with the quantum harmonic-oscillator energy restores the correct behavior: `dos-plot.py` evaluates

$$C_V(T) = 3R \int_0^\infty g(\nu) \frac{x^2 e^x}{(e^x - 1)^2} d\nu, \quad x = \frac{h\nu}{k_B T}, \quad (4)$$

which vanishes at low temperature and approaches $3R$ from below (Fig. 25, right). At 300 K the result for copper is $23.6\text{ J mol}^{-1}\text{ K}^{-1}$, within one percent of the experimental $23.8\text{ J mol}^{-1}\text{ K}^{-1}$ — a quantum-corrected thermodynamic property from a purely classical simulation.

3.10 Tutorial 10: Zone-center phonons from the dynamical matrix

This tutorial complements Tutorial 9 with the zero-temperature, finite-difference route to the phonon frequencies. Instead of watching the crystal vibrate, each atom is displaced by a tiny amount, the forces induced on every atom are recorded, and the resulting $3N \times 3N$ dynamical (force constant) matrix is diagonalized with NumPy. The method is exact and harmonic, needs only one short, deterministic run, and—unlike `fix phonon`—requires no sampling and no external tool; the price is that it gives the frequencies at a single wave vector, the zone center Γ .

Silicon is used with the same Stillinger–Weber potential [36] as the elastic constants of Tutorial 6. Silicon has the diamond structure: an fcc lattice with a two-atom basis (Fig. 26). We build the two-atom *primitive* cell—a triclinic rhombohedron, exactly as in Tutorial 9 but with a second

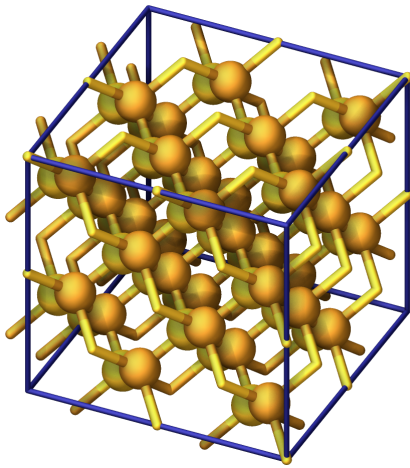


Figure 26. The diamond structure of silicon (Tutorial 10): an fcc lattice with a *two-atom basis*, i.e. two interpenetrating fcc sublattices (drawn in two shades of the same color—both are silicon) joined by tetrahedral bonds. It is the second basis atom that gives rise to the three optical branches; the monatomic copper of Tutorial 9 has none.

basis atom at $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4})$ —so the dynamical matrix is just 6×6 and yields six branches: three acoustic (zero frequency at Γ) and three optical.

3.10.1 Building and minimizing the cell

Open `dynmat.lmp`. The primitive cell is built with `lattice custom` as in Tutorial 9, now with two basis atoms, and is relaxed to its stress-free geometry:

```
atom_modify map array # dynamical_matrix needs an atom map
...
lattice custom ${a0} a1 0.7071068 0 0 ... &
  basis 0.0 0.0 0.0 basis 0.25 0.25 0.25
...
fix 1 all box/relax iso 0.0 vmax 0.001
minimize 1.0e-12 1.0e-12 1000 10000
```

The minimization is essential: a finite-difference second derivative is only meaningful about a configuration where the forces are (numerically) zero. Note the `atom_modify map array` line—`dynamical_matrix` needs to look atoms up by their global ID, which requires an atom map.

3.10.2 Computing the dynamical matrix

A single command then builds the matrix by finite difference:

```
dynamical_matrix all eskm 1.0e-6 file dynmat.dat binary no
```

Each atom is displaced by 10^{-6} Å (`eskm` selects the mass-weighted form $D_{ij} = \Phi_{ij}/\sqrt{m_i m_j}$), and the matrix is written to `dynmat.dat` as plain text. The `eskm` output is in units of fs^{-2} , chosen so that $\nu = \sqrt{\lambda}/2\pi$ of each eigenvalue λ comes out directly in THz.

3.10.3 Diagonalizing and interpreting the result

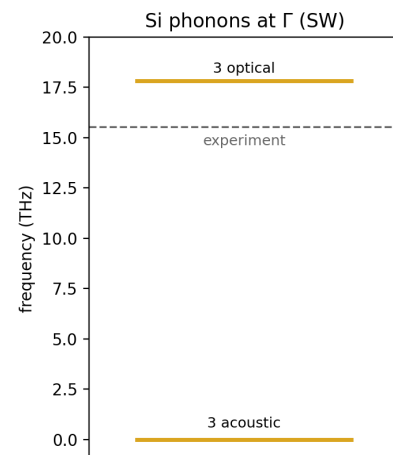


Figure 27. Zone-center (Γ) phonon frequencies of silicon (Tutorial 10) from the Stillinger–Weber dynamical matrix: three acoustic modes at zero and three degenerate optical modes. The dashed line is the experimental optical frequency, 15.53 THz; the potential overestimates it by about 15 %.

The script `phonon-gamma.py` reads `dynmat.dat`, reshapes it into the 6×6 matrix, and diagonalizes it. The six eigenvalues give three acoustic frequencies that are zero to numerical precision—the acoustic sum rule, a direct consequence of translational invariance—and three degenerate optical frequencies at 17.83 THz (Fig. 27). The experimental zone-center optical mode of silicon is 15.53 THz, so the Stillinger–Weber potential overestimates it by roughly 15 %: the same lesson as the copper longitudinal branch in Tutorial 9, that a potential fitted to elastic and cohesive properties is not guaranteed to get the high-frequency phonons right.

The three acoustic branches vanish at Γ but rise linearly away from it, with slopes equal to the sound velocities—and those, in turn, are fixed by the elastic constants computed in Tutorial 6. That long-wavelength limit is exactly the low- q region of the dispersion seen in Tutorial 9; the optical modes, absent for a one-atom basis, are the genuinely new feature here.

3.11 Tutorial 11: Radiation damage cascade

When an energetic particle—a fast neutron, an ion, or a recoiling nucleus—strikes a crystal, it knocks an atom (the *primary knock-on atom*, PKA) far off its lattice site. The PKA in turn displaces its neighbors, which displace theirs, and within a fraction of a picosecond a small region of the crystal is transiently melted. As that region recools, most of the displaced atoms drop back onto lattice sites, but a few vacancies and self-interstitial atoms—*Frenkel pairs*—survive. This *displacement cascade* is the elementary event of radiation damage

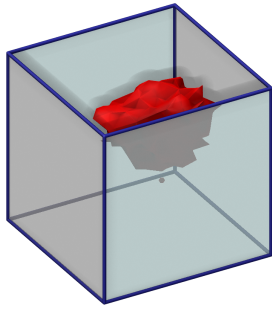


Figure 28. The disordered core of the cascade at peak damage (Tutorial 11), about 0.28 ps after the 2 keV primary knock-on atom is launched. The translucent red surface is the alpha-shape hull (built with `fix graphics/chunk`) wrapping the ~1400 atoms knocked more than $0.7 \text{ \AA}$ off their lattice sites; it marks the briefly molten region inside the $85.7 \text{ \AA}$ simulation cell.

and governs how structural materials degrade in fission and fusion reactors [46]. This tutorial simulates a 2 keV cascade in bcc iron with the embedded-atom potential of Mendelev *et al.* [33] and counts the surviving Frenkel pairs directly while the simulation runs.

3.11.1 Building and equilibrating the target crystal
`cascade.lmp` builds a $30 \times 30 \times 30$ bcc supercell (54 000 atoms, an $85.7 \text{ \AA}$ cube) and equilibrates it at 300 K:

```
lattice bcc 2.8553
region box block 0 30 0 30 0 30
create_box 1 box
create_atoms 1 box
fix eq all nvt temp 300 300 0.1
run 3000
```

The box is held at a fixed size rather than under a barostat. The defect analysis below compares the atoms against a *perfect* bcc reference lattice, so the simulation lattice constant must not drift: a fixed box keeps that reference valid throughout (under a barostat one would instead let the reference follow the box with `compute_modify rescale yes`).

3.11.2 Launching and running the cascade

⌚ After equilibration, one atom near the center of the box is selected as the PKA and given a recoil velocity. Its kinetic energy $E = \frac{1}{2}mv^2 = 2 \text{ keV}$ fixes the speed; the direction is chosen off any low-index axis so the cascade does not channel:

```
region pkareg sphere ${cx} ${cx} ${cx} 0.5 units box
group pka region pkareg
velocity pka set 140.4 421.2 702.0 units box
```

Two features make the cascade tractable. The interior of the box is integrated in the NVE ensemble, while a shell of atoms around the border is coupled to a 300 K Langevin thermo-

stat that drains the cascade heat, mimicking the surrounding bulk:

```
fix nve all nve
fix dtr all dt/reset 1 NULL 0.001 0.1
fix lang border langevin 300 300 0.1 12345
```

`fix dt/reset` adapts the timestep so that no atom moves more than $0.1 \text{ \AA}$ per step: during the violent early phase the step shrinks far below the nominal 1 fs, then grows back as the system cools. At the peak of the cascade (Fig. 28) several hundred atoms are displaced and a small region is effectively molten. At 2 keV the embedded-atom potential alone is adequate; substantially more energetic recoils additionally require a ZBL short-range repulsion (blended in with `pair_style hybrid/overlay`) to describe the hardest collisions.

3.11.3 Counting Frenkel pairs on the fly

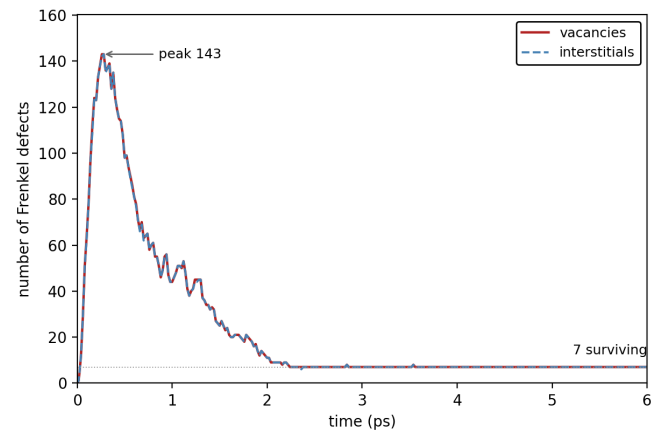


Figure 29. Number of Frenkel defects versus time during the 2 keV iron cascade (Tutorial 11), counted live with `compute frenkel`. The vacancy and interstitial counts coincide exactly—every interstitial leaves a vacancy behind. The ballistic peak of 143 Frenkel pairs at 0.28 ps recombines within ~ 2.5 ps to the seven defects that survive to the end of the run.

Traditionally the surviving defects are counted *after* the run with an external Wigner–Seitz analysis, for example OVITO’s defect modifier. Here it is done *during* the run with `compute frenkel` (EXTRA-COMPUTE package) [47], which assigns every atom to the nearest site of the reference lattice: an empty site is a vacancy, a doubly occupied one holds an interstitial. The compute’s global vector returns the number of vacancies, interstitials, and irregular sites, so `c_fr[1]` is the live Frenkel-pair count and is logged with `fix ave/time`:

```
compute fr all frenkel
fix rec all ave/time 20 1 20 v_t c_fr[1] c_fr[2] file cascade.dat
```


Plotting `cascade.dat` with `cascade-plot.py` gives the defect history of Fig. 29. The ballistic phase reaches a peak of 143 Frenkel pairs at 0.28 ps; as the molten core recrystallizes most pairs recombine, and after about 2.5 ps the count is flat at the surviving value for the remaining 12 ps of the run. The vacancy and interstitial curves lie exactly on top of one another, a built-in consistency check.

3.11.4 Surviving defects and the NRT model

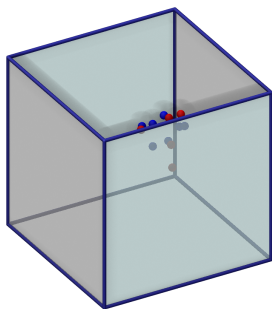


Figure 30. The seven Frenkel pairs that survive the cascade (Tutorial 11), drawn with `dump image` from the `compute frenkel` markers: blue spheres are vacancies, red spheres interstitials. They form a compact cluster near the cascade center; the individual iron atoms are omitted and the cell is shown as a transparent box.

The cascade leaves seven surviving Frenkel pairs (Fig. 30), clustered where the PKA deposited its energy. The exact number fluctuates by a few from run to run—it is sensitive to the seed and the thread count—but it is always far below the ballistic peak: in-cascade recombination annihilates the great majority of the defects. This is the central result of cascade modeling, and it can be compared with the Norgett–Robinson–Torrens (NRT) standard estimate [48],

$$N_{\text{NRT}} = \frac{0.8 E_{\text{dam}}}{2 E_d}, \quad (5)$$

with damage energy $E_{\text{dam}} \approx 2 \text{ keV}$ and displacement threshold $E_d \approx 40 \text{ eV}$ for iron, which gives $N_{\text{NRT}} \approx 20$. The molecular-dynamics cascade thus produces only a fraction—here about a third—of the stable defects the NRT model predicts. Capturing this *defect-production efficiency*, and the efficient on-the-fly counting that makes long cascade runs practical, is exactly why atomistic simulation is needed to predict radiation damage [46]. The two snapshots are rendered from the saved configurations by `peakviz.lmp` and `defectviz.lmp` with the built-in `dump image` renderer.

Figure 30 hints that the survivors are not randomly scattered, and `compute frenkel` can quantify this: besides the global counts it provides a *local* array with one row per defect cluster — a signed size (negative for vacancy clusters, positive for interstitial clusters) and the cluster center — which is also what `dump image` uses to place the defect

markers. The solution input `clusters.lmp` re-analyzes the saved final configuration and writes the cluster table with `dump local`: the seven vacancies survive as seven *isolated* monovacancies, but the seven interstitials occupy only five clusters — two of them are di-interstitials. This asymmetry is characteristic of cascade damage: interstitials are carried away from the core by replacement-collision sequences and bind strongly to each other, while the less mobile vacancies remain behind, closer to the cascade center. In reactor steels it is exactly these interstitial clusters that grow into dislocation loops under prolonged irradiation.

3.12 Tutorial 12: Melting point determination

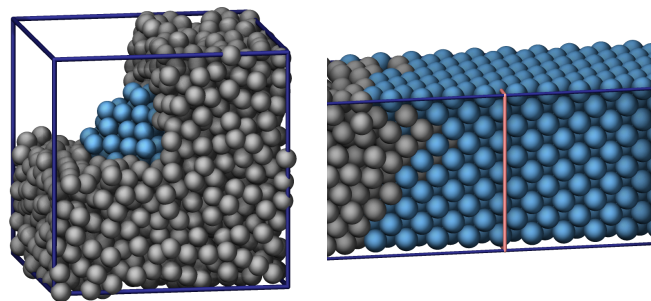


Figure 31. The atomic structures behind the two melting-point methods of Tutorial 12. Left: the seeded melt (`nucleate.lmp`), with the top-left atoms of the liquid cut away to expose the crystalline seed (steelblue) that stays ordered while the rest of the cell is molten (gray). Right: the two-phase coexistence cell (`velocity.lmp`) held below T_m , where the solid (steelblue) has grown past the pale-red frame — the original solid–liquid interface location — into the liquid (gray). Coloring is derived from CNA classification.

The melting temperature T_m is one of the most fundamental thermodynamic properties of a material. Classical interatomic potentials reproduce it with very mixed success — from a percent or two for potentials fitted with the liquid and the solid–liquid interface in mind, to tens of percent for general-purpose ones — so knowing how well a potential performs matters for any simulation at elevated temperature. This tutorial determines T_m for fcc aluminum, using the Finis–Sinclair EAM potential of Mendelev *et al.* [49] (fitted for exactly this purpose), with two complementary methods: a quick one-phase heating/cooling cycle that *brackets* T_m , and the more rigorous two-phase coexistence method [50] that pins it down. The aluminum potential of Zhou *et al.* [20] used for other calculations in this tutorial severely underestimates the melting point ($\approx 30\%$ too low).

3.12.1 Heating and cooling: the hysteresis loop

Open `hysteresis.lmp`. It heats a bulk fcc aluminum crystal from 300 K to 1500 K in the NPT ensemble at zero pressure,

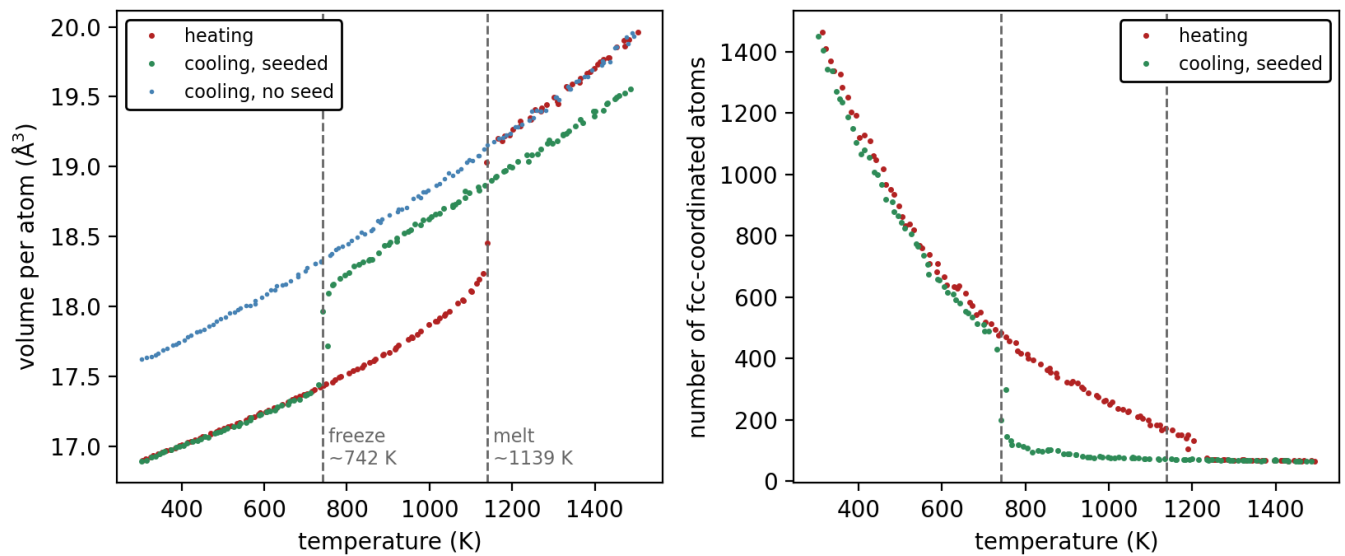


Figure 32. Heating and cooling of aluminum (Tutorial 12). *Left:* volume per atom while heating a crystal through melting (red) and cooling the melt back down without a seed (blue). A defect-free crystal superheats before it melts (dashed line, ~ 1140 K) and the melt supercools so far that it stays a glass down to 300 K; the branches do not coincide, so the true T_m lies inside this wide loop. The green points are a separate cooling run that contains a crystalline *seed* (advanced section): it refreezes near 740 K (second dashed line) instead of remaining a glass. *Right:* the number of fcc-coordinated atoms for that seeded run — the seed holds a small crystalline core through the melt and templates regrowth of the whole cell when the seeded melt finally freezes.

then cools it back, recording the volume per atom at each temperature:

```
fix hot all npt temp 300.0 1500.0 0.2 iso 0.0 0.0 2.0
fix rec all ave/time 50 20 1000 v_temp v_vat file heat.dat
run 120000
...
fix cold all npt temp 1500.0 300.0 0.2 iso 0.0 0.0 2.0
```

The volume jumps when the crystal melts and would jump back when the liquid freezes. But both transitions require *nucleating* a new phase, so neither occurs at T_m : on heating the crystal superheats and melts near 1140 K, while on cooling the melt *supercools* so strongly that, in a cell this small, it does not recrystallize at all down to 300 K. The two branches enclose a wide hysteresis loop (Fig. 32) whose width depends on the heating rate but barely on system size — cells from 256 to 4000 atoms all melt near 1150 K and none recrystallize — so the loop only *brackets* T_m . That is exactly the motivation for the two-phase method. (The green points and the right panel of Fig. 32 come from a variant run with a crystalline *seed* — a block of atoms restrained to their lattice sites — that does make the melt refreeze; that is the subject of the advanced section at the end of this tutorial.)

Although the loop only brackets the melting point, its solid branch carries quantitative physics for free: the gentle slope *is* the thermal expansion of the crystal. Converting the volume per atom back to a lattice constant,

$a(T) = (4V_{\text{at}})^{1/3}$, and fitting a straight line between 300 K and 900 K (`plot.py` prints the fit and draws it on the solution figure) gives $a(300\text{ K}) = 4.072\text{ Å}$ — 0.7% above the zero-kelvin value of Tutorial 13 — and a linear expansion coefficient $\alpha = (1/a)(da/dT) \approx 24.8 \times 10^{-6}\text{ K}^{-1}$, within 8% of the experimental $23.1 \times 10^{-6}\text{ K}^{-1}$. Thermal expansion is also why finite-temperature calculations should be set up at the temperature's own lattice constant: Tutorial 9 builds its 300 K phonon supercell at the thermally expanded constant for exactly this reason.

3.12.2 Building the two-phase coexistence cell

The way around superheating and supercooling is to start from a configuration that *already* contains a solid–liquid interface, so that no nucleation is required. Open `coexist.lmp`. It first equilibrates an elongated crystal *below* T_m so it stays solid, then creates an interface by melting the upper half while a second thermostat holds the lower half crystalline:

```
fix baro all nph z 0.0 0.0 2.0
fix hot top langevin 1400.0 1400.0 0.1 11111 # melt the upper half
fix cold bot langevin 800.0 800.0 0.1 22222 # keep lower half solid
```

Heat flows across the interface, so the two halves are thermostatted *separately* with `fix langevin`. Two choices about the barostat matter, both because the cell must stay square in cross-section (the cubic crystal sets it) while only the long z axis is free. During equilibration x and y

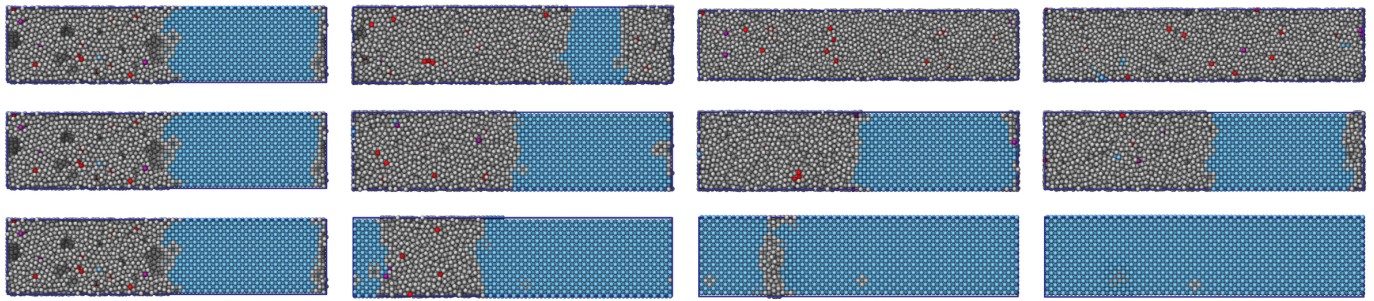


Figure 33. The two-phase method at work: the solid-liquid interface of the coexistence cell (`evolution.lmp`), held at three fixed temperatures and shown at four times each (left to right). Liquid is on the left (gray, disordered), solid on the right (steelblue, fcc), colored by common-neighbor analysis of the inherent structure. *Top*, above T_m (970 K): the solid melts and the interface recedes to the right. *Middle*, near T_m (930 K): the interface barely moves. *Bottom*, below T_m (890 K): the solid grows and consumes the liquid. T_m is the temperature at which the interface neither advances nor retreats.

are coupled (`couple xy`) so they relax *together* to the lattice the solid wants; during melting only z is barostatted, so the less-dense liquid expands along the interface normal without straining — and so melting — the solid cross-section. The result is a clean planar interface (Fig. 31, right), saved to a restart file for the next step.

3.12.3 The melting temperature from interface motion

A two-phase cell has a simple, decisive property: the interface moves only in the direction that lowers the free energy. Below T_m the solid is the stable phase, so it grows and consumes the liquid; above T_m it melts; exactly at T_m the interface is stationary (Fig. 33). Open `velocity.lmp`, which reads the cell back and holds it at a series of fixed temperatures, then reports how much of it is crystalline:

```
fix th all langevin $T $T 0.1 4321
run 25000
minimize 1.0e-8 1.0e-8 2000 20000 # inherent structure
compute cna all cna/atom 3.6
```

The minimization is essential, and worth dwelling on. *Near* T_m the instantaneous local structure of hot solid and hot liquid is almost indistinguishable: per-atom common-neighbor analysis, Steinhardt bond-order parameters, and Voronoi cell volumes all give heavily overlapping distributions, so none of them counts the solid reliably on the fly. Quenching to the *inherent structure* — a brief energy minimization that removes thermal vibration without moving the interface — collapses each phase to a sharp signature, and now `compute cna/atom` separates them cleanly (the structure-analysis appendix F compares these classifiers in more detail). Run `coexist-plot.py` to plot the crystalline fraction against temperature (Fig. 34): it is near one at low temperature (the solid grew), zero once the cell has melted,

and crosses one half at the melting point,

$$T_m \approx 920 \text{ K.}$$

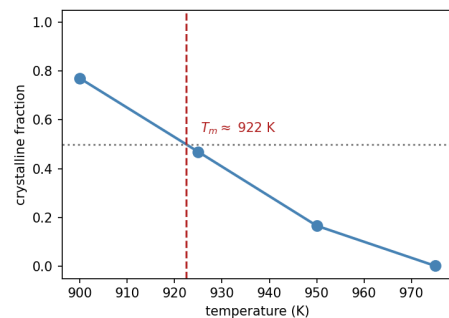


Figure 34. Crystalline fraction of the two-phase cell after holding it at each temperature (`velocity.lmp`). Below the melting point the solid grows and the fraction rises toward one; above it the cell melts and the fraction falls toward zero. The curve crosses one half at $T_m \approx 920 \text{ K}$ for the Mendelev aluminum potential, close to the experimental 933 K.

This short scan already lands within a couple of percent of the experimental 933 K, and a converged calculation reproduces it closely. That agreement is no accident: the Mendelev potential was fitted to liquid structure and solid-liquid coexistence, so getting T_m right is exactly what it is good at. Not every potential is — a general-purpose aluminum model can miss by tens of percent — so measuring T_m this way is always worth doing before trusting a potential at high temperature. The hysteresis loop bracketed T_m ; the two-phase interface measured it.

3.12.4 Advanced: seeding the supercooled melt

The one-phase cooling run never refroze because a small, defect-free periodic cell offers no site for a crystal to nucleate. We can supply one. Open `nucleate.lmp`. It plants a small spherical *seed* of atoms at the box center and restrains them

to their lattice sites, so the cluster stays crystalline through the melt (Fig. 31, left) and can template regrowth on cooling.

The restraint is built by hand so that its target co-dilates with the NPT box. `fix store/state` records the seed's initial *fractional* coordinates once; three atom-style variables then rebuild the target position each step from the current box and form a restoring force, which `fix addforce` applies:

```
fix seed0 seed store/state 0 xs ys zs
variable txx atom xlo+f_seed0[1]*lx # target_x = xlo + xs0*lx ...
variable dxx atom x-v_txx # displacement from the target
variable fxx atom -v_keff*v_dxx # restoring force (3 components)
fix pull seed addforce v_fxx v_fyy v_fzz
```

Storing the *scaled* coordinates is the trick: it keeps the seed at the current equilibrium spacing at every temperature, so it does not strain against the thermally expanded hot box the way a fixed-coordinate `fix spring/self` would. Monitoring the seed's deviation from its target (here it stays within 0.16 Å of its lattice site even in the 1400 K liquid) lets you keep the force strong but not too strong.

This time the melt *does* refreeze: on cooling the volume drops back onto the crystal branch and the fcc-coordinated count jumps from the seed alone to nearly the whole cell (Fig. 32, green points and right panel). But it refreezes only near 740 K, far below T_m . A small seed has a curved interface, which depresses its local melting point (the Gibbs–Thomson effect), so it stays *sub-critical* near T_m and becomes an effective template only once the undercooling is large. A seed therefore cures the runaway supercooling, but cannot by itself locate T_m — only the flat interface of the two-phase cell, which carries no curvature penalty, stays in balance at T_m .

3.13 Tutorial 13: Diffusion in a liquid metal

Diffusion is the elementary process by which atoms migrate through a material, and it underlies solidification, homogenization, creep, and precipitation. In a liquid the atoms are mobile enough that the self-diffusion coefficient D can be read directly from how far they wander during a single short simulation. This tutorial measures D for liquid aluminum at several temperatures from the mean-squared displacement

⌚ The scan above is deliberately *short* — a small cell, four temperatures, and brief runs — so it finishes in a few minutes on a single processor core. It therefore locates T_m only to within a few tens of kelvin. A converged value needs a larger cross-section, a finer temperature grid, and longer runs at each temperature: many times more work, best done in parallel (for example with MPI across many cores), where it still takes a good while to complete.

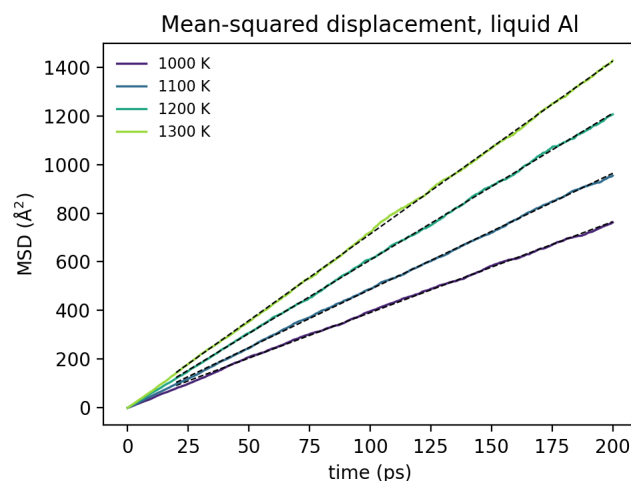


Figure 35. Mean-squared displacement of liquid aluminum at four temperatures (Tutorial 13). After a sub-picosecond ballistic onset each curve grows linearly in time — the diffusive regime — with a slope that steepens with temperature. The self-diffusion coefficient is obtained from that slope through the Einstein relation; the dashed lines are the linear fits.

(MSD), then fits the temperature dependence to an Arrhenius law to extract an activation energy. The Finnis–Sinclair EAM potential of Mendelev *et al.* [49] is used — the same potential as in Tutorial 12, chosen because it was fitted with the liquid in mind and places the melting point ($T_m \approx 933$ K) close to experiment.

A liquid is studied deliberately. In a perfect crystal an atom moves only by an occasional, thermally activated hop into a neighboring vacancy, far too rare to capture in an interactive run; measuring solid-state diffusion needs either very long simulations or dedicated rare-event methods (the migration barrier itself is the natural target of the nudged-elastic-band calculation of Appendix C). In the liquid every atom diffuses continuously, so a clean MSD accumulates in tens of picoseconds.

3.13.1 Melting and equilibrating the liquid

🔧 Open `diffusion.lmp`. For each temperature it builds an fcc aluminum crystal (2048 atoms), melts it by holding it well above T_m at 2000 K under a barostat so that all crystalline order is lost and the liquid density relaxes, and then brings it to the target temperature at zero pressure:

```
velocity all create 2000.0 5783 mom yes rot yes dist gaussian
fix melt all npt temp 2000.0 2000.0 0.2 iso 0.0 0.0 1.0
run 10000
unfix melt
fix equi all npt temp ${T} ${T} 0.2 iso 0.0 0.0 1.0
```


All four target temperatures (1000 K to 1300 K) lie above T_m , so the melt stays liquid throughout and no recrystallization can bias the measurement. The production run is then carried out at *fixed volume* with a gentle Nose–Hoover thermostat:

```
reset_timestep 0
fix nvt all nvt temp ${T} ${T} 0.2
```

Pinning the temperature with `fix nvt` during production matters more than it might seem. Handing the equilibrated liquid straight to a constant-energy (`fix nve`) run lets small residual imbalances convert potential energy into heat, so the production temperature can drift tens of kelvin away from the target and flatten the Arrhenius trend. A weakly coupled thermostat holds each run at its intended temperature while perturbing the diffusion negligibly.

3.13.2 The mean-squared displacement

The diffusion coefficient follows from the mean-squared displacement and the Einstein relation,

$$\text{MSD}(t) = \langle |\mathbf{r}_i(t) - \mathbf{r}_i(0)|^2 \rangle, \quad D = \lim_{t \rightarrow \infty} \frac{\text{MSD}(t)}{2dt}, \quad (6)$$

with $d = 3$ the number of spatial dimensions. `compute msd` accumulates the MSD from unwrapped coordinates, and the `com yes` keyword subtracts any drift of the center of mass:

```
compute msd all msd com yes
fix outmsd all ave/time 100 1 100 c_msd[4] file msd_${T}.dat
```

After a sub-picosecond ballistic transient the MSD grows linearly with time (Fig. 35). The script `plot.py` fits a straight line to the linear portion (excluding the first 20 ps) and divides the slope by $2d = 6$ to obtain D . At 1200 K, for instance, $D \approx 1.0 \times 10^{-8} \text{ m}^2 \text{ s}^{-1}$.

The same coefficient can be obtained from the Green–Kubo integral of the velocity autocorrelation function (VACF), $D = \frac{1}{d} \int_0^\infty \langle \mathbf{v}(0) \cdot \mathbf{v}(t) \rangle dt$. The solution input `solution/diffusion-vacf.lmp` computes the VACF with `compute vacf`; its negative “cage” rebound is a hallmark of a dense liquid, and integrating over the first picosecond — before a single-origin VACF becomes dominated by statistical noise — gives D in agreement with the Einstein value. The VACF can also be plotted and integrated interactively in LAMMPS–GUI (Appendix A). The MSD route is used here as the primary method because it self-averages and converges more robustly.

3.13.3 Temperature dependence: the activation energy

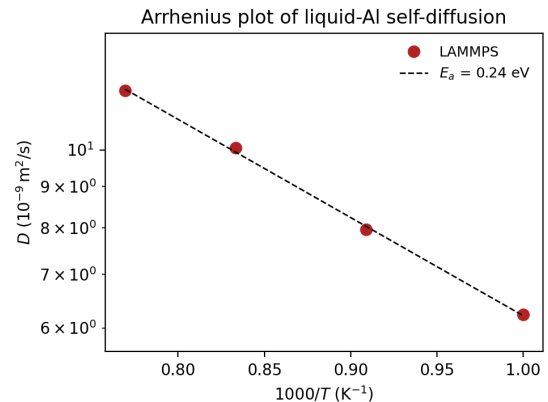


Figure 36. Arrhenius plot of the liquid-aluminum self-diffusion coefficient (Tutorial 13). Plotting $\ln D$ against $1/T$ gives a straight line whose slope is $-E_a/k_B$; the fit yields an activation energy $E_a \approx 0.24 \text{ eV}$.

Repeating the measurement at several temperatures maps out $D(T)$. Like most thermally activated processes, liquid diffusion follows an Arrhenius law,

$$D(T) = D_0 \exp\left(-\frac{E_a}{k_B T}\right), \quad (7)$$

so that $\ln D$ is linear in $1/T$ with slope $-E_a/k_B$. Plotting the four coefficients this way (Fig. 36) gives a straight line: the self-diffusion coefficient rises from about $6 \times 10^{-9} \text{ m}^2 \text{ s}^{-1}$ at 1000 K to $1.2 \times 10^{-8} \text{ m}^2 \text{ s}^{-1}$ at 1300 K, with an activation energy $E_a \approx 0.24 \text{ eV}$ (23 kJ mol^{-1}). Both the magnitude and the activation energy are comparable to experimental and first-principles estimates for liquid aluminum, a reassuring check on the potential and the method. As always for a small, brief simulation, the values should be converged against system size and run length before being quoted for research — but the linearity of both the MSD and the Arrhenius plot is already clear.

3.14 Tutorial 14: Chemical short-range order in a high-entropy alloy

High-entropy alloys combine four or more principal elements in near-equal proportions on a single crystal lattice [51]. The archetype studied here, equimolar CoCrFeNi, forms a face-centered cubic solid solution. Such alloys are often idealized as *random* solutions, but at equilibrium the chemical species are not arranged randomly: energetically favorable neighbor pairs occur more often than chance, unfavorable ones less often. This *chemical short-range order* (SRO) has been imaged directly by electron microscopy [52] and influences stacking-fault energies, dislocation motion, and strength. In this tutorial the equilibrium SRO of CoCrFeNi is computed with a

hybrid molecular-dynamics / Monte-Carlo simulation, quantified with the Warren–Cowley parameters [53] — and, in the second part, the same simulation is repeated with three different published potentials, with the instructive result that they give three qualitatively different answers.

Plain MD cannot equilibrate the chemical order of a solid solution: at the temperatures of interest, atoms exchange lattice sites only through rare vacancy hops, just as solid-state diffusion was out of reach in Tutorial 13. The Monte-Carlo part of this tutorial sidesteps the kinetics entirely by swapping the chemical identities of atom pairs, sampling the equilibrium distribution directly [54].

3.14.1 Building the random solid solution

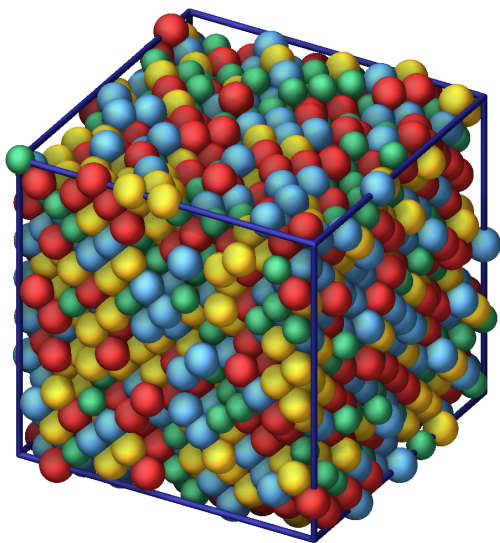


Figure 37. The equimolar CoCrFeNi solid solution after thermalization at 700 K (Tutorial 14; 2048 atoms). The four elements — Co (yellow), Cr (red), Fe (blue), Ni (green) — share one fcc lattice in a random arrangement; the same colors are used in all figures of this tutorial. Rendered with LAMMPS (`solution/geomviz.lmp`).

Open `sro.lmp`. An $8 \times 8 \times 8$ fcc supercell (2048 atoms) is created with all sites initially of type 1, and exactly one quarter of them is then reassigned to each of the other three types with the `type/subset` style of the `set` command:

```
set group all type/subset 2 512 12345
set type 1 type/subset 3 512 23456
set type 1 type/subset 4 512 34567
```

The first command picks 512 of the 2048 atoms at random and makes them type 2; the next two convert 512 of the remaining type-1 atoms to type 3 and then to type 4, leaving 512 type-1 atoms: an exactly equimolar, randomly arranged CoCrFeNi crystal. The interactions use the EAM po-

tential of Farkas and Caro [55], which was designed for fcc high-entropy alloys of Fe, Ni, Cr, Co, and Cu:

```
pair_style eam/alloy
pair_coeff * * FeNiCrCoCu-heafixed.setfl Co Cr Fe Ni
```

The five-element potential file is mapped onto only four atom types — copper is simply not used. A short 2 ps NPT run then thermalizes the atomic positions at 700 K and relaxes the lattice constant, while the chemical arrangement is still frozen at random; a snapshot of this state (Fig. 37) is saved to `random.dump` as the disordered reference.

3.14.2 Hybrid MD/MC annealing

The chemical order is now equilibrated by interleaving the MD integration with Monte-Carlo moves that swap the chemical identities of randomly chosen pairs of atoms.

`fix atom/swap` performs such swaps for one pair of atom types, so six instances cover all element combinations of the quaternary alloy:

```
fix s12 all atom/swap 40 80 701 ${T} types 1 2
fix s13 all atom/swap 40 80 702 ${T} types 1 3
fix s14 all atom/swap 40 80 703 ${T} types 1 4
fix s23 all atom/swap 40 80 704 ${T} types 2 3
fix s24 all atom/swap 40 80 705 ${T} types 2 4
fix s34 all atom/swap 40 80 706 ${T} types 3 4
```

Every 40 MD steps each `fix` attempts 80 swaps; a proposed swap is accepted or rejected with the Metropolis criterion at the run temperature, so the composition stays exactly fixed while the *arrangement* of the elements random-walks toward thermal equilibrium. The MD steps in between let the lattice relax around each new chemical configuration and keep sampling the thermal vibrations.

⌚ Each attempted swap requires a full potential-energy evaluation, so the Monte-Carlo moves — not the MD integration — dominate the cost of this run (about ten minutes on a few CPU cores). This is also one reason the inexpensive EAM potential is used here: the same protocol with a MEAM potential runs an order of magnitude longer.

Each `fix` reports its attempted and accepted swap counts; the `print` commands at the end of the script show that at 700 K between 64 % and 86 % of the proposed swaps are accepted — swapping species in this alloy costs little energy, a first hint that the potential sees a nearly ideal solution.

3.14.3 Measuring the Warren–Cowley parameters

The degree of order is quantified by the Warren–Cowley parameter of each element pair [53],

$$\alpha_{AB} = 1 - \frac{P_{AB}}{C_B}, \quad (8)$$

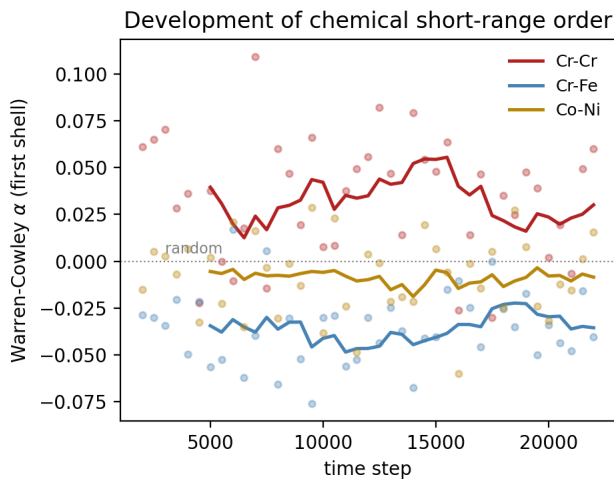


Figure 38. Development of chemical short-range order in CoCrFeNi during the hybrid MD/MC run at 700 K (Tutorial 14). Warren-Cowley parameters of three representative pairs, computed by `plot.py` for every trajectory snapshot. Starting from the random arrangement ($\alpha \approx 0$) the alloy develops a weak but persistent order: chromium avoids chromium ($\alpha_{\text{CrCr}} > 0$) and pairs preferentially with iron ($\alpha_{\text{CrFe}} < 0$), while Co-Ni remains nearly ideal. The scatter of the curves reflects the finite system size.

where P_{AB} is the probability that a first-neighbor of an A atom is of type B , and c_B is the overall concentration of B . For a random alloy $P_{AB} = c_B$ and all $\alpha_{AB} = 0$; $\alpha_{AB} > 0$ means A avoids B in its first shell, $\alpha_{AB} < 0$ means A attracts B . The script `plot.py` computes the full 4×4 matrix of first-shell parameters for every snapshot of `anneal.dump` and averages the converged tail of the run. The result (Fig. 38) is a *weak but systematic* short-range order: $\alpha_{\text{CrCr}} \approx +0.03$ (chromium avoids chromium), $\alpha_{\text{CrFe}} \approx -0.03$ (chromium prefers iron neighbors), with all other pairs close to ideal. This pattern agrees qualitatively with density-functional-theory-based Monte-Carlo studies of this alloy family, which find Cr-Cr avoidance as the dominant SRO motif — although the DFT $\alpha_{\text{CrCr}} \approx 0.2$ – 0.3 at comparable temperatures is markedly stronger [56]. The energy gained by the ordering, recorded in `energy.dat`, is correspondingly small, only a few meV per atom.

With 2048 atoms the per-snapshot statistical noise of α_{CrCr} is comparable to the signal itself (Fig. 38), which is why `plot.py` averages over the final snapshots of the trajectory. Doubling the box or the sampling time reduces the scatter accordingly — a useful exercise.

3.14.4 One question, three potentials, three answers
How much of the result above should be trusted? Short-range order is driven by tiny energy differences between

Table 2. Mixing enthalpy of the six equimolar fcc binary alloys of Co, Cr, Fe, and Ni (in meV per atom) computed with `mixing.lmp` for the three potentials used in this tutorial, compared with values derived from experimental data (Miedema model [57]). Entries with the wrong sign or with magnitudes off by an order of magnitude are the direct cause of the failures shown in Fig. 39.

Pair	Exp. [57]	MEAM [58]	MEAM [59]	EAM [55]
Co--Cr	-41	+44	-144	+2
Co--Fe	-10	-55	-277	+1
Co--Ni	0	-18	-126	+1
Cr--Fe	-10	+80	-74	+3
Cr--Ni	-73	+1	-114	0
Fe--Ni	-21	-48	-146	+2

competing chemical arrangements — exactly the kind of quantity a classical potential reproduces only if it was fitted with alloy thermodynamics in mind. Before trusting *any* Monte-Carlo prediction of chemical order, the potential should be subjected to a simple sanity check that takes less than a minute: compute the mixing enthalpy

$$\Delta H_{\text{mix}} = E_{A_{50}B_{50}} - \frac{1}{2}(E_A + E_B) \quad (9)$$

of every equimolar binary from box-relaxed energy minimizations, and compare with experiment-based reference values. The input `mixing.lmp` does this for all six binaries of CoCrFeNi. Table 2 shows the result for the Farkas-Caro EAM used above and for two more recent, more sophisticated MEAM potentials [58, 59] that also cover the CoCrFeNi system, alongside the Miedema-model reference values [57].

The table predicts what a Monte-Carlo simulation will do with each potential, and the solution folder provides the corresponding inputs (`sro-2nnmeam.lmp`, `sro-sharifi.lmp`, and the 1300 K reference run `sro-hot.lmp`; the elevated temperature speeds up the chemical equilibration, and a smaller 500-atom cell keeps even the MEAM runs to a few minutes). The outcome is shown in Fig. 39:

- The 2NN-MEAM of Wang *et al.* [58] assigns *positive* mixing enthalpies to Cr-Fe and Co-Cr. Chromium is therefore expelled from the solid solution altogether: instead of short-range order the simulation produces a compact Cr precipitate ($\alpha_{\text{CrCr}} \approx -0.3$, i.e. strong Cr-Cr clustering).
- The MEAM of Sharifi and Wick [59] over-binds *every* unlike pair, most extremely Co-Fe (-277 versus ≈ -10 meV per atom). The alloy partitions into (Co,Fe)-rich and (Ni,Cr)-rich regions ($\alpha_{\text{CoNi}} \approx +0.2$, with all four diagonal elements negative) — a different artifact, driven by over-binding rather than expulsion.
- The Farkas-Caro EAM [55] was deliberately constructed to give near-zero mixing enthalpies, and it is the only

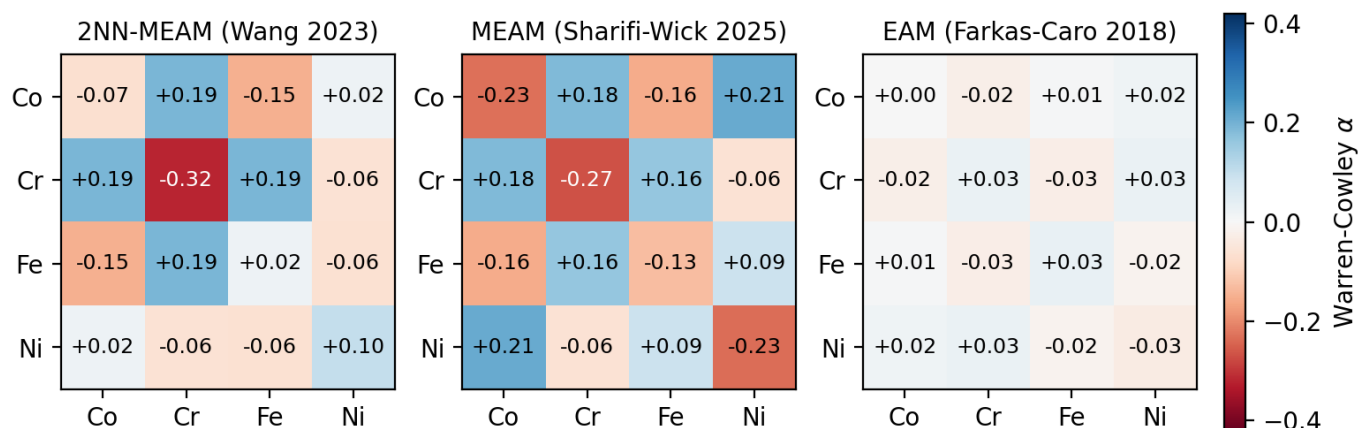


Figure 39. The same simulation, three different answers: first-shell Warren-Cowley matrices from identical hybrid MD/MC runs of a 500-atom equimolar CoCrFeNi cell at 1300 K (Tutorial 14; red = attraction, $\alpha < 0$; blue = avoidance, $\alpha > 0$; generated with `solution/compare-alpha.py`). Left: the 2NN-MEAM of Wang *et al.* [58] expels chromium from the solid solution ($\alpha_{\text{CrCr}} = -0.32$, a Cr precipitate). Center: the MEAM of Sharifi and Wick [59] demixes the alloy into (Co,Fe)-rich and (Ni,Cr)-rich domains: the unlike pairs *within* each domain attract, all pairs *across* the domain boundary avoid each other, and every diagonal element is negative because each domain is enriched in its two host elements. Right: the EAM of Farkas and Caro [55] maintains the solid solution observed experimentally, with only weak short-range order. Each failure is predicted by the mixing enthalpies in Table 2.

one of the three that keeps the solid solution stable and reproduces the experimentally and theoretically expected SRO motif (Cr–Cr avoidance), albeit weaker than DFT predicts.

None of the three potentials is “broken”: each reproduces the properties it was fitted to. The 2NN-MEAM family targets phase stability across a huge composition space for alloy design, and the Sharifi–Wick potential was explicitly fitted to elastic constants, dislocation, and defect properties — its documentation even states that it was not optimized for thermodynamic data. The lesson is not that one potential is good and two are bad, but that *a potential is only valid for the property class it was fitted and tested for*, and that the inexpensive test of Table 2 would have flagged both problems before any Monte-Carlo simulation was run. Repositories such as the NIST Interatomic Potentials Repository distribute many curated potentials and document their intended use, but the responsibility for validating a potential against the target property always remains with the user [60, 61]. The Farkas–Caro and Sharifi–Wick potential files used here were obtained from the NIST repository [62]; the 2NN-MEAM is bundled with LAMMPS. Where no classical potential passes the test, machine-learning interatomic potentials fitted to large DFT databases are the modern alternative (Appendix E).

Author Contributions

S.G. and A.K. developed the companion LiveCoMS tutorial article whose style, template, and LAMMPS-GUI-centered

A practical pitfall worth knowing: for multi-element MEAM parameter files the element *order matters*. The pair indices in the parameter file refer to the position of each element in the list given in the `pair_coeff` command, so that list must match the order the file was created for (for the file used here, `FeMnNiTiCuCrCoAl.meam`, it is the order of the file name); only the type-mapping arguments at the end may be changed. A wrong list does not produce an error — it silently assigns all pair parameters to the wrong element pairs. When in doubt, recompute a documented property of a small subsystem (Table 1 shows the kind of data to compare) before production use.

workflow are adopted here. M.T. created the original MSU/CAVS tutorials [9, 10] (Tutorials 1–4). A.K. adapted the MSU/CAVS tutorials for the format of the first LiveCoMS tutorial collection, developed Tutorials 5–14, prepared all LAMMPS inputs, analysis scripts, and figures, and integrated them with LAMMPS-GUI. All authors reviewed and edited the manuscript.

Acknowledgements

(none)

FIXME: Who should we acknowledge?

Potentially Conflicting Interests

There are no conflicting interests to declare.

Funding Information

S.G. acknowledges funding from the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement N° 101065060. A.K. acknowledges financial support by Sandia National Laboratories under POs 2149742 and 2407526.

Author Information

ORCID:

Simon Gravelle: [0000-0003-2149-6706](https://orcid.org/0000-0003-2149-6706)

Mark A. Tschopp: [0000-0001-8471-5035](https://orcid.org/0000-0001-8471-5035)

Jacob R. Gissinger: [0000-0003-0031-044X](https://orcid.org/0000-0003-0031-044X)

Axel Kohlmeier: [0000-0001-6204-6475](https://orcid.org/0000-0001-6204-6475)

References

- [1] **Thompson AP**, Aktulga HM, Berger R, Bolintineanu DS, Brown WM, Crozier PS, in 't Veld PJ, Kohlmeier A, Moore SG, Nguyen TD, Shan R, Stevens MJ, Tranchida J, Trott C, Plimpton SJ. LAMMPS -- a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*. 2022; 271:108171. <https://doi.org/10.1016/j.cpc.2021.108171>.
- [2] **Plimpton S**. Fast Parallel Algorithms for Short-Range Molecular Dynamics. *Journal of Computational Physics*. 1995; 117(1):1--19. <https://doi.org/10.1006/jcph.1995.1039>.
- [3] **LAMMPS Developers**, LAMMPS Molecular Dynamics Simulator;. Accessed: 2026. <https://www.lammps.org>.
- [4] **Zhang YW**, Sorkin V, Aitken ZH, Politano A, Behler J, Thompson AP, Ko TW, Ong SP, Chalykh O, Korogod D, Podryabinkin E, Shapeev A, Li J, Mishin Y, Pei Z, Liu X, Kim J, Park Y, Hwang S, Han S, et al. Roadmap for the development of machine learning-based interatomic potentials. *Modelling and Simulation in Materials Science and Engineering*. 2025; 33(2):023301. <https://doi.org/10.1088/1361-651X/ad9d63>.
- [5] **Jacobs R**, Morgan D, Attarian S, Meng J, Shen C, Wu Z, Xie CY, Yang JH, Artrith N, Blaiszik B, Ceder G, Choudhary K, Csányi G, Cubuk ED, Deng B, Drautz R, Fu X, Godwin J, Honavar V, Isayev O, et al. A practical guide to machine learning interatomic potentials -- Status and future. *Current Opinion in Solid State and Materials Science*. 2025; 35:101214. <https://doi.org/10.1016/j.cossms.2025.101214>.
- [6] **Gravelle S**, Alvares CMS, Gissinger JR, Kohlmeier A. A Set of Tutorials for the LAMMPS Simulation Package. *Living Journal of Computational Molecular Science*. 2025; 6(1):3037. <https://doi.org/10.33011/livecoms.6.1.3037>.
- [7] **Axel Kohlmeier**, LAMMPS-GUI Online Documentation;. Accessed: 2026. <https://lammps-gui.lammps.org/>.
- [8] **LAMMPS Developers**, LAMMPS Visualization Howto;. Accessed: 2026. https://docs.lammps.org/Howto_visualize.html.
- [9] **Tschopp MA**, MSU / CAVS LAMMPS Tutorials;. Accessed: 2026. https://www.cavs.msstate.edu/icme/code/lammps/material_models/tutorials.php.
- [10] **Tschopp MA**, LAMMPS Tutorials for Beginners;. Accessed: 2026. <https://github.com/mrkllntschpp/lammps-tutorials>.
- [11] **LAMMPS Developers**, LAMMPS Releases on GitHub;. Accessed: 2026. <https://github.com/lammps/lammps/releases/>.
- [12] **Daw MS**, Baskes MI. Embedded-atom method: Derivation and application to impurities, surfaces, and other defects in metals. *Physical Review B*. 1984; 29(12):6443--6453. <https://doi.org/10.1103/PhysRevB.29.6443>.
- [13] **Harris CR**, Millman KJ, van der Walt SJ, et al. Array programming with NumPy. *Nature*. 2020; 585:357--362. <https://doi.org/10.1038/s41586-020-2649-2>.
- [14] **Hunter JD**. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*. 2007; 9(3):90--95. <https://doi.org/10.1109/MCSE.2007.55>.
- [15] **McKinney W**. Data Structures for Statistical Computing in Python. In: *Proceedings of the 9th Python in Science Conference*; 2010. p. 56--61. <https://doi.org/10.25080/Majora-92bf1922-00a>.
- [16] **Virtanen P**, Gommers R, Oliphant TE, et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*. 2020; 17:261--272. <https://doi.org/10.1038/s41592-019-0686-2>.
- [17] **PyYAML contributors**, PyYAML: YAML parser and emitter for Python; 2006. <https://pyyaml.org/>.
- [18] **Stukowski A**. Visualization and analysis of atomistic simulation data with OVITO -- the Open Visualization Tool. *Modelling and Simulation in Materials Science and Engineering*. 2010; 18(1):015012. <https://doi.org/10.1088/0965-0393/18/1/015012>.
- [19] **Arbblaster JW**. Selected Values of the Crystallographic Properties of Elements. ASM International; 2018.
- [20] **Zhou XW**, Wadley HNG, Johnson RA, Larson DJ, et al. Atomic scale structure of sputtered metal multilayers. *Acta Materialia*. 2001; 49(19):4005--4015. [https://doi.org/10.1016/S1359-6454\(01\)00287-7](https://doi.org/10.1016/S1359-6454(01)00287-7).
- [21] **LAMMPS Developers**, LAMMPS dump image Command Documentation;. Accessed: 2026. https://docs.lammps.org/dump_image.html.
- [22] **Kittel C**. Introduction to Solid State Physics. 8 ed. Hoboken, NJ: John Wiley & Sons; 2005.
- [23] **Shewchuk JR**. An Introduction to the Conjugate Gradient Method Without the Agonizing Pain. Carnegie Mellon University; 1994.
- [24] **Jelinek B**, Groh S, Horstemeyer MF, Houze J, Kim SG, Wagner GJ, Moitra A, Baskes MI. Modified embedded atom method potential for Al, Si, Mg, Cu, and Fe alloys. *Physical Review B*. 2012; 85(24):245102. <https://doi.org/10.1103/PhysRevB.85.245102>.
- [25] **LAMMPS Developers**, LAMMPS Howto: Output Structured Data from LAMMPS;. Accessed: 2026. https://docs.lammps.org/Howto_structured_data.html.

- [26] **Birch F.** Finite Elastic Strain of Cubic Crystals. *Physical Review*. 1947; 71(11):809–824. <https://doi.org/10.1103/PhysRev.71.809>.
- [27] **Simmons G**, Wang H. *Single Crystal Elastic Constants and Calculated Aggregate Properties: A Handbook*. 2 ed. Cambridge, MA: MIT Press; 1971.
- [28] **Tschopp MA**, McDowell DL. Influence of single crystal orientation on homogeneous dislocation nucleation under uniaxial loading. *Journal of the Mechanics and Physics of Solids*. 2008; 56(5):1806–1830. <https://doi.org/10.1016/j.jmps.2007.11.012>.
- [29] **Kelchner CL**, Plimpton SJ, Hamilton JC. Dislocation nucleation and defect structure during surface indentation. *Physical Review B*. 1998; 58(17):11085–11088. <https://doi.org/10.1103/PhysRevB.58.11085>.
- [30] **Wikipedia contributors**, Ordered dithering;. Accessed: 2026. https://en.wikipedia.org/wiki/Ordered_dithering.
- [31] **Apostol F**, Mishin Y. Interatomic potential for the Al-Cu system. *Physical Review B*. 2011; 83(5):054116. <https://doi.org/10.1103/PhysRevB.83.054116>.
- [32] **Tschopp MA**, McDowell DL. Asymmetric tilt grain boundary structure and energy in copper and aluminium. *Philosophical Magazine*. 2007; 87(22):3147–3173. <https://doi.org/10.1080/14786430701255895>.
- [33] **Mendelev MI**, Han S, Srolovitz DJ, Ackland GJ, Sun DY, Asta M. Development of new interatomic potentials appropriate for crystalline and liquid iron. *Philosophical Magazine*. 2003; 83(35):3977–3994. <https://doi.org/10.1080/14786430310001613264>.
- [34] **Humphrey W**, Dalke A, Schulten K. VMD -- Visual Molecular Dynamics. *Journal of Molecular Graphics*. 1996; 14(1):33–38. [https://doi.org/10.1016/0263-7855\(96\)00018-5](https://doi.org/10.1016/0263-7855(96)00018-5).
- [35] **Tersoff J**. Modeling solid-state chemistry: Interatomic potentials for multicomponent systems. *Physical Review B*. 1989; 39(8):5566–5568. <https://doi.org/10.1103/PhysRevB.39.5566>.
- [36] **Stillinger FH**, Weber TA. Computer simulation of local order in condensed phases of silicon. *Physical Review B*. 1985; 31(8):5262–5271. <https://doi.org/10.1103/PhysRevB.31.5262>.
- [37] **Clavier G**, Desbiens N, Bourasseau E, Lachet V, Brusselle-Dupend N, Rousseau B. Computation of elastic constants of solids using molecular simulation: comparison of constant volume and constant pressure ensemble methods. *Molecular Simulation*. 2017; 43(17):1413–1422. <https://doi.org/10.1080/08927022.2017.1313418>.
- [38] **Vitek V**. Intrinsic stacking faults in body-centred cubic crystals. *Philosophical Magazine*. 1968; 18(154):773–786. <https://doi.org/10.1080/14786436808227500>.
- [39] **Hirth JP**, Lothe J. *Theory of Dislocations*. 2nd ed. New York: Wiley; 1982.
- [40] **Mishin Y**, Mehl MJ, Papaconstantopoulos DA, Voter AF, Kress JD. Structural stability and lattice defects in copper: Ab initio, tight-binding, and embedded-atom calculations. *Physical Review B*. 2001; 63(22):224106. <https://doi.org/10.1103/PhysRevB.63.224106>.
- [41] **Müller-Plathe F**. A simple nonequilibrium molecular dynamics method for calculating the thermal conductivity. *Journal of Chemical Physics*. 1997; 106(14):6082–6085. <https://doi.org/10.1063/1.473271>.
- [42] **Schelling PK**, Phillpot SR, Keblinski P. Comparison of atomic-level simulation methods for computing thermal conductivity. *Physical Review B*. 2002; 65(14):144306. <https://doi.org/10.1103/PhysRevB.65.144306>.
- [43] **Kong LT**. Phonon dispersion measured directly from molecular dynamics simulations. *Computer Physics Communications*. 2011; 182(10):2201–2207. <https://doi.org/10.1016/j.cpc.2011.04.019>.
- [44] **Svensson EC**, Brockhouse BN, Rowe JM. Crystal Dynamics of Copper. *Physical Review*. 1967; 155(3):619–632. <https://doi.org/10.1103/PhysRev.155.619>.
- [45] **Dickey JM**, Paskin A. Computer simulation of the lattice dynamics of solids. *Physical Review*. 1969; 188(3):1407–1418. <https://doi.org/10.1103/PhysRev.188.1407>.
- [46] **Nordlund K**, et al. Improving atomic displacement and replacement calculations with physically realistic damage models. *Nature Communications*. 2018; 9:1084. <https://doi.org/10.1038/s41467-018-03415-5>.
- [47] **Hammond KD**. Parallel point defect identification in molecular dynamics simulations without post-processing: A compute and dump style for LAMMPS. *Computer Physics Communications*. 2020; 247:106862. <https://doi.org/10.1016/j.cpc.2019.106862>.
- [48] **Norgett MJ**, Robinson MT, Torrens IM. A proposed method of calculating displacement dose rates. *Nuclear Engineering and Design*. 1975; 33(1):50–54. [https://doi.org/10.1016/0029-5493\(75\)90035-7](https://doi.org/10.1016/0029-5493(75)90035-7).
- [49] **Mendelev MI**, Kramer MJ, Becker CA, Asta M. Analysis of semi-empirical interatomic potentials appropriate for simulation of crystalline and liquid Al and Cu. *Philosophical Magazine*. 2008; 88(12):1723–1750. <https://doi.org/10.1080/14786430802206482>.
- [50] **Morris JR**, Wang CZ, Ho KM, Chan CT. Melting line of aluminum from simulations of coexisting phases. *Physical Review B*. 1994; 49(5):3109–3115. <https://doi.org/10.1103/PhysRevB.49.3109>.
- [51] **Cantor B**, Chang ITH, Knight P, Vincent AJB. Microstructural development in equiatomic multicomponent alloys. *Materials Science and Engineering: A*. 2004; 375–377:213–218. <https://doi.org/10.1016/j.msea.2003.10.257>.
- [52] **Zhang R**, Zhao S, Ding J, Chong Y, Jia T, Ophus C, Asta M, Ritchie RO, Minor AM. Short-range order and its impact on the CrCoNi medium-entropy alloy. *Nature*. 2020; 581:283–287. <https://doi.org/10.1038/s41586-020-2275-z>.
- [53] **Cowley JM**. An approximate theory of order in alloys. *Physical Review*. 1950; 77(5):669–675. <https://doi.org/10.1103/PhysRev.77.669>.
- [54] **Sadigh B**, Erhart P, Stukowski A, Caro A, Martinez E, Zepeda-Ruiz L. Scalable parallel Monte Carlo algorithm for atomistic simulations of precipitation in alloys. *Physical Review B*. 2012; 85(18):184203. <https://doi.org/10.1103/PhysRevB.85.184203>.

- [55] **Farkas D**, Caro A. Model interatomic potentials and lattice strain in a high-entropy alloy. *Journal of Materials Research*. 2018; 33(19):3218–3225. <https://doi.org/10.1557/jmr.2018.245>.
- [56] **Tamm A**, Aabloo A, Klintonberg M, Stocks M, Caro A. Atomic-scale properties of Ni-based FCC ternary, and quaternary alloys. *Acta Materialia*. 2015; 99:307–312. <https://doi.org/10.1016/j.actamat.2015.08.015>.
- [57] **Takeuchi A**, Inoue A. Classification of bulk metallic glasses by atomic size difference, heat of mixing and period of constituent elements and its application to characterization of the main alloying element. *Materials Transactions*. 2005; 46(12):2817–2829. <https://doi.org/10.2320/matertrans.46.2817>.
- [58] **Wang J**, Kwon H, Kim HS, Lee BJ. A neural network model for high entropy alloy design. *npj Computational Materials*. 2023; 9:60. <https://doi.org/10.1038/s41524-023-01010-x>.
- [59] **Sharifi H**, Wick CD. Developing interatomic potentials for complex concentrated alloys of Cu, Ti, Ni, Cr, Co, Al, Fe, and Mn. *Computational Materials Science*. 2025; 248:113595. <https://doi.org/10.1016/j.commatsci.2024.113595>.
- [60] **Becker CA**, Tavazza F, Trautt ZT, Buarque de Macedo RA. Considerations for choosing and using force fields and interatomic potentials in materials science and engineering. *Current Opinion in Solid State and Materials Science*. 2013; 17(6):277–283. <https://doi.org/10.1016/j.cossms.2013.10.001>.
- [61] **Hale LM**, Trautt ZT, Becker CA. Evaluating variability with atomistic simulations: the effect of potential and calculation methodology on the modeling of lattice and elastic constants. *Modelling and Simulation in Materials Science and Engineering*. 2018; 26(5):055003. <https://doi.org/10.1088/1361-651X/aabc05>.
- [62] **NIST Interatomic Potentials Repository**, NIST Interatomic Potentials Repository; <https://doi.org/10.18434/m37>, maintained by the NIST Materials Science and Engineering Division. <https://www.ctcms.nist.gov/potentials>.
- [63] **Henkelman G**, Uberuaga BP, Jónsson H. A climbing image nudged elastic band method for finding saddle points and minimum energy paths. *The Journal of Chemical Physics*. 2000; 113(22):9901–9904. <https://doi.org/10.1063/1.1329672>.
- [64] **LAMMPS Developers**, LAMMPS Online Documentation; Accessed: 2026. <https://docs.lammps.org>.
- [65] **Elliott RS**, Akerson A, Efficient “Universal” Shifted Lennard-Jones Model for All KIM API Supported Species (aluminum parameterization). OpenKIM; 2015. OpenKIM interatomic model; NIST Interatomic Potentials Repository, <https://www.ctcms.nist.gov/potentials/entry/2015--Elliott-R-S-Akerson-A-AI/>.
- [66] **Girifalco LA**, Weaver VG. Application of the Morse Potential Function to Cubic Metals. *Phys Rev*. 1959; 114(3):687–690. <https://doi.org/10.1103/PhysRev.114.687>.
- [67] **Thompson AP**, Swiler LP, Trott CR, Foiles SM, Tucker GJ. Spectral neighbor analysis method for automated generation of quantum-accurate interatomic potentials. *Journal of Computational Physics*. 2015; 285:316–330. <https://doi.org/10.1016/j.jcp.2014.12.018>.
- [68] **Zuo Y**, Chen C, Li X, Deng Z, Chen Y, Behler J, Csányi G, Shapeev AV, Thompson AP, Wood MA, Ong SP. Performance and Cost Assessment of Machine Learning Interatomic Potentials. *The Journal of Physical Chemistry A*. 2020; 124(4):731–745. <https://doi.org/10.1021/acs.jpca.9b08723>.
- [69] **Honeycutt JD**, Andersen HC. Molecular dynamics study of melting and freezing of small Lennard-Jones clusters. *The Journal of Physical Chemistry*. 1987; 91(19):4950–4963. <https://doi.org/10.1021/j100303a014>.
- [70] **Stukowski A**. Structure identification methods for atomistic simulations of crystalline materials. *Modelling and Simulation in Materials Science and Engineering*. 2012; 20(4):045021. <https://doi.org/10.1088/0965-0393/20/4/045021>.
- [71] **Larsen PM**, Schmidt S, Schiøtz J. Robust structural identification via polyhedral template matching. *Modelling and Simulation in Materials Science and Engineering*. 2016; 24(5):055007. <https://doi.org/10.1088/0965-0393/24/5/055007>.

A Using LAMMPS–GUI

Installation instructions and a full description of LAMMPS–GUI are given in the companion article [6] and the LAMMPS–GUI documentation [7]; only the features used here are summarized. An input file is opened and edited in the «Editor» window and run with «Run LAMMPS»; while it runs, a progress bar in the status bar and a marker in the editor margin show how far it has advanced and which line is executing. Index variables such as `mode` or `temp` can be set without editing the file through the «Set Variables» dialog. Snapshots are inspected in the «Image Viewer» (the snapshot icon in the editor status bar), and thermodynamic output and other tabular data are plotted in the «Charts» window. LAMMPS–GUI runs LAMMPS as a library in a single process, so it accelerates runs with threads (and, where available, GPUs) rather than MPI; these are selected in «Preferences» under «Accelerators».

Three further features, added in LAMMPS–GUI v2.1, make it possible to carry out the analysis steps of these tutorials without leaving the editor or installing external tools; they are handy for quick checks while an input is still being developed, even though the provided Python scripts remain the reference workflow for the final figures.

Plotting data files. Besides the live «Charts» window that follows a run, «Plot Data File...» in the «File» menu (Ctrl-Shift-P) opens a column-oriented data file in a standalone chart. Whitespace-separated text, CSV, YAML, and JSON are all recognized, so the structured files written by the tutorial inputs — for example `ev.yaml` in Tutorial 2 or the stress-strain CSV files of Tutorial 3 — can be plotted directly; a column picker selects the x axis and the curves to show.

Fitting data. The «Postprocess...» entry of a chart’s «File» menu analyzes the displayed data in place. Alongside

an autocorrelation analysis and a polynomial fit, it offers a Birch–Murnaghan equation-of-state fit — which returns the equilibrium lattice constant, energy, bulk modulus, and its pressure derivative directly from an energy–volume curve, reproducing the Python fit of Tutorial 2 — and a general non-linear «Custom fit» of a user-supplied expression (a Levenberg–Marquardt fit using the same Lepton expression syntax as the LAMMPS Lepton-based styles).

Viewing rendered images. «View Image File(s) ...» in the «File» menu loads one or more existing image files into the same slide-show viewer used for live `dump image` output. This is convenient for reviewing snapshots that LAMMPS rendered in a separate or parallel run — such as the cascade images of Tutorial 11 — without rerunning the simulation.

B Running LAMMPS on the command line

Every tutorial input can also be run with the command-line LAMMPS executable, which is convenient for longer runs, batch parameter scans, and high-performance computing clusters. A basic run is

```
lmp -in deform.lmp
```

Index variables are overridden with `-var`, for example `lmp -in deform.lmp -var mode compress`, which makes it easy to script a scan over parameter values from a shell loop. Shared-memory threading is enabled, as in the «Accelerators» settings of LAMMPS–GUI, by building with the OPENMP package and adding the suffix flag, e.g.

```
OMP_NUM_THREADS=8 lmp -in deform.lmp -sf omp
```

For the largest production calculations, the command-line executable additionally supports distributed-memory parallelism with MPI (`mpirun -np N lmp ...`), optionally combined with threads or GPUs, as discussed in the note in Tutorial 3.

C Multi-partition runs: the nudged elastic band and beyond

Every tutorial in this set runs as a single LAMMPS calculation, which is what LAMMPS–GUI executes. A few important methods instead couple several LAMMPS calculations running *simultaneously* as independent *partitions* (also called replicas or universes). These are launched with the `-partition` command-line switch (Appendix B): for instance,

```
mpirun -np 12 lmp -partition 12x1 -in in.neb
```

splits twelve MPI ranks into twelve partitions of one rank each. Because the partitions exchange data, such runs require MPI and the command-line executable; they cannot

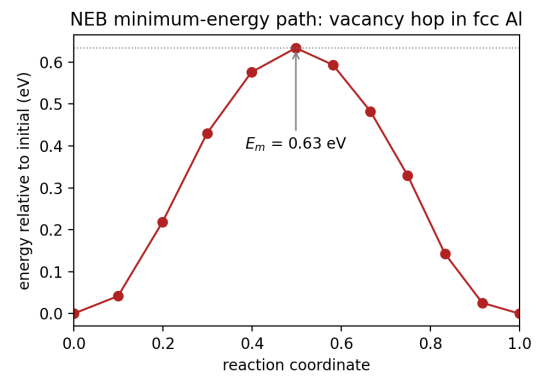


Figure 40. Minimum-energy path for a vacancy hop in fcc aluminum, computed with the nudged elastic band (NEB) method over twelve images (markers). The barrier at the saddle, $E_m \approx 0.63$ eV, is the activation energy for vacancy migration in the solid — the crystalline counterpart of the liquid-diffusion activation energy of Tutorial 13.

be run inside LAMMPS–GUI, which hosts a single LAMMPS instance. This appendix sketches the most broadly useful of them, the nudged elastic band, and points to the others.

The nudged elastic band

The *nudged elastic band* (NEB) [63] finds the minimum-energy path between two stable states, and in particular the energy barrier that separates them. A chain of replicas (“images”) is interpolated between an initial and a final configuration and joined by springs; every image relaxes toward the path of steepest descent while the springs keep the images spread along it. The highest image locates the saddle point, and a *climbing-image* variant drives it exactly onto the barrier top.

The input `in.neb` computes the barrier for a *vacancy hop* in fcc aluminum: a vacancy is created near the center of the cell and relaxed (the initial state), and a neighboring atom is driven into the vacant site (the final state, given by `final.neb`). The same Mendelev potential as Tutorial 13 is used, so the result connects directly to the liquid-diffusion measurement there. The essential commands are

```
fix 1 all neb 1.0
min_style quickmin
neb 0.0 1.0e-4 2000 2000 100 final final.neb
```

Run over twelve images, this gives the symmetric path of Fig. 40 with a migration barrier $E_m \approx 0.63$ eV, close to the experimental aluminum value of about 0.61 eV. This is precisely the activation energy that governs vacancy diffusion in the *solid* — the rare, thermally activated hop that makes crystalline diffusion too slow to follow directly, which is why Tutorial 13 measures diffusion in the liquid instead.

A multi-partition run needs some care with output, because every partition writes its own files. LAMMPS provides

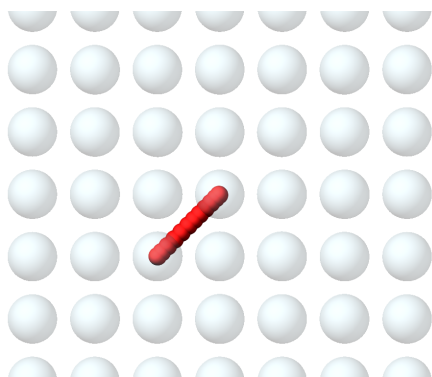


Figure 41. The NEB band visualized with `fix graphics/replica`: each red sphere is the migrating atom in one of the twelve replicas, so the overlapping chain traces its path from an occupied site into the neighboring vacant site through the aluminum crystal. The whole picture is rendered on a single partition with the `partition yes 1` prefix.

the `partition` command to send commands to chosen partitions only; combined with `fix graphics/replica` (GRAPHICS package) it gives a compact way to picture the whole band. `fix graphics/replica` collects a chosen atom's position from *every* replica, and the `partition yes 1` prefix runs a single `dump image` on just the first partition:

```
fix sf1 migrant graphics/replica 100 display 0.6
partition yes 1 dump viz all image 100 neb-path.*.png &
type type fix sf1 const 0 0
```

The result (Fig. 41) overlays the migrating atom from all twelve images on one rendering of the lattice, so the entire migration path appears in a single picture.

Other multi-partition methods

NEB is one of several partition-based methods in LAMMPS, all launched the same way from the command line:

- *Parallel tempering / replica exchange* (`temper`) runs many replicas at staggered temperatures and periodically swaps them, greatly improving the sampling of rough energy landscapes.
- *Parallel-replica* (`prd`) and *temperature-accelerated* (`tad`) dynamics accelerate the simulation of rare events such as the diffusive hops above. (The related `fix hyper` hyperdynamics reaches a similar speed-up within a *single* partition, so it does run in LAMMPS-GUI.)
- *Path-integral molecular dynamics* (`fix pimd`) represents each atom by a ring of replicas to capture quantum nuclear effects, which matter for light atoms and for heat capacities at low temperature — a quantum complement to the classical phonons of Tutorial 9 and Tutorial 10.

The LAMMPS replica how-to [64] and the bundled `examples/neb`, `examples/prd`, and `examples/tad` directories provide complete, runnable starting points.

D Pairwise versus many-body potentials

Given how strongly every result in this collection depends on the choice of interatomic potential, it is fair to ask whether the many-body models are really necessary, or whether one of the simple pairwise potentials that are occasionally applied to metals would do.

This appendix uses three properties that the tutorials already compute – crystal-structure stability (Tutorials 3.1 and 3.2), the elastic constants (Tutorial 3.6), and the vacancy formation energy (Tutorial 3.11) – to show that a pure pairwise description of a metal is not merely less accurate but *qualitatively* wrong, and that the embedded-atom family repairs each failure.

We compare five potentials for each of aluminum and iron: two parameterizations of the Lennard-Jones (LJ) form (pair style `lj/cut`), a Morse potential (pair style `morse`), the EAM potentials used in the tutorials — `Al_zhou.eam.alloy` [20] and `Fe_mm.eam.fs` [33] — and the MEAM potential of Jelinek *et al.* (`AlSiMgCuFe.meam`) [24]. The two LJ parameterizations bracket what the pairwise form can do. The *tuned* LJ uses σ and ϵ chosen so that the relaxed fcc crystal reproduces the experimental lattice constant and cohesive energy — the pairwise form at its best. The *universal* LJ is the ready-made Elliott-Akerson “universal” model distributed through OpenKIM [65], whose parameters are fixed entirely by the isolated dimer (the potential minimum at the sum of covalent radii, the well depth at the bond dissociation energy) with no reference to the solid; it is a shifted LJ, reproduced here exactly by `lj/cut` with `pair_modify shift yes`. The Morse parameters are those of Girifalco and Weaver [66], fitted to the cohesive energy, lattice constant, and bulk modulus of each metal. Every calculation here is a static energy minimization that finishes in seconds. The reproduction inputs are in [appendix/pair-vs-manybody](#).

D.1 Crystal structure

To determine the predicted crystal structure, we build each metal in the fcc, bcc, and hcp structures, relax every cell to its own equilibrium, and compare the cohesive energies; the lowest is the predicted ground state (“GS” in Table 3). All three pairwise parameterizations always favor a *close-packed* structure: fcc and hcp come out within a few meV of one another (too close to separate meaningfully), while bcc lies far *above* them. For aluminum, whose ground state is fcc, this happens to give a reasonable answer; for iron, whose

Table 3. Lattice stability of aluminum and iron. Δ_{bcc} and Δ_{hcp} are the energies of the bcc and hcp structures relative to fcc (meV/atom; positive means above fcc); “GS” is the predicted ground-state structure. Each pairwise potential favors close packing and places bcc well above it, predicting the wrong ground state for iron; the embedded-atom models reproduce fcc aluminum and bcc iron.

Potential	Aluminum			Iron		
	Δ_{bcc}	Δ_{hcp}	GS	Δ_{bcc}	Δ_{hcp}	GS
LJ, tuned	+121	-5	hcp	+191	-3	hcp
LJ, universal	+985	-6	hcp	+432	-3	hcp
Morse	+15	+1	fcc	+26	-2	hcp
EAM	+57	+9	fcc	-120	-3	bcc
MEAM	+148	+30	fcc	-8	-6	bcc
Experiment	fcc			bcc		

ground state is bcc, it is simply wrong. The Morse result is the most telling: Girifalco and Weaver fitted their potential [66] to bcc iron, yet bcc is still unstable against close-packing, because a simple pairwise interaction like Lennard-Jones or Morse always favors a close-packed arrangement — stabilizing bcc requires the many-body, density-dependent character of metallic bonding. The embedded-atom models, whose embedding term supplies exactly that, place bcc lowest for iron and fcc lowest for aluminum. This is why the bcc-iron tutorials (Tutorials 3.4 and 3.11) use an EAM potential: a pairwise model would not even hold the lattice. Comparing the two LJ rows shows that within a single form the parameters still matter enormously: the off-the-shelf universal model, fixed by the dimer alone, overbinds fcc aluminum to -22.5 eV per atom — nearly seven times the experimental -3.36 eV — and contracts its lattice constant to 3.33 Å, while, being the same pairwise form, it still favors close packing and still misorders iron.

D.2 Elastic anisotropy and the Cauchy relation

For a crystal in which every atom sits at a center of inversion and interacts through central pairwise forces, the elastic con-

Table 4. Elastic constants of fcc aluminum (GPa) at zero temperature. Central pairwise potentials obey the Cauchy relation $C_{12} = C_{44}$ exactly; the real metal and the embedded-atom models do not.

Potential	C_{11}	C_{12}	C_{44}	C_{12}/C_{44}
LJ, tuned	349	198	198	1.00
LJ, universal	4462	2546	2546	1.00
Morse	95	67	67	1.00
EAM	127	81	36	2.23
MEAM	111	61	28	2.14
Experiment [22]	108	61	28	2.2

stants obey the Cauchy relations; for a cubic crystal under no external stress this reduces to $C_{12} = C_{44}$. Table 4 confirms it: both LJ parameterizations and the Morse potential satisfy $C_{12} = C_{44}$ exactly, whereas aluminum and the embedded-atom models violate it strongly, $C_{12} \approx 2C_{44}$. The difference is again the embedding energy, whose dependence on the local electron density adds a non-central contribution to the stiffness. A pairwise potential therefore cannot match C_{12} and C_{44} simultaneously, no matter how it is fitted.

Note also that the LJ constants are far too stiff overall: reproducing the cohesive energy with a pair well over-stiffens the crystal.

These numbers were obtained with the same `in.elastic` driver input used in Tutorial 3.6.

D.3 Vacancy formation

Table 5. Cohesive energy, vacancy formation energy, and their ratio for fcc aluminum. A pairwise potential makes a vacancy cost nearly the whole cohesive energy; the embedded-atom models and experiment give a ratio near 0.2.

Potential	E_{coh} (eV)	E_{vac} (eV)	$E_{\text{vac}}/ E_{\text{coh}} $
LJ, tuned	-3.25	3.20	0.98
LJ, universal	-22.50	22.15	0.98
Morse	-2.92	2.64	0.90
EAM	-3.59	0.71	0.20
MEAM	-3.36	0.66	0.20
Experiment [22]	-3.36	0.67	0.20

For a pairwise potential, removing an atom costs nearly the full cohesive energy: the bonds to the missing atom are broken, the remaining atoms barely respond, and $E_{\text{vac}}/|E_{\text{coh}}| \approx 1$ (Table 5). In a real metal the electron density redistributes and the neighbors of the empty site re-embed, recovering most of that energy, so the ratio is only about 0.2. The embedded-atom models capture this — their vacancy energies of 0.71 eV and 0.66 eV bracket the experimental 0.67 eV — whereas the tuned LJ and Morse overestimate it four- to fivefold, and the grossly overbound universal LJ thirtyfold. The same calculation is the basis of the defect counting in Tutorial 3.11.

Together with Appendix E, these results place the potentials on a ladder. A pairwise potential is the cheapest but is qualitatively wrong for a metal — wrong ground-state structure, wrong elastic symmetry, and wrong defect energetics. The embedded-atom family (EAM and MEAM) repairs all three at modest cost and is the workhorse of these tutorials. An MLIP (Appendix E) reaches near-quantum accuracy on

static properties at far higher cost, though even it can miss properties outside its training data. The recurring lesson of these tutorials applies at every rung: validate a potential against the specific properties a study depends on.

E Machine-learning interatomic versus classical empirical potentials

The tutorials in this set use classical empirical interatomic potentials—EAM, MEAM, Stillinger–Weber, or Tersoff—whose fixed functional form is fitted to a handful of properties. *Machine-learning interatomic potentials* instead fit a flexible regression model to a large database of quantum-mechanical (DFT) energies and forces, trading a much higher evaluation cost for near-DFT accuracy across many properties at once. Until a dedicated tutorial on *building* an MLIP is available, this appendix simply quantifies that trade-off, reusing the very properties the tutorials already compute. We use the Spectral Neighbor Analysis Potential (`pair_style snap`) [67], the MLIP bundled in the LAMMPS-GUI tutorial binaries; the pre-fitted silicon and tantalum models come from the benchmark of Zuo *et al.* [68]. (LAMMPS ships other MLIP styles, notably `pair_style pod` and the `mliap` interface, but their bundled parameter sets are demonstration fits not intended for production accuracy.)

This is a results-only appendix; the inputs are provided as [downloads](#) so the numbers can be reproduced. The lattice constant, cohesive energy, and the elastic constants C_{11} , C_{12} , C_{44} (and the Voigt–Reuss–Hill bulk modulus B , shear modulus G , and Poisson ratio ν) are obtained exactly as in Tutorial 6; the vacancy-formation energy E_{vac} from a single relaxed defect cell; and the cost as the wall-clock time per atom per timestep of a serial molecular-dynamics run. Table 6 collects the results.

For silicon, no single classical potential is best at everything: MEAM reproduces C_{11} and C_{44} well but overestimates C_{12} ; Stillinger–Weber gets the lattice constant and bulk modulus but is too soft in shear; and the original 1988 Tersoff parameterization (LAMMPS’ `Si.tersoff`) collapses to $C_{44} = 10$ GPa. SNAP is never far off on any property, but measured against *experiment* it does not beat the best classical fit for silicon—because it faithfully reproduces its PBE-DFT training set, and PBE overestimates the silicon lattice constant (≈ 5.47 Å) and underestimates its stiffness. The same MLIP applied to tantalum, a metal for which PBE is accurate, reproduces the experimental elastic constants almost exactly. The cost column makes the trade-off concrete: SNAP is roughly 280 times more expensive per atom-step than Stillinger–Weber, and an order of magnitude above MEAM, yet that is still many orders of magnitude cheaper than evaluating the forces with DFT directly. The cohesive

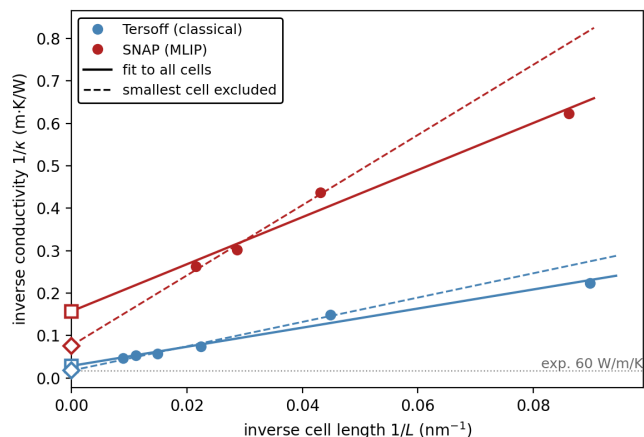


Figure 42. Finite-size scaling of the reverse-NEMD thermal conductivity of germanium at 300 K: inverse apparent conductivity $1/\kappa$ versus inverse cell length $1/L$ for the classical Tersoff and the machine-learning SNAP potential (`appendix/ge-kappa/plot_ge.py`). Each data set is fitted twice, to expose the sensitivity of the extrapolation: all cells with equal weights (solid lines, open squares at $1/L \rightarrow 0$) and with the smallest cell excluded (dashed lines, open diamonds). Tersoff extrapolates to $35 \text{ W m}^{-1} \text{ K}^{-1}$ to $58 \text{ W m}^{-1} \text{ K}^{-1}$ depending on that choice, compared with the experimental $60 \text{ W m}^{-1} \text{ K}^{-1}$ (dotted); SNAP to $6 \text{ W m}^{-1} \text{ K}^{-1}$ to $13 \text{ W m}^{-1} \text{ K}^{-1}$, a factor of five to ten too low either way.

energy is left blank for SNAP because an MLIP’s energy zero is set by its DFT reference rather than the isolated atom, so it is not comparable to the classical or experimental values; the vacancy-formation energy, an energy *difference* in which that reference cancels, is reported for all.

E.1 Thermal conductivity: a transport property

The quantities in Table 6 are static. A more searching test of a potential is a finite-temperature *transport* coefficient, which depends on the full anharmonic phonon spectrum rather than only the curvature of the energy surface at equilibrium. As a representative case we compute the lattice thermal conductivity of germanium—one of the few elements with both a bundled classical (Tersoff) and machine-learning (SNAP) potential—by the same reverse non-equilibrium method as Tutorial 8. As there, the apparent conductivity grows with the cell length L because the phonon mean free path is not fully contained in the cell, so the bulk value is recovered by extrapolating $1/\kappa$ versus $1/L$ to $1/L \rightarrow 0$.

⚠ These cells are much larger, and the runs much longer, than the silicon tutorial, and each SNAP force evaluation is about two orders of magnitude more costly than Tersoff (Table 6), so the calculation is provided as a self-contained, MPI-parallel [HPC input set](#) rather than an interactive script.

Figure 42 shows the outcome. With the Tersoff potential

Potential	a_0 (Å)	E_{coh} (eV)	E_{vac} (eV)	C_{11}	C_{12}	C_{44}	B	G	ν	cost ($\mu\text{s/at.}$)
<i>Silicon (diamond cubic)</i>										
SW	5.431	-4.34	4.33	151	76	56	101	48	0.30	0.4
Tersoff 1988	5.431	-4.64	2.80	122	86	10	98	13	0.44	0.9
MEAM	5.431	-4.62	3.82	165	82	72	110	58	0.28	4.3
SNAP	5.474	---	2.68	128	75	71	93	48	0.28	124
Experiment [22]	5.431	-4.63	~ 3.6	166	64	80	98	67	0.22	---
<i>Tantalum (bcc)</i>										
SNAP	3.316	---	2.70	270	156	74	194	66	0.35	39
Experiment [22]	3.303	-8.10	~ 2.9	266	158	87	194	72	0.34	---

Table 6. Properties of silicon (diamond cubic) and tantalum (bcc) computed with classical and machine-learning interatomic potentials, compared with experiment. Elastic moduli are in GPa; the cost is the serial wall-clock time per atom per timestep. The classical silicon potentials are accurate for some properties and poor for others (Stillinger–Weber is soft in C_{44} , the 1988 Tersoff parameterization badly so), whereas SNAP is uniformly reasonable. Because SNAP reproduces its DFT (PBE) training data, its silicon lattice constant and stiffness follow PBE—which itself overestimates a_0 and softens silicon relative to experiment—while for the metal tantalum, where PBE is accurate, SNAP matches the measured constants closely. SNAP costs one to two orders of magnitude more than the classical forms, but still far less than a direct DFT calculation.

the apparent conductivity climbs from 4.5 to 21 W m⁻¹ K⁻¹ as the cell grows from 11 to 111 nm, with no sign of saturation even in the longest cell. The SNAP potential's apparent conductivity is far lower at every length (1.6 W m⁻¹ K⁻¹ to 3.8 W m⁻¹ K⁻¹ between 12 and 46 nm), and this is not a statistical artifact: at comparable cell length and heat flux the SNAP temperature gradient is three times steeper than the Tersoff one; the model scatters phonons far too strongly.

Extrapolating such data to the bulk deserves a caution of its own. No cell is longer than the effective phonon mean free path, so every point lies short of the asymptotic regime in which $1/\kappa$ is strictly linear in $1/L$, and the intercept then depends strongly on which cells enter the fit and with what weight. The smallest cell is the worst offender on three counts at once: it sits farthest out on the $1/L$ axis, where it has the greatest leverage on a least-squares line; it deviates most from the asymptotic law; and its ordinate is the least reliable, since an uncertainty in κ enters $1/\kappa$ with a factor $1/\kappa^2$ —equal weights therefore systematically over-weight the smallest, least bulk-like cells. Figure 42 accordingly shows two fits per potential: all cells with equal weights (solid) and the smallest cell excluded (dashed). The Tersoff extrapolation moves from 35 to 58 W m⁻¹ K⁻¹ between the two, with reliability-weighted fits falling in between, although every variant describes its points with $R^2 \geq 0.97$: goodness of fit does not discriminate between bulk values that differ by nearly a factor of two. Honestly quoted, the classical result is 35 W m⁻¹ K⁻¹ to 58 W m⁻¹ K⁻¹—the magnitude of the experimental 60 W m⁻¹ K⁻¹, but not the pinpoint agreement the dashed fit alone would suggest. The same exercise for SNAP spans 6 W m⁻¹ K⁻¹ to 13 W m⁻¹ K⁻¹: the conclusion that this potential underpredicts κ by a factor of five to ten

survives every fitting choice, precisely because the failure is so large.

That result sharpens the message of Table 6. The same SNAP model that is among the best in the table for static properties fails by close to an order of magnitude on a transport coefficient, because nothing in its training data—energies, forces, and stresses of moderately distorted configurations—constrains the anharmonic phonon–phonon scattering that limits κ ; the flexibility that lets an MLIP reproduce its training set so closely provides no physics to fall back on outside of it. The fixed-form Tersoff potential, although also fitted only to static properties, inherits realistic anharmonicity from its physically motivated functional form. Neither outcome could be guessed from the static table alone, which is precisely the point—as in the alloy comparison of Tutorial 14, a potential is only trustworthy for the properties, and the region of phase space, against which it has been validated. The cost asymmetry completes the picture: the six-length Tersoff scan took about 36 core-hours, the four shorter SNAP lengths about 1800.

F Structure-analysis tools

Most tutorials in this article answer their scientific question with a *per-atom structure analysis*: a compute that classifies each atom from the geometry of its neighborhood, so that defects, phases, or interfaces can be counted and visualized. These analyses are spread over the tutorials where they first appear; this appendix collects them in one place (Table 7) and adds practical guidance on choosing between them.

The tools fall into three families. *Symmetry detectors* (compute centro/atom) measure how far an atom's neigh-

Table 7. Per-atom structure analyses used in (or relevant to) this article. All are computes that produce per-atom values, usable for visualization (`dump image`), counting (`compute reduce`), or per-region statistics (`fix ave/chunk`).

Compute	Identifies	Used in	Caveats
<code>centro/atom</code>	broken inversion symmetry: defects in fcc/bcc lattices	T3, T4, T5	threshold choice; centrosymmetric lattices only
<code>cna/atom</code>	local crystal structure (fcc, hcp, bcc, icosahedral)	T7, T12	fixed cutoff; degrades with temperature (Fig. 43)
<code>ptm/atom</code>	as CNA, plus simple cubic, diamond, graphene; orientation and strain	this appendix	slightly costlier than CNA
<code>orientorder/atom</code>	Steinhardt bond-order parameters ($Q_4, Q_6, \dots$): crystallinity	—	thresholds are system-specific
<code>voronoi/atom</code>	atomic volume and coordination	—	volume fluctuations are broad
<code>cluster/atom</code>	connected clusters within a cutoff	—	needs a defect/selection criterion
<code>frenkel</code>	vacancies and interstitials by Wigner–Seitz site occupancy	T11	needs the reference lattice
<code>displace/atom</code>	displacement from a stored reference configuration	T5	reference must be stored first
<code>rdf</code>	coordination-shell fingerprint, phase identification	this appendix	cannot separate hot solid from liquid near T_m
<code>xrd</code> , <code>saed</code>	virtual powder/electron diffraction patterns (DIFFRACTION package)	—	reciprocal-space mesh can be memory-hungry

borhood deviates from a perfect reference symmetry; they are simple and fast and were used to expose dislocations and surfaces in Tutorials 3–5, but they only flag *that* something is broken, not what the local structure is. *Structure classifiers* (`compute cna/atom` [69, 70], `compute ptm/atom` , `compute orientorder/atom`) assign each atom a lattice type, which Tutorial 7 used to pick out the hcp-coordinated stacking fault and Tutorial 12 used to count the crystalline fraction of a melting cell. *Site-based analyses* (`compute voronoi/atom` , `compute cluster/atom` , `compute frenkel`) partition space rather than classify geometry; Tutorial 11 used the Wigner–Seitz variant to count irradiation-induced vacancies and interstitials live during a collision cascade. Finally, the DIFFRACTION package computes virtual x-ray and electron-diffraction patterns from a snapshot — the most direct bridge from a simulation to an experimental observable.

Before any per-atom classification, the oldest structural measure of all deserves a place in the toolkit: the radial distribution function $g(r)$ (`compute rdf`), the histogram of interatomic distances normalized to an ideal gas. It requires no reference lattice, no cutoff tuning, and no assumption about what structure to expect — and it is directly related to what a diffraction experiment measures. The inputs `rdf-al.lmp`, `rdf-fe.lmp`, `rdf-mg.lmp`, and `rdf-si.lmp` compute $g(r)$ for the four common lattices — fcc aluminum, bcc iron, hcp magnesium, and diamond-cubic silicon — at three temperatures each, the highest chosen above the superheating limit so that the crystal melts during the run. Figure 44 collects the

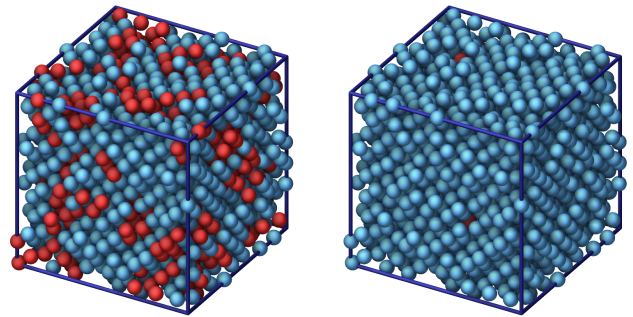


Figure 43. The same perfect aluminum crystal, thermally vibrating at 800 K, classified by common neighbor analysis (left) and polyhedral template matching (right); atoms recognized as fcc are blue, unclassified atoms red (`appendix/structure/hotsolid.lmp`). CNA classifies only 70 % of the atoms correctly because thermal displacements push neighbors across its fixed cutoff; PTM, which fits structural templates with an adjustable tolerance, still recognizes 99.6 %.

twelve curves (`rdf-plot.py`). Each lattice has its own fingerprint of coordination shells: the bcc second shell sits close behind the first, the open diamond structure has only four nearest neighbors and a wide gap beyond them, and fcc and hcp — identical in their first shell — differ in the shell structure beyond. Heating broadens every shell without moving it much, and on melting the fingerprint collapses to the universal liquid form: one broad nearest-neighbor peak, a weak second oscillation, and $g \rightarrow 1$ beyond — short-range order persists in the liquid, long-range order is gone. This is why $g(r)$ alone cannot tell a hot solid from a liquid near the melting point (the Tutorial 12 problem again), but it remains the

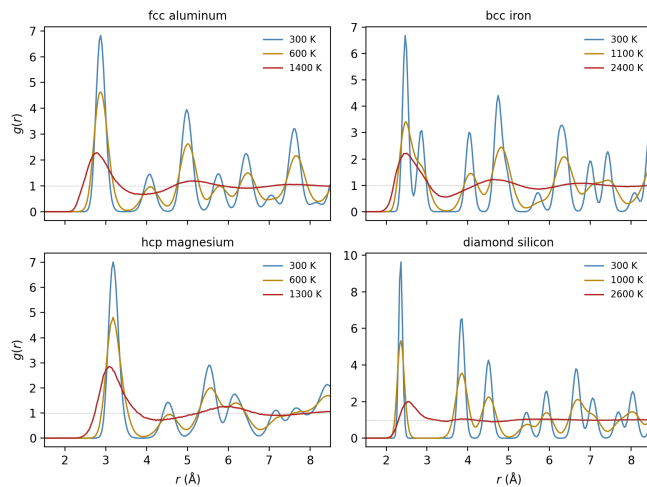


Figure 44. Radial distribution functions of the four common crystal lattices at three temperatures each (appendix/structure/rdf-*.lmp). Every lattice has a distinct fingerprint of coordination shells; thermal motion broadens the shells, and above the melting point (red curves) only the broad nearest-neighbor peak of the liquid survives, with $g(r) \rightarrow 1$ at large distance.

quickest qualitative answer to “what phase am I looking at?”.

A practical caveat that affects every cutoff-based method is illustrated in Fig. 43 with the input `hotsolid.lmp`: at 800 K common neighbor analysis misclassifies a third of a *perfect* aluminum crystal, simply because thermal vibrations move neighbors across the fixed cutoff that separates the first and second neighbor shell. Polyhedral template matching [71] (`compute ptm/atom`, PTM package) instead matches the neighborhood against ideal structure templates and accepts a match below an adjustable root-mean-square deviation, which makes it far more robust on hot solids — and it returns the local lattice orientation and elastic strain as by-products. Two practical notes: first, when CNA must be used at temperature, compute its cutoff from the *current* (thermally expanded) lattice constant rather than the zero-kelvin value, as `hotsolid.lmp` does; second, robustness against thermal vibration does not remove the harder problem of Tutorial 12 — near the melting point the instantaneous neighborhoods of hot solid and hot *liquid* genuinely overlap for every classifier, and quenching to the inherent structure before the analysis remains the reliable approach there.